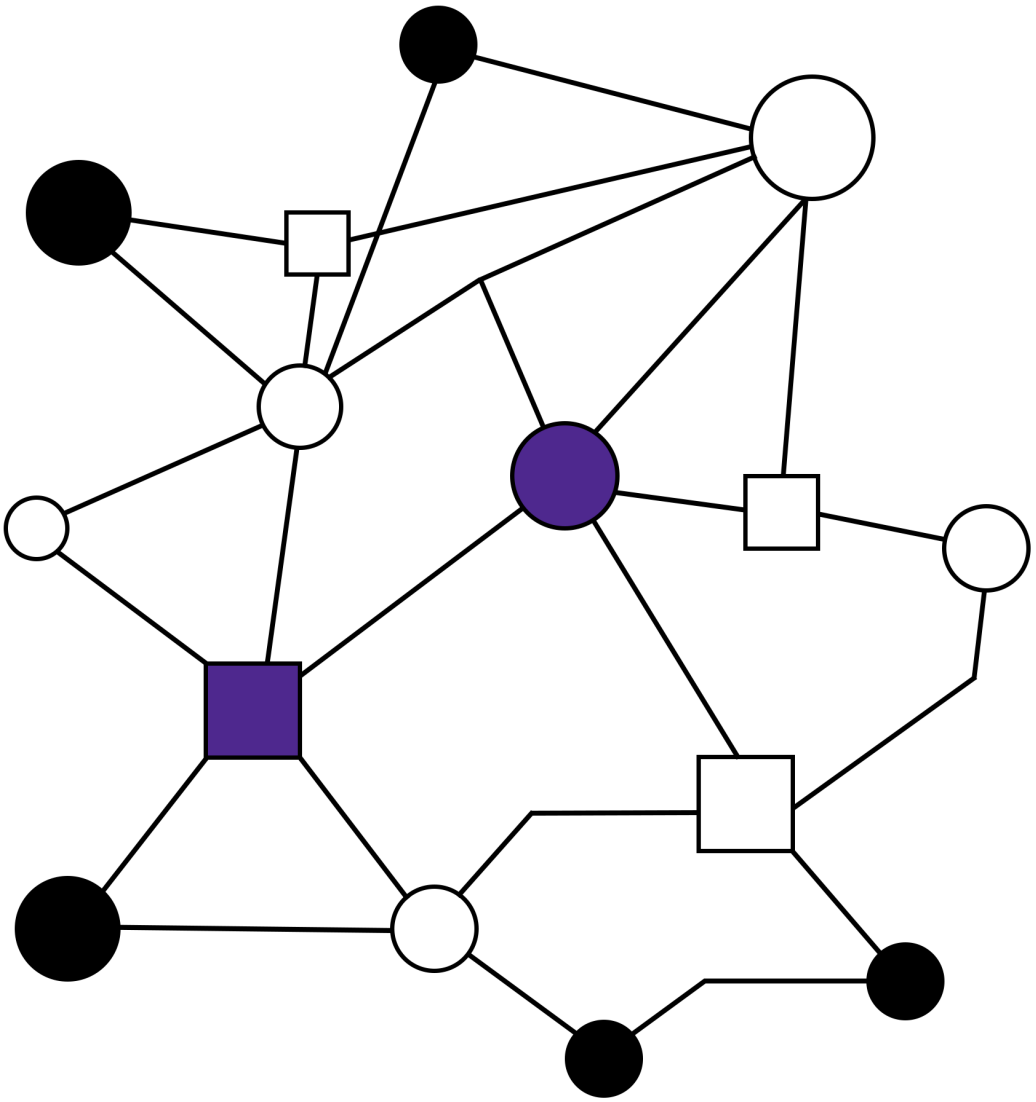
Reti Informatiche



Davide Squeri

2025/2026

Computer Networking Index

1 Internet Protocol Stack

- 1 Introduction
- 2 The Internet protocol Stack
- 3 ISO/OSI reference model

2 Network Applications

- 1 Introduction
  - 1.1 Network Application Architectures (2.1.1)
  - 1.2 Process Communicating (2.1.2)
  - 1.3 Transport Services for Applications (2.1.3)
  - 1.4 Application-layer protocols (2.1.5)
- 2 Client-Server Applications
  - 2.1 Web and HTTP (2.2)
  - 2.2 E-mail, SMTP, IMAP (2.3)
  - 2.3 The Domain Name System (DNS) (2.4)
- 3 Peer-to-Peer File Distribution (2.5)
  - 3.1 Content Indexes
  - 3.2 How much time does it take to distribute one file to N peers?
  - 3.3 BitTorrent
- 4 Video streaming and content distribution networks (2.6)
  - 4.1 Coding
  - 4.2 Challenges of streaming
  - 4.3 Compression
  - 4.4 HTTP streaming (2.6.2)
  - 4.5 Content Distribution Networks (CDNs) (2.6.3)
- 5 Sockets (2.7)
  - 5.1 Blocking vs Non-Blocking
  - 5.2 TCP vs UDP
  - 5.3 Sending Data
  - 5.4 Closing a connection
  - 5.5 Unix Socket primitives
  - 5.6 I/O Multiplexing

3 Transport Layer

- 1 Introduction
- 2 Multiplexing and Demultiplexing (3.2)
- 3 Connectionless transport: UDP (3.3)
  - 3.1 UDP segment header
  - 3.2 UDP checksum (3.3.2)
- 4 Reliable Data Transfer protocols (3.4)
  - 4.1 Stop-and-Go protocols
  - 4.2 Pipelining
- 5 Connection-Oriented transport: TCP (3.5)
  - 5.1 TCP segment structure (3.5.2)
  - 5.2 Sequence Numbers and ACKs
  - 5.3 Round-Trip Time Estimation and Timeout (3.5.3)
  - 5.4 TCP reliable data transfer (3.5.4)
  - 5.5 TCP flow control (3.5.5)
  - 5.6 TCP connection Management (3.5.6)
  - 5.7 Congestion Control (3.6)
  - 5.8 TCP Congestion Control (3.7)
- 6 Evolution of transport-layer functionality
  - 6.1 QUIC: Quick UDP Internet Connections

4 Network Layer: Data Plane

- 1 Introduction
- 2 Network Layer: overview (4.1)
  - 2.1 Network service Model (4.1.2)
- 3 Routers (4.2)
  - 3.1 Forwarding
  - 3.2 Switching Fabrics
  - 3.3 Queueing
  - 3.4 Packet Scheduling
- 4 IP: the internet protocol (4.3)
  - 4.1 IPv4 Datagram format (4.3.1)
  - 4.2 Fragmentation (4.3.2)
  - 4.3 IPv4 Addressing (4.3.3)
  - 4.4 Network Address Translation (4.3.4)
  - 4.5 IPv6 (4.3.5)
  - 4.6 Transition from IPv4 to IPv6
  - 4.7 ICMP protocol
- 5 Generalized Forwarding and SDN (4.4)
  - 5.1 OpenFlow
- 6 Middlebox
- 7 Where's the intelligence

5 Network Layer: Control Plane

- 1 Introduction
- 2 Routing Algorithms (5.2)
  - 2.1 Routing protocols classification
  - 2.2 Link state Algorithms
  - 2.3 Distance Vector Algorithm
- 3 Autonomous Systems (5.3)
  - 3.1 Intra-AS routing: OSPF
  - 3.2 Inter-AS routing: BGP (5.4)

6 Link Layer

- 1 Introduction
- 2 Error-Detection and Correction (EDC) Techniques
  - 2.1 Parity Checking
  - 2.2 Checksumming (6.2.2)
  - 2.3 Cyclic Redundancy Check (CRC) (6.2.3)
- 3 Multiple Access Links and Protocols (6.3)
  - 3.1 MAC protocols taxonomy
- 4 LANs
  - 4.1 MAC addresses
  - 4.2 Ethernet
  - 4.3 Physical topology for LANs
  - 4.4 Ethernet frame structure
  - 4.5 Ethernet CSMA/CD protocol
- 5 Link Layer Switches
  - 5.1 Ethernet Switch
- 6 Virtual LANs (VLANs)
- 7 Link Virtualization (6.5)
  - 7.1 MPLS
- 8 Wide-Area Packet Switched Networks (WANs)
  - 8.1 Connection Oriented Packet Switched WANs
- 9 Datacenter Networks

7 Wireless and Mobile Networks

- 1 Introduction
  - 1.1 Characteristics of different wireless links
- 2 Wireless Links and network characteristics (7.2)

    - 2.1 WiFi: 802.11 Wireless LANs (7.3)
    - 2.2 Wireless LANs: LiFi: Light Fidelity
    - 2.3 Wireless PANs: Bluetooth (7.3.6)
    - 2.4 Cellular networks: 4G and 5G (7.4)
- 3 Mobility Management (7.5)
    - 3.1 Mobility management
    - 3.2 Mobility's impact on higher-layer protocols (7.7)

8 Network Security

- 1 Introduction
- 2 Principles of Cryptography (8.2)
    - 2.1 Breaking encryption
    - 2.2 Symmetric Key cryptography (8.2.1)
    - 2.3 Public Key Cryptography (8.2.2)
- 3 Message integrity (8.3)
    - 3.1 Message digests
    - 3.2 Digital signatures (8.3.3)
- 4 Authentication (8.4)
- 5 Securing e-mail (8.5.1)
- 6 Securing TCP connections: TLS (8.6)
    - 6.1 The almost-TLS protocol
    - 6.2 The actual TLS protocol (8.6.2)
- 7 Network Layer Security: IPsec (8.7)
    - 7.1 Virtual Private Networks (VPNs)
    - 7.2 SAs: Security Associations (SAs) (8.7.3)
    - 7.3 IPsec protocols (8.7.2)
    - 7.4 IPsec sequence Numbers
- 8 Operational security: firewalls and Intrusion detection systems (8.9)
    - 8.1 Firewalls (8.9.1)
    - 8.2 Intrusion detection systems

Internet Protocol Stack

#uni

- 1 Introduction
- 2 The Internet protocol Stack
- 3 ISO/OSI reference model

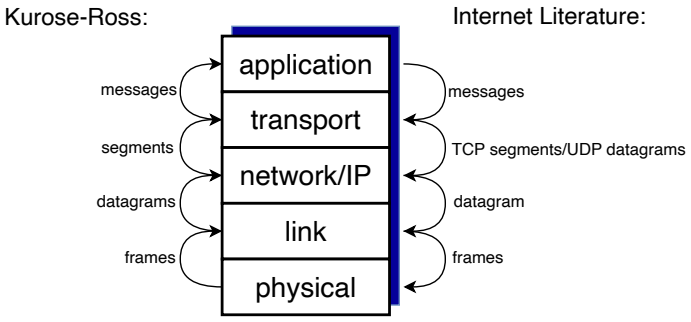
1 Introduction

Protocol Layering offers many advantages: it provides a structured way to discuss system components and enables modularity, making it easier to update system components.

Some researchers are opposed to layering for many reasons, for example one layer may duplicate lower-layer functionality, or one layer may need information held only in another layer, which violates the goal of separation of layers.

Protocol Stack: the set of protocols of the various layers.

2 The Internet protocol Stack

> the Internet Protocol Stack:
>
> 
>

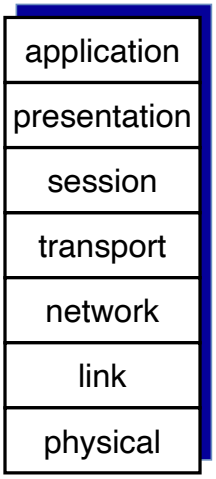
The **Internet Protocol Stack** consists of 5 layers:

- **application layer**: where the Network Applications (http, imap, smtp) operate. Packets of information at this layer are called **messages**.
- **transport layer**: transports application-layer messages between application endpoints: process to process data transfer (TCP: reliable, ordered delivery and connection oriented; UDP: connectionless). This layer's packets are called **segments**.
- **network layer**: this layer routes **datagrams** through a series of routers between the source and the destination using the IP address (IP is the protocol that forwards datagrams; the routing protocols, such as OSPF and BGP, also belong to this layer).
- **link layer** (or data layer, or data link): this layer transfers **frames** between neighboring network elements using the MAC address (Ethernet, 802.11 (Wi-Fi), PPP).
- **physical layer**: this layer moves the individual **bits** within the frame across the physical medium of a single link.

3 ISO/OSI reference model

This model adds two layers (the **session** and **presentation** layers) that are not found in the Internet protocol stack: if they are needed, they must be implemented in the application layer.

> iso/osi reference model:


application
presentation
session
transport
network
link
physical

Network Applications

#uni

- 1 Introduction
  - 1.1 Network Application Architectures (2.1.1)
    - 1.1.1 Client-Server paradigm
    - 1.1.2 Peer-to-Peer paradigm
  - 1.2 Process Communicating (2.1.2)
    - 1.2.1 Socket
    - 1.2.2 Addressing Processes
  - 1.3 Transport Services for Applications (2.1.3)
    - 1.3.2 Securing TCP
  - 1.4 Application-layer protocols (2.1.5)
- 2 Client-Server Applications
  - 2.1 Web and HTTP (2.2)
    - 2.1.1 HTTP messages
    - 2.1.2 Status Codes
    - 2.1.3 TELNET
    - 2.1.4 Cookies (2.2.4)
    - 2.1.5 Web Caches (proxy servers) (2.2.5)
    - 2.1.6 Important updates
  - 2.2 E-mail, SMTP, IMAP (2.3)
    - 2.2.2 Mail message format
    - 2.2.3 Mail access protocols (2.3.4)
  - 2.3 The Domain Name System (DNS) (2.4)
    - 2.3.1 DNS services (2.4.1)
    - 2.3.2 ICANN
    - 2.3.3 DNS structure
    - 2.3.4 DNS records (2.4.3)
    - 2.3.5 DNS protocol messages (2.4.3)
    - 2.3.6 Name resolution Approaches
    - 2.3.7 Inserting Names into the DNS
    - 2.3.8 Example of an (iterative) DNS resolution
    - 2.3.9 DNS security
- 3 Peer-to-Peer File Distribution (2.5)
  - 3.1 Content Indexes
    - 3.1.1 Query Flooding
    - 3.1.2 Hierarchical Overlay
  - 3.2 How much time does it take to distribute one file to N peers?
  - 3.3 BitTorrent
    - 3.3.1 Entering a Torrent
- 4 Video streaming and content distribution networks (2.6)
  - 4.1 Video basics
    - 4.1.1 Coding
  - 4.2 Challenges of streaming
  - 4.3 Compression
  - 4.4 HTTP streaming (2.6.2)
    - 4.4.1 Dynamic Adaptive Streaming over HTTP (DASH) (2.6.2)
  - 4.5 Content Distribution Networks (CDNs) (2.6.3)
    - 4.5.1 CDN modus operandi
    - 4.5.2 Cluster Selection Strategies
    - 4.5.3 Netflix Case Study
- 5 Sockets (2.7)
  - 5.1 Blocking vs Non-Blocking
  - 5.2 TCP vs UDP
  - 5.3 Sending Data
    - 5.3.1 Endianness
    - 5.3.2 Message delimitation

    - 5.3.3 Partial sends and reads
    - 5.3.4 Structs and serialization
    - 5.3.5 Size, timing and errors
    - 5.3.6 Other protocol design considerations
- 5.4 Closing a connection
- 5.5 Unix Socket primitives
- 5.6 I/O Multiplexing

1 Introduction

Creating a network app means writing a program that runs on different end systems and communicates over a network.
There is no need to write software for devices in the network core: these devices do not run user applications. Routers don't need to implement the whole Internet Protocol Stack.
Running applications only on systems on the edge of the Internet allows for rapid app development and propagation.
Internet is transparent, meaning the two (or more) processes communicating do not worry about how the messages are exchanged and act like they are directly connected.

1.1 Network Application Architectures (2.1.1)

The two most important application paradigms are:

- Client-Server: web, http, e-mail, smtp, imap, dns
- Peer to peer: bitTorrent

1.1.1 Client-Server paradigm

A server is an always-on host with a permanent IP address (Data Plane > 4 IP: the internet protocol (4.3)), often found in data centers, for scaling advantages.

Clients contact and communicate with the server intermittently, they may have dynamic IP addresses and *do not* communicate directly with each other.

1.1.2 Peer-to-Peer paradigm

Every peer looks more like a powerful client from the Client-Server paradigm.
Technically every peer is on the same level and there isn't an always-on server: end systems arbitrarily communicate with each other.
The availability of resources this way is not guaranteed by a powerful server, but by the sheer number of peers $\implies$ ***self scalability***.

> Peers request service from other peers, providing service to other peers in return.
> **Indexes** are needed for mapping information to the right host locations.

Common challenges this paradigm faces are related to security, performance and reliability, due to its highly decentralized architecture.

There exist also mixed architectures, where both the peer-to-peer and client-server paradigms are used together (some messaging apps use the server to find the IP address of the destination host and then use it to directly communicate with the destination host).

1.2 Process Communicating (2.1.2)

A process is a program running within a host.
Within a host two processes communicate using *inter-process communication*, which is offered and ruled over by the operating system.
Processes in different hosts instead communicate by exchanging **messages** on the network (obviously two processes on the same host can still communicate using messages if needed).

> In the client-server paradigm the ***client process*** (often just called *the client*) is the process that initiates the communication, the ***server process*** (often just called *the server*) is the process that waits to be contacted.
> P2P applications have both a client process and a server process!

1.2.1 Socket

A process sends and receives messages to/from its **socket(s)**.
Sending a message means leaving it in the socket, *trusting* the transport layer infrastructure will bring it to the receiving process' socket; there are in fact two sockets involved in every message exchange, one on each side.

The application developer has little control over the transport-layer side: they can choose the transport protocol and fix a few transport-layer parameters (maximum buffer and maximum segment size for example).

1.2.2 Addressing Processes

To receive messages a process must have an *identifier*. The host device has a unique 32-bit IP address (or multiple unique IPs) used to identify it in the internet, but this is not enough since a single host can run multiple processes concurrently, therefore a process is identified

using *both* the host's IP address and a **port number** associated with said process on its host.

This means different processes on the same host have the same IP address but different ports.

1.3 Transport Services for Applications (2.1.3)

When designing a network application we must choose the correct transport-layer service for our needs: some apps require a 100% reliable data transfer, some are loss tolerant, some apps require a minimum amount of throughput to be effective, some don't, some apps need security and the list goes on.

The internet makes 2 transport layer protocols available to applications: **UDP** and **TCP**, in short:

- TCP services:
  - reliable data transfer (RDT)
  - flow control
  - congestion control
  - connection-oriented
- UDP services:
  - unreliable data transfer
  - connectionless

Examples

Here is a table with different application-layer protocol and its used transport-layer protocol:

application	application-layer protocol	transport-layer protocol
file transfer	FTP	TCP
e-mail	SMTP	TCP
web documents	HTTP 1.1	TCP
internet telephony	SIP, RTP, or proprietary	TCP or UDP
streaming audio/video	HTTP, DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

And here is a table with different services some common applications need:

application	data loss tolerance	minimum throughput	time sensitivity
file transfer	no	no	no
e-mail	no	no	no
web documents	no	no	no
real-time audio/video	yes	audio: 5Kbps-1Mbps video: 10Kbps-5Mbps	10's msec
interactive games	yes	Kbps+	10's msec
text messaging	no	no	depends

1.3.2 Securing TCP

Normally TCP and UDP sockets do not offer encryption: **Transport layer security** (**TLS**) must be implemented in network applications.

The internet community first developed the **Secure Sockets Layer** (**SSL**), the precursor of **Transport layer security** (**TLS**): SSL is not a third transport layer protocol, but an enhancement for TCP that lives at the application layer: developers who wish to utilize it must include SSL code (SSL libraries are highly optimized and available for use) inside both the client and the server's side of their application.

SSL has its own socket API, similar to the traditional TCP socket API: when an application uses SSL the sending process passes cleartext data to the SSL socket, which encrypts it and sends it to the normal TCP socket. The encrypted data travels to the TCP socket in the receiving host, the TCP socket passes the encrypted data to the SSL socket, which decrypts it and finally sends the cleartext data to the destination process. See Network Security > 6 Securing TCP connections TLS (8.6) for more details on TLS.

1.4 Application-layer protocols (2.1.5)

An application-layer protocol defines:

- the types of messages exchanged

- the message syntax: what fields are in the message and how these fields are delineated
- the message semantics: what the different fields inside a message mean
- rules for when and how processes should send and respond to messages

Protocols can be:

- **open protocols**: defined by RFCs (requests for comments) and allow for interoperability.
- **proprietary protocols**

2 Client-Server Applications

2.1 Web and HTTP (2.2)

Web pages consist of **web objects**, each of which can be stored on different web servers. An object can be an HTML file, an image, a Java applet, etc.
A web page consists of a base HTML file, which includes several referenced objects, each addressable by a URL.

HTTP stands for HyperText Transfer Protocol, it is based on the Client-Server paradigm and it defines how web clients request web pages from web servers and how servers transfer web pages to clients.

HTTP uses TCP:

1. the client initiates a TCP connection to the server, port 80 is the default for HTTP
2. the server accepts the TCP connection from the client
3. now HTTP messages (which are application-layer protocol messages) are exchanged between the client (the browser) and the web server
4. the TCP connection is finally closed

HTTP is a *stateless* protocol: the server maintains no information about past client requests.
Stateful protocols are complex: past history must be maintained, if the server or the client crashes their views of the "state" may be inconsistent and must be reconciled.

There are two types of HTTP connections:

- *non-persistent HTTP*:
  At most one object is sent over the TCP connection between the client and the server, after which the connection is closed. For n objects requested sequentially:

  $$\text{response time for } n \text{ objects} = n \cdot (2 \cdot \text{RTT} + \text{file transmission time})$$

- *persistent HTTP* (default):
  Multiple objects can be sent over a single TCP connection, the connection is arbitrarily closed by either the client or the server. With pipelining, the client can request all referenced objects as soon as it parses the base HTML:

  $$\text{response time} \approx 3 \cdot \text{RTT} + \sum \text{file transmission times}$$

  (1 RTT to open TCP connection, 1 RTT to request/receive base HTML, and ~1 RTT to request/receive pipelined referenced objects).
  Without pipelining, each referenced object costs an additional RTT.

Where **RTT** (**round trip time**) is the time it takes for a small packet to travel from the client to the server and back.
Generally speaking RTT is much larger than a single file transmission time, thus leading to MUCH longer response times for non-persistent HTTP connections when dealing with multiple objects.
For example when transferring two small objects, *the response time is cut in half* using persistent HTTP.
When using non-persistent connections, by default most browsers open 5 to 10 parallel TCP connections, and each one of these connections handles one request-response transaction, still, persistent connections are preferred.

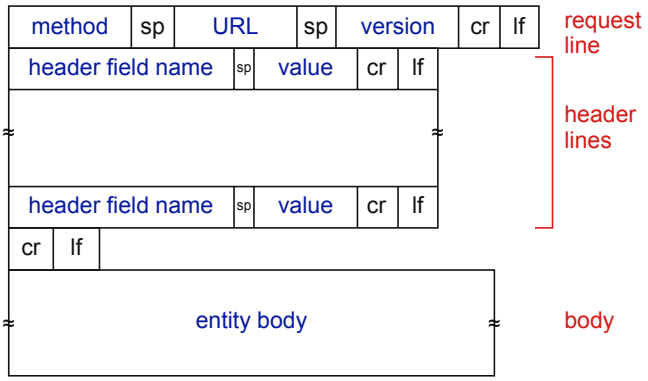
After a timeout the connection is closed automatically: TCP connections consume memory: leaving them open leaves the data structures allocated, wasting memory.

2.1.1 HTTP messages

There are two types of HTTP messages:

- *request*
- *response*
  The message format is **ASCII**.

HTTP request messages

> HTTP request message general format:
>
> 
>

There are multiple possible request messages:

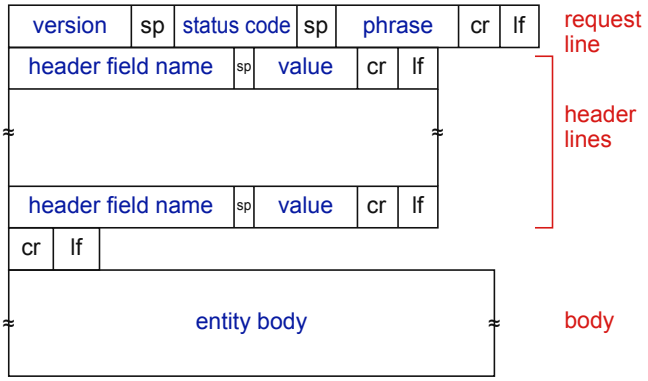
- **POST** method: web pages often include form inputs, the user's input is sent from the client to the server *in the body* of the HTTP POST request message.
- **GET** method: this is often used (apart from asking for web objects) to send data to a server: in HTML forms the user's data is included in the URL field of a HTTP GET request message, following a '?' (`www.site.com/subdomain?userdata1&userdata2`).
- **HEAD** method: this is used to request only the headers, without the requested objects, it is most used during debugging.
- **PUT** method: this uploads a new object to the server, completely replacing the file that exists at the specified URL with the content in the body of the PUT HTTP request message.
- **DELETE** method: it allows a user – or an application – to delete an object on a Web Server.

Here are some header field examples for a request message:

- Host: www.someschool.edu
- User-Agent: Mozilla/5.0
- Accept-Language: fr
- Connection: keep-alive / close

HTTP response messages

Let's now take a look at the structure of HTTP response messages:

> HTTP response message general format:
>
> 
>

Here are some header field examples for a response message:

- Date: dateAndTimeOfTheResponseMessage
- Server: Apache/2.2.3 (CentOS)
- Last-Modified: critical for object caching
- Content-Length: Number of Bytes in the Object being sent
- Keep-Alive: timeout=10, max=100
- Connection: keep-alive / close
- Content-Type: text/html

2.1.2 Status Codes

The status code appears in the first line in the server-to-client response message together with its associated phrase; there are multiple status codes for HTTP responses:

- 200 $\Longrightarrow$ `Ok` : request succeeded, the requested object is included later in this message.

- 301 $\implies$ `Moved Permanently` : the requested object got moved, the new location is specified later in the message within the `Location` header.
- 400 $\implies$ `Bad Request` : request message not understood by the server.
- 404 $\implies$ `Not Found` : the requested document was not found on this server.
- 505 $\implies$ `HTTP Version Not Supported` : the requested HTTP version is not supported by the server.

2.1.3 TELNET

Telnet is a client-server application protocol that provides access to virtual terminals of remote systems on local area networks (LANs) or the Internet.
It is a protocol for bidirectional 8-bit communications. Its main goal was to connect terminal devices and terminal-oriented processes.

Example

`telnet <host> <port>` : this cli command opens a tcp connection to the specified host and port, then for example:
type: `GET <url> HTTP/1.1` ;
press enter;
type: `Host: <host>` ;
press enter twice to finally send the HTTP request message;
Read the HTTP response from the server;
quit with `^]` .

2.1.4 Cookies (2.2.4)

Cookies are used by some browsers to maintain *some* state between transactions (vanilla HTTP is *stateless*).

The Cookies mechanism can be divided into four components:

- **Cookie Header Line** in the HTTP *response* message
- **Cookie Header Line** in the next HTTP *request* message
- **Cookie File** kept on the user's end system and managed by the browser
- **Back-End Database** at the Web site

Let's now analyze how cookies are used by browsers and servers:

1. The client makes a simple HTTP request to a server.
2. The server creates an ID for the user and creates an entry in the backend database indexed by said ID.
3. The server responds and includes a *cookie* with this new ID in the response message to the client, in the `Set-Cookie` header.
4. The client's browser sees the cookie and appends the cookie's ID and the server's hostname in its cookie file, stored on the client's computer.
5. The next time that same client will make a HTTP request to that same server, it will include this cookie ID in the `Cookie` header line of the request message.
6. The server receiving this request message with this cookie ID will now be able to identify the client and offer extra functionality bound to this new kind of *statefulness*

Cookies have various uses: Authorizations, Shopping Carts, Recommendations, User session state (web e-mail) and plenty more.

> - **Cookies** permit sites to learn (*a lot*) about you on *their* site.
> - **Third party persistent cookies** allow common identity to be tracked across *multiple web sites*.

2.1.5 Web Caches (proxy servers) (2.2.5)

The goal of Web Caches (A.K.A. Proxy Servers) is to satisfy client requests without involving the origin servers, to alleviate the load on them, reduce response time, and alleviate the load on the Internet as a whole.

Proxy servers act both as a client and as a server and are typically installed by ISPs. They keep copies of recently requested objects in their persistent storage.

The user configures the browser to point to a **web cache**, then when the client requests an object from a server:

- if the requested object is in the cache: the cache returns the object to the client.
- if the requested object isn't in the cache: the latter asks and receives the object from the origin server, caches it and sends it to client.

Problem of recency of cached objects

What do we do if the object is updated on the remote servers and the proxy server isn't aware?
Conditional GET: don't send an object if the cache already has an up-to-date version.
The conditional GET works as follows: the proxy server includes the date of its cached copy in the `If-Modified-Since` header when sending the normal GET statement; the server's response contains no object if the cached copy is up to date (`HTTP/1.0 304 Not Modified`),

otherwise, if the object has been updated, the new version is included in the HTTP response (`HTTP/1.0 200 OK <data>`).
This way the object is only sent if necessary, keeping stress on the link low.

Note: the date/time in the `If-Modified-Since` header is the same included in the `Last-Modified` header in the server's response.

2.1.6 Important updates

HTTP 1.1

This version introduced *multiple pipelined GETs over a single TCP connection*.

In this version the scheduling algorithm for GET requests is **FCFS**: first come first served.
Issue: small objects may wait for bigger (slower) objects (requests) (HOL blocking: **Head of line blocking**).

Also loss recovery (retransmitting lost TCP segments) stalls object transmission.

HTTP 2

RFC 7540, 2015: The key goal for this version was decreasing delay in multi-object HTTP requests.
Increased the flexibility for servers when sending objects to clients: objects are divided into frames and the frame transmission is interleaved.
Frames of different objects are interleaved and scheduled according to the priorities specified by the client, so as to mitigate HOL blocking.
The order of transmission of requested object is now based on client-specified object priority, not necessarily FCFS.

Problems: recovery from packet loss still stalls transmission, also no extra security over vanilla TCP connections.

HTTP 3

The key goal for this version was to decrease delay in multi-object HTTP requests:
Adds:

- security
- per-stream error and congestion control (QUIC, more pipelining) over UDP

2.2 E-mail, SMTP, IMAP (2.3)

The e-mail infrastructure is made of these three major components:

- **user agents**: the user reading, composing and editing mail messages
- **mail servers**: these have a *mailbox* containing incoming messages for the user and a *message queue* of outgoing (waiting to be sent) messages
- a **mail transfer protocol**: for example **Simple Mail Transfer Protocol** (**SMTP**)

The E-Mail protocol is described in RFC 5321.
It uses **TCP** to reliably transfer email messages from clients (mail server that initiates the connection) to servers, **port 25**. It essentially is a direct transfer from the sending server (acting as a client) to the receiving server.

> SMTP email messages must be in 7-bit ASCII. Binary multimedia data must be encoded to ASCII before being sent over SMTP, and decoded after.

Sending Emails

This is a command/response interaction, like HTTP:

- commands: ASCII text
- response: status code and phrase

1. User composes email
2. sends email to his email server
3. the client side of the sender's SMTP server opens a TCP connection with the destination user's server
4. the servers do a SMTP application layer handshake:
    1. HELO (or EHLO)
    2. AUTH LOGIN
    3. MAIL FROM: sender@domain
    4. RCPT TO: recipient@domain
5. the email is sent over the SMTP connection
    1. DATA
6. the receiving server puts the email in the destination user's mailbox
7. the connection gets closed
    1. QUIT
8. the destination user can access the emails in his mailbox via his email server

Comparing SMTP and HTTP

1. HTTP is primarily a *pull protocol*, whereas SMTP is primarily a *push protocol*.
2. both have ASCII command/response interaction and status codes.
3. SMTP requires the message (header and body) to be in 7-bit ASCII, HTTP doesn't.
4. HTTP encapsulates each object in its own HTTP response message, SMTP instead places all of the message's objects into one message.

2.2.2 Mail message format

The format of e-mail messages is specified in RFC 5322 (defines the syntax of the message). First comes the message header, then a blank line (CRLF) and finally the message body:

```
From: user@server
To: user@server
Subject: subject
CRLF
message body in ASCII
```

2.2.3 Mail access protocols (2.3.4)

A typical user runs a user agent on the local pc but accesses its mailbox stored on an always-on shared email server, shared with other users and typically maintained by the user's ISP.

SMTP defines delivery and storage of e-mail messages from the sender to the receiver's server:
the sender's user agent uses SMTP to push the e-mail message into his mail server. Then SMTP is used again to transfer the e-mail between the sender's and the receiver's server. Finally, how does the receiver's user agent retrieve the messages from the user's mailbox on the server?
We use a **mail access protocol** like **IMAP** (Internet Message Access Protocol, RFC 3501) or **POP3** (Post Office Protocol – Version 3).

Mail Access Protocols define how messages are stored on the server and provide retrieval, deletion and organization of stored messages on the mail server.

We can also use HTTP to access web-based interfaces (like gmail, hotmail etc.) that work on top of SMTP (to send e-mails) and IMAP (or POP) (to retrieve e-mails). With this service the user agent is a web browser, and the user communicates with its remote mailbox via HTTP, notably also when *sending* messages, not only when retrieving them.

POP: Post Office Protocol

POP3 is an extremely simple MAP (mail access protocol), defined in RFC 1939.
POP3 begins when the user agent begins a **TCP** connection with the mail server, on **port 110**.
The server accepts the connection and the protocol now processes through 3 phases:

1. **authorization**: the user agent sends a username and a password (in clear-text) to authenticate the user. To do this the user uses two commands: `USER <username>` and `PASS <password>`
2. **transaction**: the user agent retrieves messages and issues commands: marks messages for deletion, removes deletion marks, and obtains mail statistics. Commands: `list`, `retr`, `dele`, `quit`
3. **update**: this phase occurs after the client has issued the *quit* command, ending the POP3 session. At this time, the mail server deletes the messages marked for deletion.

A user agent using POP3 can be configured by the user to adhere to one of two modes: *download-and-keep* and *download-and-delete*.

In a POP3 transaction, the user agent issues commands, and the server responds to each command with a reply.
There are two possible responses: *+OK* (sometimes followed by server-to-client data), used by the server to indicate that the previous command was fine; and *-ERR*, used by the server to indicate that something was wrong with the previous command.

A POP3 server only holds *some* state in a single session, like the list of marked messages, it does not hold state between sessions, this greatly simplifies the implementation of this protocol.
POP servers also do not offer any kind of message organization functionality (folders etc.).

IMAP

To solve this and other problems, the IMAP protocol, defined in RFC 3501, was invented. Like POP3, IMAP is a mail access protocol. It has many more features than POP3, but it is also significantly more complex.

An IMAP server will associate each message with a folder; when a message first arrives at the server, it is associated with the recipient's INBOX folder. The recipient can then move the message into a new, user-created folder, read the message, delete the message, and so on. IMAP also provides commands that allow users to search remote folders for messages matching specific criteria.
Another important feature of IMAP is that it has commands that permit a user agent to obtain components of messages. For example, a user agent can obtain just the message header of a message or just one part of a multipart MIME message. This is useful in low-bandwidth situations.

This means that an IMAP server *holds state between sessions*.

2.3 The Domain Name System (DNS) (2.4)

Problem: internet hosts and routers have both an IP address (IP), used for addressing datagrams, and a "name", used by humans (like www.google.com).
How do we map between IP address and name, and vice versa?

The ***Domain Name System*** (DNS) is a *distributed database* implemented in a hierarchy of many DNS servers.
It is also an application-layer protocol that allows hosts to query the distributed database.

Registering a subdomain means linking it univocally to an IP address, registering it in the DNS database.

2.3.1 DNS services (2.4.1)

- hostname to IP address **translation**
- host **aliasing**: provides alias names for the **canonical hostname**
- mail server aliasing
- **load distribution**: replicated web servers: many IP addresses correspond to one name. When a client asks for the resolution for a hostname, the DNS server answers with the whole list of associated IP addresses, but each time in a different order, since the client normally goes for the first in the list.

2.3.2 ICANN

The distributed DNS database is managed by the ***Internet Corporation for Assigned Names and Numbers*** (***ICANN***), which defines what the **top-level domains** (**TLDs**) are and accredits the various *registrars* (read more on registrars below).

2.3.3 DNS structure

The DNS is a distributed, hierarchical database, which can be approximated to these 3 classes of DNS servers:

- **ROOT**: the local DNS server queries the root DNS server to find the IP address of the Top-Level Domain's DNS server.
- **TOP LEVEL DOMAIN**: the local DNS server queries the top-level domain DNS server to find the address of the authoritative DNS server.
- **AUTHORITATIVE**: the local DNS server queries the authoritative DNS server to find the IP address for the desired host.

A distributed structure was chosen because a centralized DNS:

- doesn't scale
- has a single point of failure (SPF)
- cannot be near every host
- cannot handle that much traffic volume (Comcast DNS servers serve 600B DNS queries per day)
- cannot be easily maintained

ROOT name servers

These servers are the official contact-of-last-resort for name servers that cannot resolve the queried name.
There are 13 logical root name servers worldwide, each server is replicated many times (there are hundreds of root server instances worldwide using anycast).

Local DNS name servers

Also called *default name server*s, these do not strictly belong to the hierarchy of servers and are installed into each ISP and they act as a sort of proxy DNS server, they have a local cache of recent name-to-address translation pairs, BUT it may be out of date!
When a host connects to an ISP, the latter provides the host with the IP addresses of one or more of its local DNS servers, typically through DHCP.
When a host makes a DNS query it is sent to its local DNS server which then queries the main hierarchy.

DNS caching

DNS caching is essential for reducing the load on the DNS servers and the internet as a whole, given the frequency of such requests.
In a query chain when a DNS server receives a DNS reply, it stores the translation in its local memory, then, when it receives a DNS query for that same host, it replies with its stored translation – if it is recent enough – even if it isn't an authoritative server for that hostname.

Cached translations last for their initial TTL (time-to-live) – which usually is two days – before getting discarded.

2.3.4 DNS records (2.4.3)

The DNS is a distributed database storing **resource records** (**RR**). Each DNS reply contains one or more of these records.
A Resource Record is a 4-tuple that contains these fields: `name, value, type, ttl`, whose meaning depends on the **type** of record:

- **type=A** (address)

    - name is a hostname
    - value is the associated IP address
- **type=NS** (name server)
    - name is the domain (what you enter in the search bar)
    - value is the hostname of the authoritative name server for said domain
- **type=CNAME** ("canonical name")
    - name is an alias name for some "canonical" name (the real name)
    - value is the canonical name for the alias
- **type=MX**
    - name is a hostname (or a domain name)
    - value is the name of the mailserver associated with `name`
- there are even more types

If a server is authoritative for some hostname, it will contain a type A record for it.
If a server is *not* authoritative for some hostname, it will contain a type NS record for the domain that includes the hostname and it will also contain a type A record that provides the IP address of the DNS server referenced in the NS record.

2.3.5 DNS protocol messages (2.4.3)

The first 12 bytes of a DNS message are the **header section**, of which the first 2 bytes are the DNS query ID (identification); this identifier is copied into the reply message to the query, allowing the client to match received replies with sent queries.

<- 2 bytes ->	**<- 2 bytes ->**
identification (ID)	**flags**
number of questions	number of answer RRs
number of authority RRs	number of additional RRs
questions	variable length (name, type query)
answers	variable length (Resource Records)
authority	variable length (Resource Records)
additional info	variable length (Resource Records)

flags:

- query (0) or reply (1).
- recursion desired; can be set by both hosts and DNS servers.
- recursion available; set by DNS servers who offer recursion.
- reply is authoritative; set by DNS servers when their reply is authoritative.

`nslookup` : command-line tool to query the DNS server.

2.3.6 Name resolution Approaches

Iterated Query

The host first asks the local DNS server, which in turn contacts every required DNS server until resolution, the contacted servers reply with the name of the server to contact and the local DNS server executes.
When the local DNS server has resolved the name, it replies to the host with the answer.

This approach is better.

Recursive Query

The host first asks the local DNS server, which in turn contacts the next server (the root), which then asks the next (TLD) etc., until resolution. Every contacted server asks the next, in a chain of queries and answers, until the authoritative server has the answer, at this point every server replies with the answer down the chain, until it arrives to the host.

This leads to more load on the servers, apart from the local DNS servers.

2.3.7 Inserting Names into the DNS

To insert a domain in the DNS, the domain name must be registered at a **DNS registrar**. Registrars are commercial entities that verify the uniqueness of the domain name and enter it into the DNS database; Network Solutions had the monopoly of most domains until 1999.

To register a name you must create the primary and secondary authoritative DNS servers locally, provide names and IP addresses of your primary and secondary authoritative name server to the Registrar, who then inserts **NS** and **A** records (RRs) into the TLD (top level domain) servers.

Until recently, DNS records had to be manually configured, now an `UPDATE` option has been added to the DNS protocol, to allow data to be dynamically added or deleted from the database via DNS messages.

2.3.8 Example of an (iterative) DNS resolution

Requesting host (alice.iet.unipi.it) asks the local DNS server what the IP for www.networkutopia.com is.
Local DNS server contacts the root DNS server, which replies with the IP of the .com DNS server.
Local DNS server now contacts the .com DNS servers, which replies with the IP of the authoritative server for networkutopia.com.
Now the local DNS server contacts the authoritative server, which replies with the information needed, which is now routed back towards the initial client with finally a reply to the requesting host.

2.3.9 DNS security

DNS servers are susceptible to:

- DDoS bandwidth-flooding attacks, which have not been successful against root servers to date due to extensive replication and caching
- Redirect attacks:
  - man-in-the-middle: intercepting DNS queries and returning bogus replies to hosts
  - DNS poisoning: sending false replies to DNS servers, which then get cached
- Exploit of DNS for DDoS (amplification attack): spoofing the source IP address of DNS requests to appear as the victim's IP. When DNS servers respond, they send much larger replies to the victim, overwhelming it with traffic.

Redirect attacks and DNS cache poisoning are mitigated by **DNSSEC** (**Domain Name System Security Extensions**), which signs DNS records with cryptographic digital signatures. This guarantees the authenticity and integrity of DNS replies. (Note: DNSSEC does *not* prevent DDoS amplification attacks; in fact, its larger record sizes can increase the amplification factor).

3 Peer-to-Peer File Distribution (2.5)

Every peer looks more like a powerful client from the 1.1.1 Client-Server paradigm, technically every peer is on the same level.
The availability of resources this way is not guaranteed from a powerful server, but from the sheer number of peers $\implies$ ***self scalability***.
There isn't an always on server.
End systems arbitrarily communicate with each other.

> Peers request service from other peers, providing service to other peers in return.

Indexes are needed for mapping information to the right host locations.

3.1 Content Indexes

Indexes are needed for mapping information to the right host locations.
A content index is a Database with `(key,value)` **pairs**.

- *key* is what identifies the content (for example its name)
- *value* is the IP address
  The peers query the database with the key and the database replies with values that match the key.
  Peers can also insert pairs.

One possible way to implement an index could be a **centralized index**: a service provided by a server (or a server farm) in which when a user becomes active, the application notifies the index with its IP address and a list of available files. This is the system used by Napster, a P2P system for music sharing.
Note that the files are distributed by peers, but the search is client-server style: it is a **Hybrid Approach.**

This approach has a few drawbacks: it presents a single point of failure, the central index could be a performance bottleneck and copyright problems.

3.1.1 Query Flooding

This is a completely decentralized approach.
When searching for one item, a peer starts querying other peers, which in turn contact other "neighbors" (**flooding**), until one sends the item to the original peer searching for it.
The file download is done from a single peer.

Limited-scope query flooding: query flooding with a predetermined, limited number of *hops* (scope).

- it reduces the query traffic and therefore congestion.
- but decreases the probability to locate the content.
- *limited-scope*: the query flooding stops at a certain "level" of subquery (for example each query starts with x, the peers receiving it decrement it and sends it over and again, until $x = 0$)

It is possible to plot an Overlay Network: a graph formed of all active peers as nodes and the TCP connection among them as edges.

This system was used in the original Gnutella version (LimeWire).

3.1.2 Hierarchical Overlay

This approach is a middle ground between centralized and completely decentralized.
Not all peers are equal, ***Super Nodes*** (***SN***) exist, these are peers with high bandwidth and high availability.
SN have **local indexes**: peers inform their SN about content they have available, and SNs form an **SN overlay net**.
Peers ask their local SN where they can find an item, SN responds with the IP of the peer with the item, or, if the item is not in the local index, the SN asks other SNs, until the item is found.
Again the file download is done from a single peer.
Used in modern Gnutella.

3.2 How much time does it take to distribute one file to N peers?

time to distribute in client-server approach:

$$D_{c-s} \geq \max\left\{\frac{N \cdot F}{u_s}, \frac{F}{d_{\min}}\right\}$$

the first component scales linearly with N.

time to distribute in peer-to-peer approach:

$$D_{P2P} \geq \max\left\{\frac{F}{u_s} \quad , \quad \frac{F}{d_{\min}} \quad , \quad \frac{N \cdot F}{u_s + \sum u_i}\right\}$$

If we suppose $\sum u_i = N \cdot \overline{u}$, as $N \to \infty$, we have $\lim_{N\to\infty} \frac{N \cdot F}{u_s + N \cdot \overline{u}} = \frac{F}{\overline{u}}$.

So client-server scales linearly with N, while peer-to-peer approaches a finite limit.

3.3 BitTorrent

The file that needs to be transferred gets divided into 256KB chunks, and the peers in the **torrent** send and receive these chunks.
A **Torrent** is a group of peers participating in the distribution of a particular file, a **tracker** is a node that tracks peers participating in the torrent and a **Torrent Server** is a server that knows the IPs of the trackers. Peers may discover other peers by trackers, DHT (distributed hash table), or PEX (peer exchange).
Chunks don't even get transferred sequentially, every chunk has an ID so the entire file gets rebuilt by the asking peer once every chunk has arrived.
While a peer is downloading chunks it also uploads already downloaded chunks, then, when it has every chunk, it may leave or remain in the torrent to aid other peers.

3.3.1 Entering a Torrent

1. Alice contacts the torrent Server and gets the IP of the tracker.
2. Alice contacts the tracker and asks to be included in the torrent.
3. The tracker adds Alice to the list of peers participating to the torrent and then sends the list to Alice.
4. Alice *tries* to open concurrent TCP connections with every peer in the list.
5. Alice manages to open TCP connections with only a subset of the torrent list, the peers she manages to connect with are called **neighboring peers**. Peers are divided in ***leeches*** (ones who don't have a complete copy of the file), ***seeders*** (ones who have every chunk) and ***free-riders*** (who only want to download the file, not seed the file).
6. Alice asks now her neighbors what chunks they have.
7. Alice asks first for the **rarer** chunks, the ones that are scarce in the "neighborhood". This approach is called **rarest first**.
8. Now that Alice has some chunks, other neighbors ask her for chunks, she now has to choose which neighbors to send the chunks to. She now adopts the ***tit for tat*** approach, she starts with sending the data to the **4** neighbors sending her chunks at the highest rate (these peers get **unchoked**). Alice re-evaluates the top 4 peers every 10 seconds. Note that **choked** peers (every not-unchoked peer) do not get any chunks from Alice.
9. Every 30 seconds Alice randomly selects another peer for sending to: she "**optimistically unchokes**" this peer, who may also join the top 4.

This method of distribution has the effect of pairing peers with similar bandwidth capability and limiting freeriders.

4 Video streaming and content distribution networks (2.6)

The video stream traffic is the major consumer of internet bandwidth.
Content Distribution services are responsible for 80% of residential ISP traffic (data from 2020).

An image is an array of pixels, where each pixel is represented by bits.
A video is a sequence of images displayed at a constant rate (*framerate*), normally 24,25 or 30 images per second.

4.1 Coding

We can use redundancy *within* and *between* video frames to decrease the number of bits used to encode the image.

- **spatial coding**: instead of sending N values of the same color, just send the color and the number of times it repeats (N).
- **temporal coding**: instead of sending the complete $i+1$ frame, just send the differences from the i frame

Also **video encoding rate** can be: **CBR** (constant bit rate) and **VBR** (variable bit rate): the video encoding rate changes as the amount of spatial and temporal coding changes

Examples of compression standards:

- MPEG1 (CD-ROM): 1.5 Mbps
- MPEG2 (DVD): 3-6 Mbps
- MPEG4 (often used in internet): 64Kbps - 12 Mbps
  NOT to be confused with "MPEG4 part 14", aka "MP4", which is a container format.
  MPEG stands for Moving Picture Experts Group, a collection of standards developed by ISO/IEC for audio and video coding.

Note: coding and encoding here are synonyms.

4.2 Challenges of streaming

Server-to-client bandwidth, packet loss and delay vary over time. Also we must respect the *continuous playout constraint*: once the client's playout begins, playback must match the original timing.

In an ideal world the servers send one frame every 1/30th of a second, the user receives one frame at the same interval, with a fixed network delay, and displays the frames at the correct framerate.
But **Network delay is variable**! Solution: clients store the received frames (*client-side buffering*) and display them at the correct framerate with a *client playout delay*.

4.3 Compression

An important characteristic of video is that it can be compressed, trading quality with lower file sizes. Today we have algorithms that can compress videos to any desired bit-rate, the higher the bitrate the higher the quality and the overall user viewing experience.

Compressed video typically ranges between 100 Kbps to over 10 Mbps for 4k streaming. This amounts to a huge amount of traffic and storage: a single 2 Mbps video of a duration of 67 minutes will consume 1 GB of storage and traffic ($\text{size} = \text{bitrate} * \text{time}([\text{s}])$).
In order to provide continuous playout, the network must provide an average end-to-end throughput to the streaming application that is at least as large as the desired bitrate for the video.

We can use compression to create and store the same video at different bitrates, so that the streaming application (user) can choose which one to request based on the internet throughput it knows it can achieve.

4.4 HTTP streaming (2.6.2)

The video is simply stored at an HTTP server as an ordinary file with its URL. When a user wants that video, it establishes a TCP connection with the server and issues an HTTP GET request for that URL.
The server then sends the video file within an HTTP response message, as quickly as the network allows.
On the client side the bytes are **buffered**, once the number of collected bytes achieves a predetermined threshold, the client application begins playback:
the streaming application gets the bytes from the streaming buffer, decompresses them into frames, and plays them back at the right framerate on the user's screen, all while continuing to receive new information (the video) from the server.
This way of streaming has worked well but it has a shortcoming: every user receives the same video, independently from its network capability.

4.4.1 Dynamic Adaptive Streaming over HTTP (DASH) (2.6.2)

In DASH the video gets stored on the server at different bitrates (every version is a different file so it has a different URL), the streaming application dynamically chooses chunks of a few seconds of the video in whatever bitrates it deems adequate as a function of user connection bandwidth.

The HTTP server has a **manifest file**, which provides an URL for each version of the video along with its bitrate.

1. The client requests the manifest file
2. the client requests a chunk of the video at a desired bitrate
3. while downloading the chunk the client measures the received bandwidth and runs a rate determination algorithm to select the chunk to request next.

4.5 Content Distribution Networks (CDNs) (2.6.3)

When content providers choose how to deliver content to millions of users the simplest approach would be to just build a single capable server, but this has many drawbacks (single point of failure, inefficiency – popular media sent many times over the same network, congestion etc.),

thereby the applied strategy is to build servers all around the world, that sort of act like proxies, in which they store the most requested files in their area of operation.

The user asks the general server where it can find the file and the server redirects it to a server near the client which has that piece of media stored. If there is no such server near the client, CDNs utilize the *PULL* approach: the main server will quickly send the media to one of the servers accessible by the client, who will then download the file from the latter. When a server is full it removes the less frequently requested files.

There are two different server placement philosophies:

- **Enter deep**: building many servers and incorporate them near access ISPs, improving user perceived delay and overall user experience.
- **Bring home**: building fewer servers and employing them near IXPs. This results in *lower maintenance* needed compared to "enter deep", but *higher average latency* and more congestion.

4.5.1 CDN modus operandi

Let's now analyze a common way – using DNS – in which CDNs intercept user's requests for content they provide, choose a cluster and a server from said cluster from which the user should pull the content:

1. The user visits a web page from a content provider.
2. The user clicks on a link identifying a piece of content resulting in the user's host sending a DNS query for the subdomain of said file.
3. The user's local DNS server (LDNS) relays the query to an authoritative server for the requested domain, which – observing the subdomain indicating a piece of content – returns a hostname in the domain of the CDN.
4. The LDNS sends a second query, this time for the hostname in the CDN's domain, getting the IP address for the server that will provide the piece of content. Here the CDN's DNS servers determine the best fitting node for that exact client and reply with its IP address.
5. The LDNS replies to the user's host with the IP of the content-serving CDN node.
6. The client's host now initiates a TCP connection to said CDN node and issues an HTTP GET request for the piece of content it has been searching. If DASH is used the server will first send the manifest of the video and the client will dynamically select chunks from the differently compressed versions listed in the manifest.

4.5.2 Cluster Selection Strategies

When the CDN learns the IP address of the client's LDNS server via the client's DNS lookup, with this info it can choose the best fitting CDN node to serve the request.
One cluster selection strategy is to simply choose the CDN node geographically closest to the client (assuming the latter is located near the LDNS).
A more complete strategy is to perform periodic real-time measurements of delay and loss between their clusters and clients, for example by having each cluster periodically send probes to all of the LDNS servers around the world. One drawback is that many LDNS servers are configured to ignore such probes.

4.5.3 Netflix Case Study

Information regarding Netflix account is stored on the *Netflix registration and accounting servers*. The user browses the Netflix catalog, which is stored on *Amazon cloud*. These Amazon cloud servers upload copies of the multiple versions of these videos to the *DASH CDN servers*.
When the user asks the Amazon cloud servers for a piece of content, the servers return the manifest file for it, at this point the user's machine contacts the correct DASH CDN server and the streaming begins.

5 Sockets (2.7)

Sockets are a mechanism of communication between processes, on the same host or on different hosts: a socket is an end of a communication channel, an abstraction implemented by the operating system.
A socket is identified by an address, which is composed of an IP address and a port number, which respectively identify the host and the process running on said host.

The C/Unix side of sockets — primitives, `errno`, blocking vs non-blocking, `select()` / `poll()` — is in Unix Socket programming in C; here we keep only what shapes the *application protocol*.

5.1 Blocking vs Non-Blocking

A call on a socket can either **block** until it can be satisfied, or — if the socket is set **non-blocking** — return immediately reporting that nothing can be done right now, in which case something must tell the application *when* to retry (I/O multiplexing, 5.6 I/O Multiplexing).
This is not only an API detail: a blocking design cannot serve two connections at the same time unless the server uses a thread or a process per connection.

5.2 TCP vs UDP

TCP guarantees that every byte arrives, in order, but it has no notion of *message*: the receiver cannot know where a message ends unless the protocol delimits it (see 5.3.2 Message delimitation). **UDP** preserves message boundaries — one datagram is one message — but can lose, duplicate or reorder datagrams, and a datagram must fit in a single IP packet (Transport Layer > 3 Connectionless transport: UDP (3.3)).

5.3 Sending Data

A socket only gives us a *stream of bytes* (TCP) or a sequence of independent *datagrams* (UDP): everything an application protocol needs — where a message begins and ends, what a field means, how many bytes it occupies — must be agreed upon by the two endpoints and written explicitly into the bytes that travel on the wire. A reliable transport does not free us from thinking about the structure of our data: it only guarantees that the bytes arrive, not that they arrive the way we wrote them. These are the considerations one must make when designing an application-layer protocol and, more generally, when sending data through sockets.

5.3.1 Endianness

When designing a network application one must account for system endianness differences between the two (or more) communicating hosts: machines can either be **big-endian** or **little-endian**.
A **big-endian** system stores the Most Significant Byte (MSB) at the lowest memory address (first in memory), while a **little-endian** system stores the Least Significant Byte (LSB) first.

Different systems use different endianness, so a protocol must state the byte order of every multi-byte field: the convention is **network byte order** (big-endian), i.e. the sender converts to it and the receiver converts back to its own **host byte order**, which could be either big or little endian — the programmer has often no way of knowing beforehand.

5.3.2 Message delimitation

TCP does not preserve message boundaries: it guarantees that the bytes arrive *in order*, not that they arrive *in the same chunks in which they were written*. One write of 1000 bytes can be delivered to the receiver as two reads of 300 and 700 bytes, and two consecutive writes can be coalesced into a single read (Transport Layer > 5.2 Sequence Numbers and ACKs). The receiver must therefore be able to tell where a message ends, and the usual options are:

- **length prefix**: a fixed-size field (for example 4 bytes, in network byte order) states the length of the payload that follows. It is unambiguous, it works with binary data and it lets the receiver know in advance how much to read — but the length must be validated (see 5.3.5 Size, timing and errors).
- **delimiter**: the message ends with a special sequence (`\r\n\r\n` to end the headers in HTTP, `\n` in line-based protocols, `\0` for C strings). It is human readable and needs no length field, but the delimiter must be escaped (or forbidden) inside the payload, and the receiver must scan for it while receiving.
- **fixed-size messages**: viable only when the payload size is known in advance.
- **connection close**: everything until `EOF` is one message (as HTTP/1.0 did when `Content-Length` was absent); it costs a connection per message.

UDP instead preserves message boundaries — one datagram is one message — but a datagram can be lost, duplicated or reordered, and it must fit in a single datagram (Transport Layer > 3 Connectionless transport: UDP (3.3)).

5.3.3 Partial sends and reads

- Data can be *written* in pieces: a write can hand the transport less than what was asked (buffers are finite), so the protocol must not assume that one write = one message, and the writer must be prepared to send the rest later.
- Data can be *read* in pieces: the receiver must accumulate the bytes in a buffer and only interpret them once a complete message (as defined in 5.3.2 Message delimitation) has arrived — one read does not correspond to one write.
- The API consequences of this (short `send()` s, `EAGAIN` , `EINTR` , `SIGPIPE`) are in Unix Socket programming in C.

5.3.4 Structs and serialization

- Never send a `struct` as it is: the compiler may insert **padding** between fields for alignment reasons, and the size and layout of the same `struct` differ between architectures (and even between compilers), so the receiver may interpret the same bytes differently or read a different number of them.
- Send each field separately, with the multi-byte ones in network byte order (5.3.1 Endianness), or define an explicit, architecture-independent serialization format and document it.
- For textual protocols be explicit about the **encoding** (ASCII, UTF-8) and about how strings are delimited: a length prefix is binary-safe, a `\0` terminator is not.
- Add a **version** field and a **type** (or opcode) field to the header: they let the receiver demultiplex different messages and let the protocol evolve without breaking the old peers (see the DNS header in 2.3.5 DNS protocol messages (2.4.3)).

5.3.5 Size, timing and errors

- Always bound the size of a message: a bogus or malicious length field must not make the application allocate gigabytes of memory — reject anything above the maximum size the protocol allows.
- Keep messages small enough to avoid IP fragmentation (a typical MSS is 1460 bytes, see Transport Layer > 5 Connection-Oriented transport: TCP (3.5)); if the data is larger, split it in the application and add a sequence number to reassemble it.
- Blocking calls can wait *forever* if the peer goes silent, so the protocol must say what a party does when it gets no answer: retransmit, give up, report an error. Every waiting operation needs a timeout or a readiness check (5.6 I/O Multiplexing).

- Neither a slow receiver nor a fast one is an error: when the receiver cannot keep up, the transport's flow control slows the sender down (Transport Layer > 5.5 TCP flow control (3.5.5)); the application must not busy-wait on it.

5.3.6 Other protocol design considerations

- **Message identity**: if a request can be retransmitted (timeout, duplicate), add a request ID so that the server can recognise and discard duplicates, and decide whether the protocol is *at-least-once* or *at-most-once*.
- **Error reporting**: define messages (or status codes, see 2.1.2 Status Codes) for failures instead of just closing the connection.
- **Liveness**: an idle TCP connection can be half-open (the peer crashed or the network went away) without either side noticing; use TCP keepalive or an application-level heartbeat.
- **Coalescing**: many small writes are inefficient (each one pays for a header and for the system call, and small segments interact badly with delayed ACKs in request/response protocols): if the protocol allows it, batch several messages into one write.
- **Confidentiality and integrity**: sockets are cleartext, so TLS/SSL must be layered on top (1.3.2 Securing TCP); checksums such as CRC only protect against accidental corruption, not against an attacker.

5.4 Closing a connection

When a process closes its socket, the operating system starts the normal FIN handshake: the remote side sees the end of the byte stream (EOF), which is the "connection close" delimitation of 5.3.2 Message delimitation.

An **RST** (reset) is sent instead on an abnormal termination — for example if a process dies while unread data is still pending in its receive buffer, or if data arrives for a socket that has already been closed. The remote side then gets an immediate error instead of a clean end of stream, so a protocol should not rely on `close()` to signal "message complete".

5.5 Unix Socket primitives

The C prototypes (`socket` , `bind` , `listen` , `accept` , `connect` , `send` , `recv` , `sendto` , `recvfrom` , `close`), the address structures (`sockaddr_in` , `in_addr`), the `errno` table and working examples are in Unix Socket programming in C.

5.6 I/O Multiplexing

A server that must handle many clients cannot afford to block on one of them: **I/O multiplexing** lets a single process wait on many sockets at once and be woken when at least one is ready, i.e. when the next operation on it will not block. It is the structure of an event loop, and it is why "non-blocking" is useful at all.

Two consequences that matter for the protocol: *ready* does not mean *a whole message has arrived* (see 5.3.3 Partial sends and reads), and the number of sockets a single process can watch is bounded — thousands with `select()` , practically unbounded with `epoll()` .

`select()` , `poll()` and `epoll()` — syntax, ready conditions, descriptor sets, limits — are in Unix Socket programming in C > I-O multiplexing.

Transport Layer

#uni

- 1 Introduction
- 2 Multiplexing and Demultiplexing (3.2)
- 3 Connectionless transport: UDP (3.3)
  - 3.1 UDP segment header
  - 3.2 UDP checksum (3.3.2)
- 4 Reliable Data Transfer protocols (3.4)
  - 4.1 Stop-and-Go protocols
  - 4.2 Pipelining
    - 4.2.1 Error recovering strategies
- 5 Connection-Oriented transport: TCP (3.5)
  - 5.1 TCP segment structure (3.5.2)
  - 5.2 Sequence Numbers and ACKs
  - 5.3 Round-Trip Time Estimation and Timeout (3.5.3)
  - 5.4 TCP reliable data transfer (3.5.4)
    - 5.4.1 Doubling the Timeout Interval
    - 5.4.2 TCP fast retransmit
  - 5.5 TCP flow control (3.5.5)
  - 5.6 TCP connection Management (3.5.6)
    - 5.6.1 The 3-way handshake
    - 5.6.2 Closing a TCP connection
  - 5.7 Congestion Control (3.6)
    - 5.7.1 Approaches to congestion control (3.6.2)
  - 5.8 TCP Congestion Control (3.7)
    - 5.8.2 TCP cubic
    - 5.8.3 ECN: explicit congestion notification echo (3.7.2)
    - 5.8.4 TCP fairness (3.7.1)
- 6 Evolution of transport-layer functionality
  - 6.1 QUIC: Quick UDP Internet Connections

1 Introduction

The transport layer manages the logical communication *between application processes running on different hosts*, whereas the network layer manages the logical communication *between hosts*.

The Sender in transport protocols breaks application messages into **segments** and passes them to the *network layer*, which is going to form packets with them and make them arrive at destination. At the receiving end the network layer passes the received packets to the transport layer, which reassembles them into messages and demultiplexes them to the *application layer* via sockets.

Since the transport layer uses the network layer to actually bring its segments to the right hosts, there are certain services it cannot provide if they aren't enabled by the underlying network protocol — for example, delay or bandwidth guarantees. There are instead some services that even when not offered by the network, can be offered by the transport layer: reliable data transfer and encryption are only two examples.
Keep in mind that the internet protocol is a **best-effort delivery service**: it does not guarantee segment delivery, orderly delivery of segments and integrity of segment data on arrival.

The internet offers two transport layer protocols: TCP and UDP, which we are going to analyze later.

2 Multiplexing and Demultiplexing (3.2)

The fact that there are multiple processes running on the same host calls for the need for multiplexing and demultiplexing: they are the mechanism that extends host-to-host delivery to process-to-process delivery.

Multiplexing is done at the sender side: data from multiple sockets is handled and put into segments, to which the respective transport layer header is added.
Demultiplexing is done at the receiver side: the transport header info is used to deliver the received segment to the correct socket—the correct process.

Processes in a host use different ports, which are identified by unique **port numbers**: each port number is a 16 bit number ranging from 0 to 65535, where the port numbers from 0 to 1023 are called **well-known port numbers** and are restricted to well-known application protocols.

Connectionless demultiplexing

A UDP socket is fully identified by a two-tuple consisting of a destination IP address and a destination port number.
The host receives IP datagrams: each datagram carries one transport-layer segment, has a source IP address and a destination IP address and has source and destination port numbers.
At the receiving host, the transport layer uses the *port number* to direct the message extracted from the segment to the appropriate socket.

Connection-Oriented demultiplexing

A TCP socket gets identified by a four-tuple:

- source IP address
- source port number
- destination IP address
- destination port number
  The receiver uses *all four* of these values to direct the message extracted from the received segment to the appropriate socket.

This means that two arriving TCP segments with same destination but different source get routed to different sockets.

3 Connectionless transport: UDP (3.3)

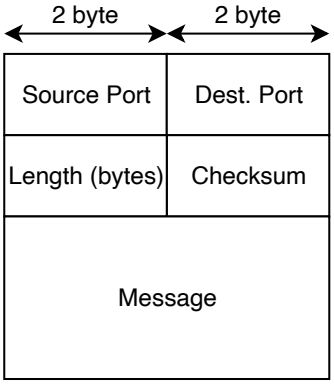
UDP (**User Datagram Protocol**) is a "best effort" service, like IP.
Segments may be lost and delivered out of order to the application.
This is because UDP is ***connectionless***: there is no handshake between UDP sender and receiver, each UDP segment is handled independently of others.

Some applications are better suited for UDP for the following reasons:

- no connection: no establishment delay, no connection state memory, more concurrent users
- no congestion control: faster sending rate
- smaller packet header overhead

This leads to UDP being useful for streaming and multimedia, DNS, network management like SNMP and more.

3.1 UDP segment header

> UDP segment header:
>
> 
>

Since the Message (the application layer data) can be of variable length, we need to introduce a Length field, which states the size—in bytes—of the UDP segment, both header and message.

3.2 UDP checksum (3.3.2)

UDP needs to introduce error detection on an end-to-end basis since there is no guarantee that all links between source and destination provide error checking, some bits could even flip while being stored in routers.

The sender treats the contents of the UDP segment (header and IP address included) as a sequence of 16-bit integers, then the checksum is computed as the sum of these 16-bit integers (with overflows being wrapped around) and the 1s complement of this checksum value is put into the UDP checksum field.

The receiver computes the checksum as well and compares it to the checksum included in the header: if these are not equal and an error is detected, but nothing is done to correct it by UDP: some implementations discard this segment, others still deliver it to the application but with a warning.

4 Reliable Data Transfer protocols (3.4)

Before presenting TCP, let us analyze how we can achieve a **reliable data transfer** over a lossy and unreliable channel.

4.1 Stop-and-Go protocols

1. Reliable data transfer over a perfectly reliable channel
2. reliable data transfer over a channel with bit errors: 2.0(ACKs) 2.1(seq. numbers) 2.2(no NACK)
3. reliable data transfer over a lossy channel with bit errors: timer

RDT3.0 "Stop and Go" Performance evaluation

"Stop and Go": sending one packet and waiting for ACK.

$$
\begin{gathered}
D_{trans} = \frac{L}{R} \\
RTT = 2 \cdot d_{prop} \\
t = RTT + D_{trans} \\
U = \frac{D_{trans}}{t}
\end{gathered}
$$

But since $D_{trans} << d_{prop}$ we get that U is extremely low.

4.2 Pipelining

The answer to the stop-and-go protocol's performance problem is sending multiple packets back to back, without waiting for an ACK for every packet.
To allow for pipelining we must increase the range of sequence number and give sender and receiver buffers.

4.2.1 Error recovering strategies

Go-back-N – GBN

In this protocol (**GBN**) the sender is allowed to transmit multiple packets (when available) without waiting for an acknowledgement, but is limited to a maximum of **N** unacknowledged packets in the pipeline.
We define **base** the sequence number of the oldest not acknowledged packet and **nextseqnum** the smallest unused sequence number (the sequence number of the next packet to be sent). So:

- the sequence numbers in $[0, \text{base} - 1]$ correspond to acknowledged packets
- the interval $[\text{base}, \text{nextseqnum} - 1]$ corresponds to packets sent but not acknowledged
- the sequence numbers in the interval $[\text{nextseqnum}, \text{base} + N - 1]$ can be used for packets that can be sent immediately if data arrives from the upper layer
- the sequence numbers $\geq \text{base} + N$ cannot be used until an unacknowledged packet currently in the pipeline (the one with sequence number base) has been acknowledged.
  This means the range of permissible sequence numbers can be viewed as a **sliding window** of size N over the range of sequence numbers.
  For this reason N is called **window size** and the Go-Back-N protocol gets referred to as a **sliding window protocol**.

A packet's sequence number is carried in a fixed-length field in the packet header, suppose this field is k bits in size, we get 2^k possible sequence numbers (numbered 0 to $2^k - 1$), of which at most $2^k - 1$ can be in use at the same time. When working with a finite range of sequence numbers, every operation must be done in modulo- 2^k arithmetic.

As we said the sender is allowed to send packets only while the window is not full, one must choose what to do with this restriction: **buffering**, telling the upper layer to retry later or a **synchronization** mechanism that allows the upper layer to send data only when it's possible are examples.

As for the receipt of ACKs: an acknowledgement for a packet with sequence number n will be taken to be a **cumulative acknowledgement** for every packet up until n.

The protocol's name is derived from the sender's behavior in the presence of lost or delayed packets: the sender keeps a timer for the oldest unACKed packet, if an ACK is received but there are still additional transmitted unACKed packets it restarts the timer, if every packet gets ACKed the timer is stopped, instead, when the timer expires, the sender *resends every unacked packet*.

When the receiver receives a packet, if it is the one it expected (the one following the last ACKed packet), it sends an ACK for it and delivers the data to the upper layer, in every other case the packet gets discarded and an ACK for the last accepted packet is sent.
The receiver *could* buffer out-of-order packets while waiting for the next in line, but if the desired packet gets lost, every packet up until the latter gets resent, so the buffering becomes useless.

GBN can however still suffer from performance problems, for example while working with a large window and a big $\text{bandwidth} \cdot \text{delay}$ product whenever a packet error happens a large amount of packets gets retransmitted, many unnecessarily.

Selective repeat – SR

Selective-Repeat protocols aim to solve retransmission-related problems in GBN protocols: the sender sends a maximum of N packets starting from the **base** (the oldest not ACKed sent packet) in the pipeline and the receiver *ACKs individual packets*.
The window slides on arrival of in-order packets and out-of-order packets get buffered until any preceding missing packets have arrived, so to deliver them in order to the upper layer.

The sender implements a timer for every unacked packet: if the timer expires it retransmits only the corresponding packet.
Note that on arrival of a duplicate packet, it must still be ACKed, since the sender could have not received the ACK for it and therefore resent it.

This is more efficient although it may happen that the sender sends a duplicate packet and the receiver accepts it: for example with only 4 sequence numbers (0 to 3), if the sender sends 4 packets and all 4 ACKs are lost, the sender retransmits packet 0 and the receiver, whose window has already slid past it, accepts it as a new packet.
A solution for this problem could be to make the sequence number space at least double the window size.

5 Connection-Oriented transport: TCP (3.5)

TCP (**Transmission Control Protocol**) is a connection-oriented (since a handshake is required before sending data) transport layer protocol.
It introduces the **point-to-point** concept: one sender, one receiver, multicasting is not allowed.
It achieves a reliable, in-order *byte stream* from the application standpoint and provides a **Full duplex** communication: the flow is bidirectional in the same connection.

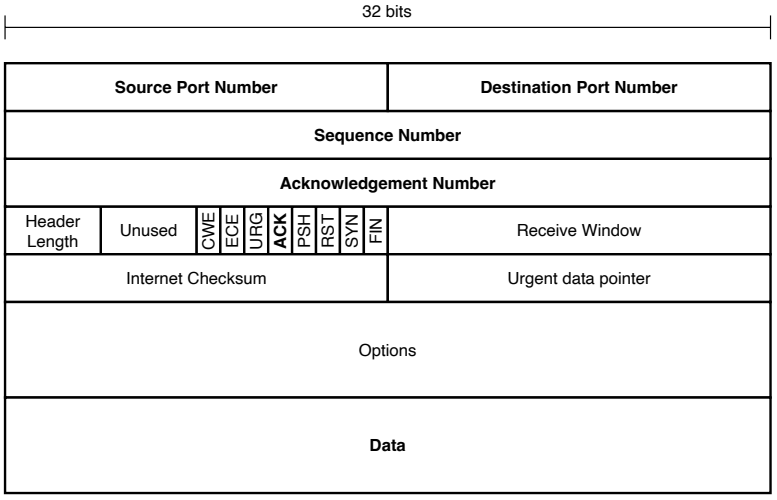
After a connection has been established, TCP passes the application's data to the TCP send buffer, which was allocated during the three-way-handshake, from where it will eventually be passed to the network layer and sent, at TCP's convenience (depending on flow control, congestion, etc.). When arriving at the receiver host, these segments will momentarily be stored at the TCP receive buffer, from where the application will read the byte stream of data from the sender.

Every segment has a **maximum segment size** (**MSS**): the maximum size of a segment's data field, typically set by first determining the largest link-layer frame that can be sent by the local sending host (called **maximum transmission unit** — **MTU**) and subtracting 20 bytes for the IP header and another 20 bytes for the TCP header, considering that both ethernet and PPP link-layer protocols have an MTU of 1500 bytes a typical MSS is 1460 bytes. Obviously when sending more data than what the MSS allows, the message will be broken into different segments, each respecting the maximum data field size.

TCP's congestion and flow Control sets the window size of the **pipelining** (Direct Connection Networks > 3.2 Pipelining).

5.1 TCP segment structure (3.5.2)

> TCP segment structure:



The **receive window** field is used for flow control: it indicates the number of bytes a receiver is willing to accept. The **options** field is used to negotiate MSS, used as a scaling factor, timestamping or used for other things, and it is of variable size (hence the header length field).
The flag field contains 8 bits:

- the **ACK** field is used to indicate that the acknowledgement field is valid
- the **RST**, **SYN** and **FIN** fields are used for connection setup and teardown
- the **CWR** and **ECE** bits are used in explicit congestion notification
- the **PSH** bit indicates that the receiver should pass the data to the upper layer immediately
- the **URG** bit is set when the sender's upper layer entity has marked the data in the segment as urgent, it indicates that the **urgent data pointer** field is significant

5.2 Sequence Numbers and ACKs

TCP uses a Go-Back-N error recovering approach with a little difference: the **sequence number** in a segment is the *byte stream number* of the first byte in the segment's data, because as we said TCP views data as an unstructured—but ordered—stream of bytes.

The **acknowledgement number** is instead the sequence number of the next byte it is expecting from the sender. Because TCP only acknowledges bytes up to the first missing byte in the stream, TCP is said to provide **cumulative acknowledgments**.
To minimize the possibility that a segment from a previous, terminated connection gets mistaken for a valid segment, *both sides of a TCP connection randomly choose an initial sequence number*.

TCP doesn't specify how the receiver must handle out-of-order segments, that is up to the implementor, it could either immediately discard out-of-order bytes or buffer them up until arrival of previous bytes—which is the approach taken in practice.

5.3 Round-Trip Time Estimation and Timeout (3.5.3)

Because TCP uses a timeout/retransmit mechanism to recover from lost segments, it needs to actually determine a suitable length for the timeout intervals, for example, it needs to be longer than the connection's RTT to avoid unnecessary retransmission.

This estimate is done circa once every RTT: TCP simply measures the time from the sending of a segment to the arrival of the corresponding ACK, not for every sent segment but only once at a time. Obviously this is done only on not-duplicate segments.

Because of the variation in the results TCP computes the **exponential weighted moving average** (**EWMA**):

$$\text{estimatedRTT}_{n+1} = (1-\alpha) \cdot \text{estimatedRTT}_n + \alpha \cdot \text{sampleRTT}_n$$

where sampleRTT_n is the last measured sampleRTT. The influence of past samples decreases exponentially fast.
A typical value for α is 0.125 ($\frac{1}{8}$). The bigger α the more responsive the estimate (but less stable) — when computing the first ERTT (ERTT_1) we simply set it equal to the first sample, which corresponds to $\alpha = 1$.

In addition to an estimate of the RTT, it is also valuable to hold an estimate of the variability of the RTT as a EWMA of the difference between SampleRTT and EstimatedRTT (the recommended value for β is 0.25):

$$\text{DevRTT} = (1-\beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

We can use this new value to determine a suitable timeout interval: since it must be greater than RTT and also depends on the variability of the latter we can compute it as $\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$. Note that an initial TimeoutInterval of 1 second is recommended in RFC 6298.
If a timeout occurs, the value of TimeoutInterval is doubled to avoid a premature timeout occurring for a subsequent segment that will soon be acknowledged. However as soon as a segment is received and EstimatedRTT is updated, the TimeoutInterval is again computed as above.

5.4 TCP reliable data transfer (3.5.4)

As we said the TCP protocol creates an RDT service on top of IP's unreliable service. For more info on RDT see 4 Reliable Data Transfer protocols (3.4), here we analyze TCP's RDT implementation.

EVENT AT RECEIVER	TCP RECEIVER ACTION
arrival of in-order segment with expected seq# . All data up to expected seq# is already ACKed	delayed ACK. Wait up to 500ms for the next segment. If no next segment, send ACK
arrival of in-order segment with the expected seq# . One other segment has pending ACK	immediately send a single cumulative ACK, ACKing both the in-order segments
arrival of out-of-order segment, higher than the expected seq# . *A Gap is detected*	immediately send a *duplicate* ACK, indicating the seq# of the next expected byte
arrival of segment that partially or completely fills the Gap	immediately send ACK, provided that the segment starts at the lower end of the gap

TCP uses a GBN error recovering approach, keeping a single timer for the oldest unACKed segment.
Let's see two common modifications to TCP.

5.4.1 Doubling the Timeout Interval

A modification that most TCP implementations employ is as follows: whenever the timeout event occurs, TCP retransmits the not-yet-acknowledged segment with the smallest sequence number, but each time it retransmits it sets the next timeout interval to twice the previous value. The timeout is then set again as we have seen before when a segment normally arrives. This modification provides at least some sort of congestion control.

5.4.2 TCP fast retransmit

If the sender receives 3 additional ACKs for the same data ("triple duplicate ACKs"), resend the unACKed segment with the smallest sequence number, because it likely got lost, this way we don't have to wait for the timeout.

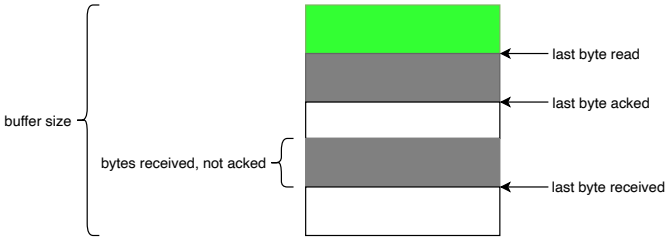
5.5 TCP flow control (3.5.5)

Recall that each host sets aside a receive buffer for the connection, when the TCP connection receives bytes that are correct and in-order it places them in the buffer. The associated application process will read the data from this buffer, but not necessarily immediately upon arrival. When the network layer delivers data faster than the speed at which the application layer removes data from the socket buffers we get an overflow and we can lose segments:

We need to introduce some sort of **flow control**: the receiver controls the sender, so that no **overflow** occurs: the TCP receiver advertises the free buffer space in the `rwnd` ("receive window") field in the TCP header (5.1 TCP segment structure (3.5.2)).
The sender limits the amount of unACKed ("in flight") data to the received `rwnd`, this way we guarantee that the buffer will not overflow: $\mathrm{LastByteSent} - \mathrm{LastByteAcked} \leq \mathrm{rwnd}$.
We compute `rwnd` as follows: $\mathrm{rwnd} = \mathrm{RcvBuffer} - (\mathrm{LastByteRcvd} - \mathrm{LastByteRead})$, indicating the spare room in the buffer.

The receiver application process sets `RcvBuffer` (the size of the receive buffer) via socket options (the default is typically 4096 bytes), many systems auto-adjust `RcvBuffer`.

There is an issue with this scheme though: suppose the buffer becomes full so that $\mathrm{rwnd} = 0$, after advertising $\mathrm{rwnd} = 0$ to the sender, the application slowly empties the buffer, but if the receiver has no segments to send to the sender, the latter will never know that there is room again in the receive window. To solve this issue the TCP specification requires the sender to continue to send segments with one data byte even while the receiver's receive window is 0. This way as soon as there is place in the receive buffer, the sender will know it and send new data.

> receiver buffer:
>
> 
>

5.6 TCP connection Management (3.5.6)

Before exchanging data, the sender and the receiver must perform a handshake: they agree to establish a connection and agree on the connection parameters (for example the starting sequence numbers).

5.6.1 The 3-way handshake

When the sender wants to initiate a new connection, it sends a TCP segment — with no application-layer data — with the SYN bit set; for this reason, this special segment is called the **SYN segment**. Furthermore the client chooses — randomly — an initial sequence number and includes it in the segment.

Once this SYN segment arrives at the receiver, it allocates the TCP buffers and variables to the connection, and sends a connection-granted segment to the sender TCP. This new segment still has the SYN bit set, acknowledges the arrived SYN segment and advertises the newly — randomly — chosen receiver initial sequence number. Because this segment purely exists to acknowledge the connection request, it is called a **SYNACK segment**.

Upon receiving the SYNACK, the sender also allocates buffers and variables to the connection and sends another TCP segment, this time with the SYN bit *unset* since the connection is already established — this new segment can already include application-layer data.

Note that if the initial sequence number was always the same, we would risk mixing segments from two different connections, the solution is to start with a random initial sequence number.

5.6.2 Closing a TCP connection

Closing a connection means deallocating the data needed for it — let us see how two hosts decide on doing it:

1. the client sends a TCP FIN control segment
2. the server receives the FIN and replies with an ACK
3. the server closes the connection and sends a FIN
4. the client receives the FIN, replies with an ACK and starts a **timed wait** (designed to close the connection shortly after the supposed arrival of the ACK to the server — typically 30 seconds)
5. the server receives the ACK. The connection is now closed.
   If the server doesn't receive the ACK (it timeouts), it resends the FIN, this is the reason of the timed wait of the client.

5.7 Congestion Control (3.6)

Congestion is informally defined as follows: too many sources are sending too much data too fast for the *network* to handle — at the router's input ports more datagrams per time unit are arriving than the router can send per time unit.
This may happen either because the router has a too small input capacity, or because of internal congestion.

We want to avoid congestion because it brings long delays (because of queueing in router's buffers) and packet loss (buffer overflow at routers).
When too many senders overflow routers with segments, delay rises and as soon as the buffer is full, packet loss happens — which leads to retransmission, making congestion worse.

The goal of congestion-control is to throttle the transmission rate just enough to avoid congestion.

5.7.1 Approaches to congestion control (3.6.2)

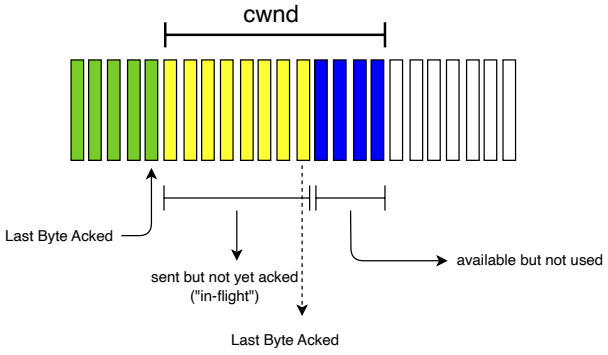
We can distinguish between congestion-control approaches based on whether or not the network layer provides explicit support to the transport layer for congestion-control purposes.

In **network-assisted congestion control** routers provide direct feedback to every sending and receiving host with flows passing through congested routers. This feedback may indicate the congestion level or explicitly set a sending rate.
This approach is implemented by: ATM, DECbit and TCP ECN protocols.
Routers provide senders with feedback either through **choke packets** or — more typically — by marking incoming packets with congestion notifications before forwarding them, when a host receives a marked packet it will tell the sender that the network is congested. This latter method of notification takes a full RTT.

The preferred approach is **end-to-end congestion control**: the network core does not provide direct feedback, congestion is ***inferred*** from observed loss and delay — this is the approach taken by TCP.

5.8 TCP Congestion Control (3.7)

TCP uses (and must use) **end-to-end congestion control**, since the IP layer provides no explicit feedback to the end systems regarding network congestion.
The approach taken by TCP is to have each sender limit the rate at which it sends traffic into its connection as a function of perceived network congestion.
To do this TCP keeps track of another variable: the **congestion window** (**cwnd**). This variable (and `rwnd`) imposes a limit to the amount of unACKnowledged data: $\text{LastByteSent} - \text{LastByteAcked} \leq \min\{\text{cwnd}, \text{rwnd}\}$. To simplify the discussion on congestion control we will assume `rwnd` to not be a problem and that the sender always has data to send.



Congestion is detected through packet loss: the sender learns that the network is congested either by receiving 3 duplicate ACKs from the receiver or through a timeout. In TCP delay in ACK receipt is only used in determining how much it should increase the congestion window when not having packet loss.
Since TCP uses ACKs to trigger its increase in congestion window size it is said to be **self-clocking**.

So to summarize, TCP increments its congestion window based on ACK receipt delay up until packet loss occurs, at which point it decreases its transmission rate: this approach is called **bandwidth probing**.

TCP Congestion control is made of three phases: (1) **slow start**, (2) **congestion avoidance** and (3) **fast recovery**.

TCP's congestion control's 3 phases

When a connection begins, the sending rate is increased *exponentially* until the first loss event:
we start with $\text{cwnd} = 1\text{MSS}$ (resulting in a sending rate of $\frac{MSS}{RTT}$) and `cwnd` is then *doubled* every RTT (if an ACK is received).

When TCP detects a packet loss, it sets the variable `ssthresh` to half the `cwnd` when the loss was detected, resets `cwnd` to 1 MSS and enters **slow start**.

Slow start goes as follows: starting from $\text{cwnd} = 1\text{MSS}$, `cwnd` gets doubled once every `RTT`. When `cwnd` gets to the threshold value of `ssthresh`, slow start stops and TCP transitions into **congestion avoidance**.

When TCP enters congestion avoidance `cwnd` is half the size of what it was when the last packet loss was detected, this means we could be approaching congestion again, thus we need to be more careful and cannot continue doubling `cwnd`.
During **congestion avoidance** TCP increases `cwnd` by `MSS` bytes every `RTT`, resulting in a linear increase. This can be done by increasing `cwnd` on every ACK receipt: if at some point we send `n` segments every `RTT`, on ACK receipt we must increase `cwnd` by $\frac{1}{n}$ MSS, resulting

in an increase of one `MSS` every `RTT`, up until a timeout happens (to which TCP responds by setting `ssthresh` at half the current value of `cwnd` and switching back to slow start) or a fast-recovery is triggered.

On triple duplicate ACKs receipt TCP sets `ssthresh` to half the value of `cwnd` at the time the duplicate ACKs were received, halves `cwnd` (and adds 3 `MSS`, accounting for the triple duplicate ACKs) and finally enters **fast-recovery**.

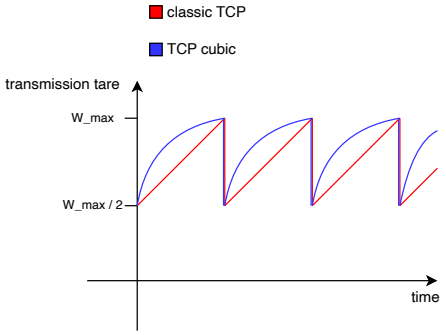
In **fast-recovery** the value of `cwnd` is increased by 1 `MSS` for every duplicate ACK received for the missing segment that caused the fast-recovery, up until receipt of an ACK for the missing segment, which triggers TCP into entering **congestion-avoidance** again.
If instead a timeout is detected TCP returns to **slow-start**.

AIMD: Additive increase, multiplicative decrease

TCP congestion control is often referred to as an **additive-increase, multiplicative-decrease** (**AIMD**) form of congestion control.
Additive increase: the sending rate is increased by 1 maximum segment size every RTT, until loss is detected.
Multiplicative Decrease: the sending rate is cut in half when loss is detected by triple duplicate ACK, otherwise the sending rate is cut to 1 MSS when loss is detected by timeout.
AIMD has been found to have desirable stability properties, is distributed and asynchronous.

5.8.2 TCP cubic

There is a better way to "probe" for usable bandwidth:
We define as $W_{\max}$ the congestion window size at which the congestion loss was detected, we suppose the congestion state of the bottleneck link probably hasn't changed much.
After a loss we cut as we said the window in half, then we initially ramp to $W_{\max}$ *faster* and we slow down the more we approach it.
This leads to the integral of the transmission rate over time in TCP cubic being larger than the same integral in classic TCP, this means in average more segments are transmitted in the same time, leading to TCP cubic being the default in linux and the most popular TCP for popular Web Servers.

> transmission rate over time:
>
> 
>

5.8.3 ECN: explicit congestion notification echo (3.7.2)

Modern TCP deployments often implement **network-assisted** congestion control, in the form of **explicit congestion notification**:

1. the ECN bits in the IP datagram header's ToS (Type of Service) field get marked by the network router to indicate congestion — the policy for determining congestion is chosen by the network operator.
2. This congestion indication is carried to the destination.
3. The receiver sets the **ECE** (**explicit congestion echo**) bit on the TCP ACK segment, to notify the sender of the congestion.
4. The sender on receipt of a set ECE bit, halves `cwnd` and sets `CWR` (**control window reduced**) in the next TCP segment header.

5.8.4 TCP fairness (3.7.1)

Since rate increase is linear and rate decrease is multiplicative, the rate at which n senders transmit segments through the same node gets slowly evened out to $\frac{\text{max node rate}}{n}$, this means that TCP, under idealized assumptions, is fair (these assumptions are: same RTT for every sender, and fixed number of sessions only in congestion avoidance).

Transport services and fairness

Since UDP does not provide flow and congestion control, apps using it can send data at a constant, desired rate — this is not fair.
TCP applications in response can open **multiple parallel connections** between two hosts. Because of this lots of network administrators limit the amount of parallel connections between two hosts.

6 Evolution of transport-layer functionality

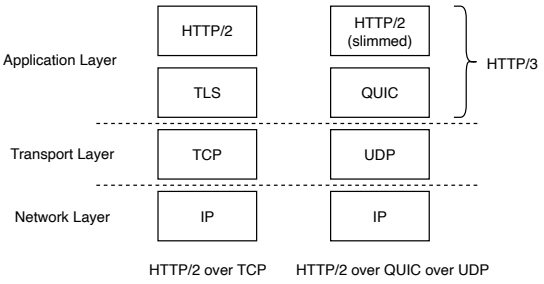
TCP and UDP have been the principal transport protocols for over 40 years. In this time the functionality of these protocols has been expanded with new application layer protocols.

6.1 QUIC: Quick UDP Internet Connections

This is an application-layer protocol on top of UDP.
It increases performance of HTTP by avoiding the TCP three way handshake and is deployed on many Google servers and apps.

QUIC employs:

- error and congestion control
- connection establishment: reliability, authentication, encryption
- multiple application-level "streams" multiplexed over a single QUIC connection

> TLS vs QUIC:
>
> 
>

Data Plane

#uni

- 1 Introduction
- 2 Network Layer: overview (4.1)
  - 2.1 Network service Model (4.1.2)
- 3 Routers (4.2)
  - 3.1 Forwarding
  - 3.2 Switching Fabrics
  - 3.3 Queueing
  - 3.4 Packet Scheduling
- 4 IP: the internet protocol (4.3)
  - 4.1 IPv4 Datagram format (4.3.1)
  - 4.2 Fragmentation (4.3.2)
  - 4.3 IPv4 Addressing (4.3.3)
    - 4.3.1 Obtaining an Address
  - 4.4 Network Address Translation (4.3.4)
  - 4.5 IPv6 (4.3.5)
    - 4.5.1 IPv6 datagram format
  - 4.6 Transition from IPv4 to IPv6
  - 4.7 ICMP protocol
- 5 Generalized Forwarding and SDN (4.4)
  - 5.1 OpenFlow
    - 5.1.2 OpenFlow abstraction
- 6 Middlebox
- 7 Where's the intelligence

1 Introduction

The Data Plane Functions of the **network layer** are the *per-router* functions in the network layer that determine how a datagram arriving on one of a router's input links is forwarded to one of that router's output links.

Traditionally, these control-plane routing protocols and data-plane forwarding functions have been implemented together, monolithically, within a router. **Software-defined networking** (**SDN**) explicitly separates the data plane and control plane by implementing these control plane functions as a separate service, typically in a **remote controller**.

2 Network Layer: overview (4.1)

This layer receives segments from the transport layer and transports them from the sending to the receiving host: the sender encapsulates segments into datagrams and passes them to the link layer, the receiver delivers these segments to the transport layer protocol.
Two important network-layer functions can be identified: **forwarding** (**data plane** — moving packets from a router's input link to the appropriate router output link) and **routing** (**control plane** — determining the route taken by packets in their path).

Switches are **link-layer devices**: they base their forwarding decision on values in the fields of the link-layer frame and the only function they offer is transferring a packet from an input link to an output link — so called **packet switches**.
Routers also implement the network layer, they are therefore called **network-layer devices**. A router examines the header fields in all IP datagrams that pass through it and according to these fields it moves these datagrams from input ports to the correct output ports. A key element in every router is its **forwarding table**, which determines which packet gets forwarded to which link.

2.1 Network service Model (4.1.2)

Services can be offered for *individual* datagrams, or for a *flow* of datagrams.
Here are some examples of services that the network layer *could* offer: guaranteed delivery, guaranteed delivery with a bounded delay, in-order packet delivery, guaranteed minimal bandwidth and security.

The **Internet Protocol** (**IP**) offers a **best effort service**: it gives no guarantees and does not offer security, in-order delivery or any other service.

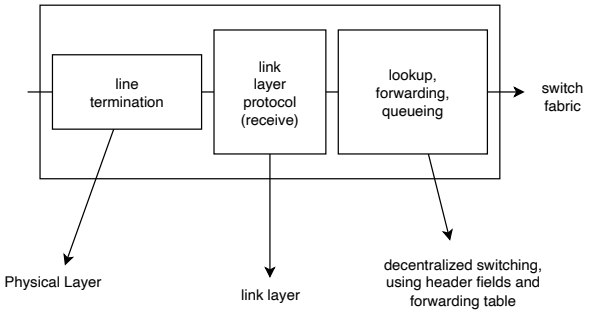
Interestingly, in spite of these well-developed alternatives, the Internet's basic best-effort service model combined with adequate bandwidth provisioning have arguably proven to be more than "good enough" to enable an amazing range of applications.

3 Routers (4.2)

Four router components can be identified:

- **routing processor**: responsible for the control plane (software): it can either execute routing protocols itself or communicate with the remote SDN controller. The processor also performs network management functions.
- **switching fabric**: responsible for the data plane (hardware): it connects input ports to output ports.
- **input ports**: these perform the necessary link-layer and physical-layer functions to receive packets from the incoming physical link, and they perform the lookup that determines the output port to which an arriving packet will be forwarded.
- **output ports**: these store packets and perform the necessary link-layer and physical-layer functions to transmit them onto the outgoing physical link.

router input port:



3.1 Forwarding

The forwarding can either be **destination-based** (the forwarding is based only on the destination IP address) or **generalized** (the forwarding is based on any set of header field values).

In destination-based forwarding, when looking for the entry of the forwarding table for a given destination address, we use the *longest* address prefix that matches the destination address.
This is often performed using **ternary content addressable memories** (**TCAM**s) — present the address to the TCAM and in one clock cycle retrieve the entry, regardless of table size.

3.2 Switching Fabrics

The switching Fabric transfers a packet from an input link to the appropriate output link. The **switching rate** is the rate at which packets can be transferred from input links to output links, it is often measured as a multiple of the input/output line rate (rate R).

There are 3 types of switching fabrics:

- **Switching via memory**: the switching is under direct control of the CPU. The packet is first copied to the system's memory. The speed is limited by the memory bandwidth.
- **Switching via bus**: **Bus contention**: packets are transferred from input ports to output ports over a shared bus: all ports receive the packet but only the right one will keep it. The switching speed is limited by the bus bandwidth — a 32 Gbps bus is sufficient for access routers.
- **Switching via interconnection network**: one way to pass the bandwidth limitation of a single bus is to utilize a crossbar switch consisting of $2N$ buses that connect N input ports to N output ports. Using this technology only the right port receives the packet and multiple packets can be forwarded in parallel, if the destination ports differ.

It is also possible to run multiple switching fabrics in parallel and break packets in smaller chunks, each traveling on a different switching fabric before getting reassembled into the original datagram at the end of the packet forward.

3.3 Queueing

Packet queues may form at both the input ports and the output ports and the extent of it will depend on **traffic load**, the **relative speed of the switching fabric** and the **line speed**.
Queueing can cause delay and packet loss due to buffer overflow.

If the switching fabric is slower than the input ports' combined throughput, queueing at the input ports may occur.
Another reason for input port queueing could be **Head of line (HOL) blocking**: the queued datagram at the front of the queue—maybe waiting for other packets destined to the same output port—prevents others in the queue from moving forward. This type of queue can unfortunately grow to unbounded lengths as soon as the packet arrival rate on the input links reaches only 58% of their capacity.

Buffering—a type of output queueing—instead happens when datagrams arrive from the fabric faster than the output link's transmission rate — if such a situation continues, the buffer will not have enough memory to store new incoming packets and a decision must be made to either drop the arriving packet (**drop-tail** policy) or remove one or more already-queued packets to make room for the new one. It is sometimes better to preemptively drop—or mark—a packet when nearing the limit, to signal congestion: policies employing this strategy are known as **active queue management** (**AQM**) algorithms.

The **scheduling discipline** chooses among the queued datagrams for transmission -> **priority scheduling**.

3.4 Packet Scheduling

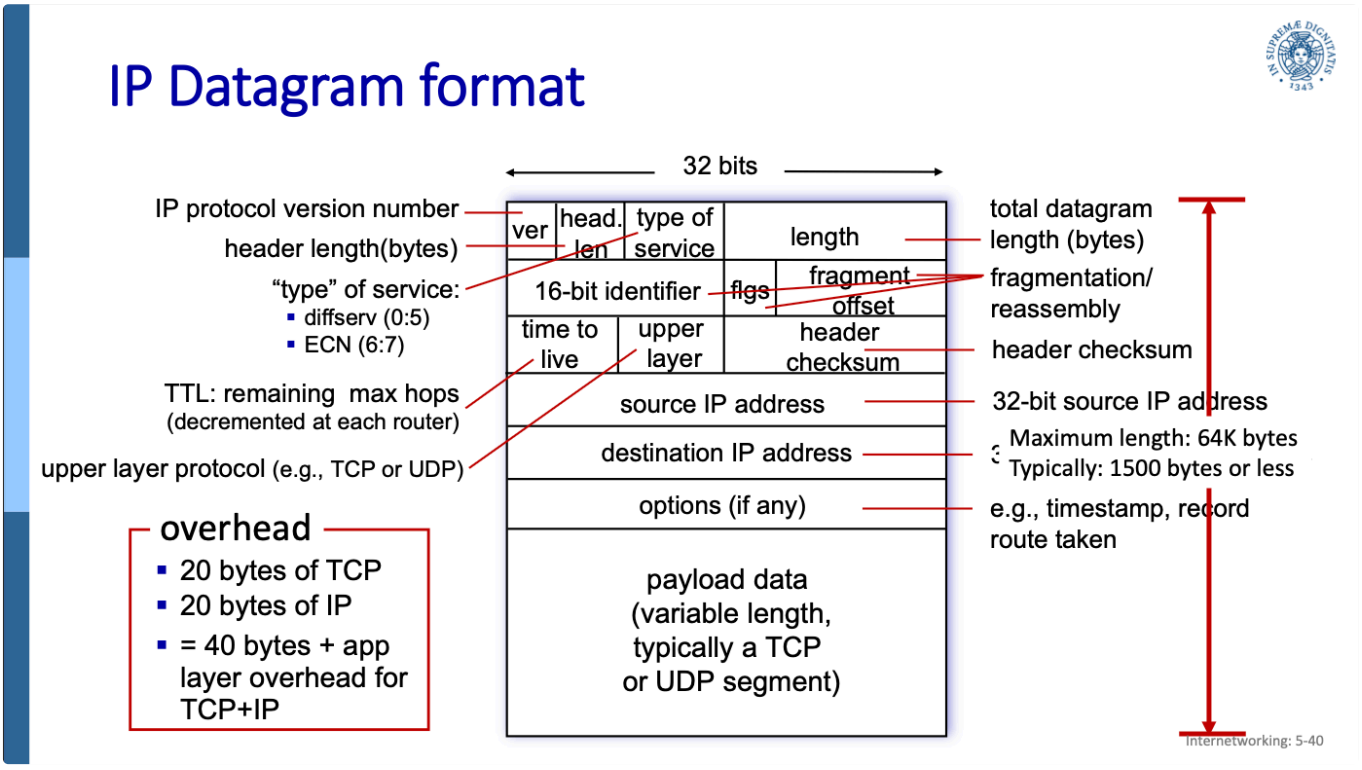
Packet scheduling is the problem of determining the order in which queued packets are transmitted over an outgoing link.
There are many possible policies for packet scheduling, here are some examples:

- **FCFS**/**FIFO**: Packets are sent in the same order they were buffered.
- **Priority**: The arriving traffic is classified and queued by class. Inside a priority queue, FCFS is used. A packet is always taken from the highest-priority queue that is not empty.
- **Round Robin**: The arriving traffic is classified and queued by class and the sending is alternated between classes. A so-called **work-conserving queueing** discipline will never allow the link to remain idle whenever there are packets of any class queued for transmission.
- **Weighted fair queueing** (**WFQ**): This is a generalized Round Robin, where the queues are not handled the same: every queue has a weight, that determines its available percentage of traffic (See Network Neutrality).

4 IP: the internet protocol (4.3)

The **Internet Protocol** (**IP**) is the network layer communications protocol in the Internet protocol suite for relaying datagrams across network boundaries. Its routing function enables **internetworking**, and essentially establishes the Internet.
IP has the task of delivering packets from the source host to the destination host solely based on the IP addresses in the packet headers. For this purpose, IP defines packet structures that encapsulate the data to be delivered. It also defines addressing methods that are used to label the datagram with source and destination information.
There are two versions currently in use: IP protocol version 4 and 6 — IPv4 and IPv6 for short.
The IP protocol uses as support ICMP (4.7 ICMP protocol): this handles error reporting and router signaling.

4.1 IPv4 Datagram format (4.3.1)



- **header length** in bytes: this is needed because the **options** field can be of variable length although not often used, leading to a typical IP datagram header length of 20 bytes.
- **type of service** (**ToS**) bits: these allow different types of IP datagrams to be distinguished from each other (without looking at the payload). It can for example be useful to distinguish real-time from non-real-time datagrams. Bits 0 to 5 represent diffserv (priority). Bits 6 and 7 are used for **ECN** (not really used) (see Transport Layer > 3.8.3 ECN explicit congestion notification echo (3.7.2)).
- **identifier**, **flags** (3 bits) and **fragment offset** (13 bits) all have to do with IPv4 fragmentation. Fragmentation does not exist in IPv6. Fragment Offset is the offset of the first byte of the fragment inside the original datagram, expressed in 8-byte units. `fragflag=1` means the current datagram is NOT the last fragment
- **TTL**: time to live, maximum number of forwards in routers before being dropped: decremented at each router.
- **upper layer protocol**: a value of 6 indicates TCP, 17 indicates instead UDP.
- **header checksum**: computed as the Internet Checksum (treating each 2 bytes in the header as a number and summing them using 1s complement arithmetic). Routers perform integrity checks on every incoming packet, when an error is detected the packet is dropped. Note that the checksum for a packet must be computed and updated at each hop, because IP header values can change in transit — TTL for example.
- **options**: this field allows IPv4 headers to be extended. Although meant to be used rarely, its sole existence causes problems: because of its variable length the exact start of the payload in a datagram varies, also since the use of options in datagrams varies, the processing

time of an IP datagram at routers can vary greatly. Because of these issues the options field is not present in IPv6 headers.
- **data**: this field usually contains the transport layer segment and is obviously of variable length, though it can carry other types of data, such as ICMP messages.

4.2 Fragmentation (4.3.2)

As we said, different network links have a different **MTU** (maximum transmission unit): the largest possible size for a link-level frame, typically 1500 bytes (for Ethernet for example).
When a router receives a datagram too big for the desired output link, it fragments it in smaller datagrams, which then get reassembled only at the final destination (this keeps complexity at the network's edge). If the next link has a smaller MTU, the new router has to again fragment the fragments.

If not every fragment arrives in a determined timeframe, every fragment is dropped and the original datagram is therefore lost.

This means that when a destination host receives a series of datagrams from the same source, it needs to determine whether any of these datagrams are fragments of a larger datagram and if so it needs to determine how these fragments must be put together again.
This is the reason the IPv4 header contains the fragmentation fields: **identification**, **flags** and **fragmentation offset**.
When a host creates a datagram, it inserts in it an identification number (typically incrementing it with each new datagram), together with source and destination IP addresses. When a router fragments a datagram, the new resulting datagrams all include this ID and addresses and the last fragment has a flag bit set to 0 (all the others have it set to 1). Furthermore every fragment of the datagram includes a fragmentation offset, that tells the host where the fragment fits within the original IP datagram.

4.3 IPv4 Addressing (4.3.3)

Every interface—the boundary between the host and the physical link—of a host or router is associated with an **IP address**, which is a 32-bit (4 bytes) number that uniquely (not actually, see 4.4 Network Address Translation (4.3.4)) identifies it on the internet.
IP addresses are usually written in so-called **dotted-decimal notation**, in which each byte of the address is written in its decimal form and is separated by a dot from other bytes in the address.

A portion of an interface's IP address is determined by the **subnet** to which it is connected. In particular, IP addresses have **subnet masks** which indicate how many of the first bits in the IP address are determined by the subnet: the `x` most significant bits in a `a.b.c.d/x` address constitute the network portion of the IP address and are often referred to as the **prefix** of the address.
The routers that are outside the destination subnet of a datagram only consider the prefix of the address, the remaining `32-x` bits compose the identifier of the interface in its subnet and are used by routers inside it.
This address assignment strategy is known as **Classless Interdomain Routing** (**CIDR**), before it there was an addressing scheme known as **classful addressing** which constrained the network portions of an IP address to be 8,16 or 24 bits, respectively **class** A,B and C. Classful addressing led to a quick depletion of IPv4 addresses because small sized organizations still had to use class B network portions, since class C ones could only support up to $2^8 - 2 = 254$ hosts (interfaces), obviously too little.

IPv4 offers a special type of address: the **broadcast address** `255.255.255.255`, datagrams with this destination address are delivered to every host on the same subnet — some routers also forward broadcast messages into neighboring subnets.

4.3.1 Obtaining an Address

The **ICANN** (Internet Corporation for Assigned Names and Numbers) allocates IP addresses, through 5 **regional registries** (RRs), who may then allocate to local registries.
These local registries assign blocks of IP address to ISPs, which in turn allocate a portion of their block to organizations.

Organizations can at this point finally assign IP addresses from their subnet to hosts, this can be done manually by network administrator but is usually done automatically through DHCP.

ICANN allocated the last chunk of IPv4 addresses to RRs in 2011...
NAT helps IPv4 address space exhaustion.
IPv6 has a 128 bit (instead of 32) address space.

DHCP

The **Dynamic Host Configuration Protocol** (**DHCP**) is a client-server protocol: when a host joins a network it can ask a DHCP server for a temporary IP address, this way IP addresses are dynamically and automatically assigned.
This assigned address is only held while connected to the network (IP addresses have a due date, so the hosts need to renew their lease), this leads to the possibility of reuse of IP addresses, so it is better for mobile users who often join and leave a network.
(See DHCP for more details on the interaction).

4.4 Network Address Translation (4.3.4)

With **Network Address Translation** (**NAT**) all devices in a local network share just **one** IPv4 address from the outside world point of view.
This means every datagram that leaves a local network that uses NAT has the same source NAT IP address, but different source port numbers.

Every device in a local network has a 32-bit address in a "private" IP address space, that can only be used in the local network, since these private IP address spaces only work in the local network and don't communicate directly with the outer internet, only through NAT.
These private IP address spaces are: `10.0.0.0/8`, `172.16.0.0/12` and `192.168.0.0/16`.

NAT offers several advantages: all devices in a local network can use a single IP address, the ISP can change without changing addresses of devices in the local network and **security**: devices inside a local network are not directly accessible by the outside world, only through the NAT router.

The NAT router must now keep a **NAT translation table** and do the following:

1. replace every source IP address and port number of every outgoing datagram with the NAT IP address and a new port number.
2. remember every `(source IP address, port number)` to `(NAT IP address, new port number)` translation pair.
3. for the incoming datagrams perform the inverse translation.

NAT is widely deployed but still has some issues (or critiques):

- port numbers should be used to identify processes, not hosts
- routers should only process up to layer 3, instead it rewrites layer 3 addresses and even layer 4 information such as port numbers
- address shortage should be solved by IPv6, NAT is only a bandaid
- it violates end-to-end argument, information should be exchanged end-to-end, nobody in the middle should change this information, instead the router does
- NAT traversal: what if a client wants to connect to a server behind NAT? We need to use **Port Forwarding**: we have to insert manually lines in the NAT table so that incoming messages directed to ports used by the server are directed to the server.

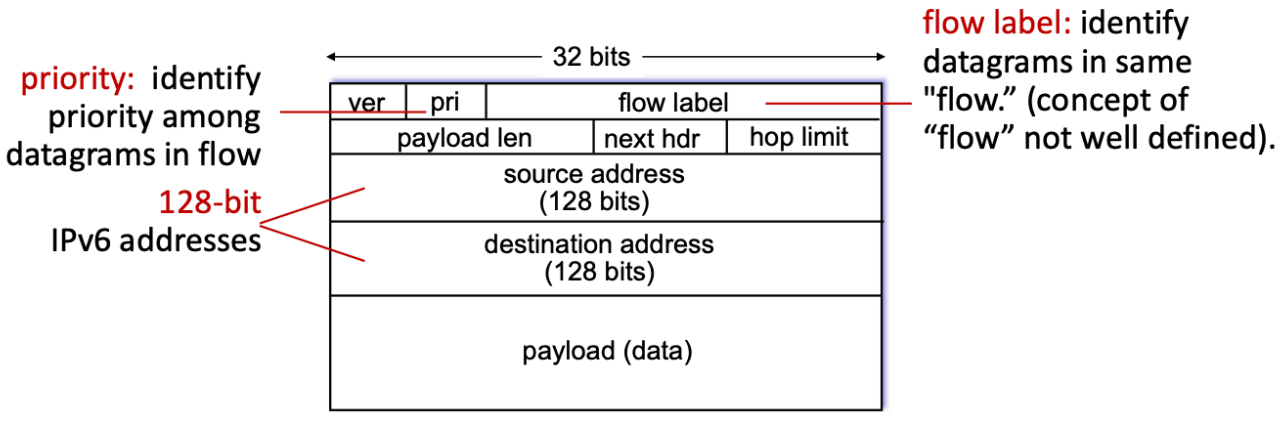
4.5 IPv6 (4.3.5)

IP protocol version 6 (**IPv6**) was introduced because the 32 bit address space was not enough (IPv6 uses 128 bits, allowing for many more addresses) and it was rapidly being all used up.
The designers took this opportunity to speed up processing/forwarding: instead of a variable length header, IPv6 uses a 40 byte fixed length header. It also enables different network-layer treatments of "flows".
(Why not IPv5? It was already taken by the Internet Stream Protocol, but it failed).

The most important changes introduced with version 6 are:

- 128-bit addresses and an **anycast address**: with IPv6 every grain of sand on planet Earth can be addressed. An anycast address allows a datagram to be delivered to *any one* of a group of hosts (typically the closest one).
- *fixed* 40 byte header.
- introduction of a 20-bit **flow label** which allows for labeling of packets belonging to particular flows for which the sender requests special handling, such as non-default quality of service or real-time service.

4.5.1 IPv6 datagram format

> IPv6 datagram format:
>
> 
>

These are the fields of this new fixed-length header:

- 20-bit **flow label**: this identifies to which "flow" a datagram belongs.
- 8-bit **traffic class** (or priority): this indicates the priority of this datagram inside its flow.
- hop limit: this is a different name for the TTL field but works exactly the same.
- **next header**: in IPv4 we had a field that indicated the upper layer protocol, this is the same thing: inside the payload there is the header for the transport layer protocol, the "next header" field indicates what this header inside the payload is.

This means that to speed up processing the following was removed: (1) the checksum (because it had to be computed at every hop and is redundant), (2) the options field (because of its variable length, in IPv6 this field is inside the payload, pointed to from the `next header` field)

and (3) fragmentation: if a datagram is too big for the next link, it gets discarded, leading to the need for a new type of message that can notify the sender of the MTU problem (when the sender receives this message tries again with smaller datagrams). For IoT instead we introduce a new kind of fragmentation, because these links typically have really low MTUs.

4.6 Transition from IPv4 to IPv6

Let's now consider a practical manner: how will the internet be transitioned from version 4 to version 6: new IPv6 capable systems can be made backward-compatible but already deployed IPv4 systems cannot handle IPv6 datagrams.
A **flag day** (a given time and date when all internet machines would be turned off and upgraded) is unthinkable because we now have billions of devices, so we must turn to **tunneling**: the IPv6 datagram gets carried as a payload in an IPv4 datagram among IPv4 routers ("packet within a packet"), these IPv4 routers between two IPv6 routers are referred to as a **tunnel**.
An IPv4 datagram carrying an IPv6 datagram in its payload is signaled by **protocol number 41**: this protocol number lets IPv6 nodes know that the original datagram must be extracted from the IPv4's payload.
Note that when an IPv6 node begins tunneling with another IPv6 node and creates an IPv4 datagram, it puts the IP address of the next IPv6 router in this datagram's destination address, not the original destination address.

4.7 ICMP protocol

The **Internet Control Message protocol** (**ICMP**) is used by hosts and routers to communicate *network-level* information.
The ICMP protocol is **above** the IP protocol: ICMP messages are carried *inside* IP datagrams, as IP payload.
But the IP protocol itself heavily relies on the ICMP protocol.

The ICMP messages contain:

- type
- code
- the header and the first 8 bytes of the IP datagram that caused the ICMP message

Type	Code	Description	notes
0	0	echo reply (ping)	
3	0	dest. network unreachable	
3	1	dest host unreachable	
3	2	dest protocol unreachable	
3	3	dest port unreachable	
3	6	dest network unknown	
3	7	dest host unknown	
4	0	source quench (not used)	not used anymore: used for rate control, for congestion ("you are going too fast, I'm gonna lose packets!")
8	0	echo request (ping)	
9	0	router advertisement	router announces itself as router
10	0	router solicitation	host asks routers to advertise themselves
11	0	TTL expired	when TTL (hop limit) expires
12	0	bad IP header	IP header can get corrupted -> checksum fails

5 Generalized Forwarding and SDN (4.4)

What we have seen until now can be named "Destination Based Forwarding": the only thing it cares about is the destination IP address.
Generalized Forwarding is instead a significantly more general "*match plus action*" approach: we can use every information included in a datagram and take different actions based on said information. Examples of actions can be: **forward**, **drop**, **copy**, **modify** and **log**. Since generalized forwarding is **software defined networking**, in this approach routers are better referred to as **packet switches**, with their forwarding tables being computed, installed, and updated by a remote controller.

5.1 OpenFlow

Our following discussion of generalized forwarding will be based on **OpenFlow**, a highly visible and successful standard that has pioneered the notion of the match-plus-action forwarding abstraction and controllers, as well as the SDN revolution more generally.

To implement this new approach we can't use forwarding tables anymore, we need a new, more powerful tool: the **flow table**.
A flow table's entry—also known simply as a **flow**—contains a **set of header field values** to which an incoming packet will be matched, a **set of counters** that are updated as packets are matched to flow table entries and a **set of actions to be taken** on match.

Flow table:

match	action
`src=*.*.*.*, dest=3.4.*.*`	`forward(2)`
`src=1.2.*.*, dest=*.*.*.*`	`drop`
`src=10.0.0.3, dest=*.*.*.*`	`send to controller`

Match

OpenFlow 1.0 packet matching fields can be grouped by layer: there are link layer fields (SRC MAC, DST MAC, ETH TYPE, VLAN ID, VLAN PRI), network layer fields (IP SRC, IP DST, IP PROTOCOL, IP TOS) and transport layer fields (TCP/UDP SRC PORT, TCP/UDP DST PORT). Obviously flows can have **wildcards** ($*$).

Action

Each flow table entry has a list of zero or more actions that determine the processing that is to be applied to a packet that matches a flow table entry. If there are multiple actions, they are performed in the order specified in the list.
The absence of actions in a flow table entry means the packet is to be dropped.

5.1.2 OpenFlow abstraction

This match+action abstraction unifies different kinds of devices:

device	match	action
router	longest destination IP prefix	forward out a link
switch	destination mac address	forward or flood
firewall	IP addresses and TCP/UDP port numbers	permit or deny
NAT	IP address and port	rewrite address and port

6 Middlebox

By the definition in RFC 3234: “**middlebox**: any intermediary box performing functions apart from normal, standard functions of an IP router on the data path between a source host and destination host”.
From this definition these are middleboxes:

- caches
- load balancers
- firewalls, IDS
- NAT
- application specific
- etc.

Middleboxes were initially proprietary hardware solutions, now there are programmable middleboxes: middleboxes that expose APIs.
We are moving towards software programmable networks.

7 Where's the intelligence

- Traditional phone net: intelligence and computing are in the center, at the network switches
- Internet (pre 2005): intelligence and computing are at the edge
- Internet (post 2005): now we have programmable network devices, intelligence and computing are both at the network switches and at the edge.

Control Plane

#uni

- 1 Introduction
- 2 Routing Algorithms (5.2)
  - 2.1 Routing protocols classification
  - 2.2 Link state Algorithms
    - 2.2.1 Dijkstra's algorithm
  - 2.3 Distance Vector Algorithm
- 3 Autonomous Systems (5.3)
  - 3.1 Intra-AS routing: OSPF
  - 3.2 Inter-AS routing: BGP (5.4)
    - 3.2.1 Advertising BGP Route Information (5.4.2)
    - 3.2.2 Determining the Best Routes (5.4.3)
    - 3.2.3 IP-Anycast (5.4.4)

1 Introduction

The **control plane** functions of the network layer compose the *network-wide* logic that controls (1) how a datagram is routed among routers along an end-to-end path from the source host to the destination host and (2) how network layer components and devices are configured and managed.

As we said, **software-defined networking** explicitly separates data plane and control plane: the **remote controller** interacts with a **control agent** (**CA**) in each router and manages its flow tables.
This is opposed to the traditional way where the control is per-router distributed: the routing algorithms run in each router instead of on separated controllers and the CA of each router communicates with other CAs.

2 Routing Algorithms (5.2)

Routing algorithms must determine "**good**" (least expensive, least congested, fastest) **paths** (sequence of routers) from the sender to the receiver, through the network of routers.

This problem is abstracted via **graphs**, where `G=(N,E)` is the graph of nodes `N` connected by the edges `(x,y)` in `E`, where each edge has a different `c(x,y)` **cost** (representing physical length, speed, etc.). Edges not in `E` are indicated by an infinite cost: $c(x,y)=\infty$, two connected nodes are said to be *neighbors*. We will only consider *undirected graphs*.
Today the most accurate way to determine the link's cost is based on the retransmission needed on said link.

$$N: \text{router nodes} = \{u,v,w,x,y,z\}$$
$$E: \text{links} = \{(u,v),(u,x),(v,x),\text{etc.}\}$$

2.1 Routing protocols classification

Routing Algorithms can be classified based on three criteria: (1) centralized versus decentralized, (2) static versus dynamic and (3) load-sensitive versus load-insensitive.

A **centralized routing algorithm** computes the least-cost path between source and destination using complete, global knowledge about the network. Algorithms with global state information are often referred to as **link-state (LS) algorithms**, since the algorithm must be aware of the cost of each link in the network.
In a **decentralized routing algorithm**, such as distance-vector algorithms, the calculation of the least-cost path is instead carried out in an *iterative*, *distributed* manner by the routers. No node has complete information about the costs of all network links. Instead, each node begins with only the knowledge of the costs of its own directly attached links. Then, through an iterative process of calculation and exchange of information with its neighboring nodes, a node gradually calculates the least-cost path to a destination or set of destinations.

In **static routing algorithms**, routes change very slowly over time, often as a result of human intervention (for example, a human manually editing link costs). **Dynamic routing algorithms** change the routing paths as the network traffic loads or topology change. A dynamic algorithm can be run either periodically or in direct response to topology or link cost changes.

In a **load-sensitive algorithm**, link costs vary dynamically to reflect the current level of congestion in the underlying link. If a high cost is associated with a link that is currently congested, a routing algorithm will tend to choose routes around such a congested link. Modern routing algorithms are load-insensitive.

2.2 Link state Algorithms

In practice the global knowledge of link costs and topology is accomplished by having each node broadcast link-state packets to *all* other nodes in the network, with each link-state packet containing the identities and costs of its attached links: we need a **link-state broadcast**

algorithm.

2.2.1 Dijkstra's algorithm

This algorithm computes the least expensive route between a source node and every other, this means it returns the **forwarding table** for the source node. This is an iterative algorithm: after k iterations we know the least expensive path from the source node to k destinations.
Normally Dijkstra's algorithm has $O(n^2)$ complexity, but with the aid of the *heap* we can get it as low as $O((n+e)\cdot \log n)$, where e is the number of edges.
We also need to take into account the broadcast phase, this has $O(n^2)$ complexity.

- $D(v)$: current cost estimate for the path between the source node and the destination v
- $p(v)$: predecessor node along the path to v
- N' : set of nodes for which the least expensive path is known

2.3 Distance Vector Algorithm

Whereas the LS algorithm is an algorithm using global information, the **distance-vector (DV)** algorithm is iterative, asynchronous, and distributed. It is *distributed* in that each node receives some information from one or more of its *directly attached* neighbors, performs a calculation, and then distributes the results of its calculation back to its neighbors. It is *iterative* in that this process continues on until no more information is exchanged between neighbors.

This algorithm is based on the **Bellman-Ford** (**BF**) equation (dynamic programming):

$$
\begin{gathered}
D_x(y) : \text{cost of the least-cost path from } x \text{ to } y. \\
\implies D_x(y) = \min_v\{c_{x,v} + D_v(y)\} \quad \forall y \in N \\
\min_v : \text{minimum taken between } x\text{'s neighbors } v \\
c_{x,v} : \text{direct link cost of } x, v \\
D_v(y) : \text{least-cost path from } v \text{ to } y \text{ estimate}
\end{gathered}
$$

The idea is each node begins with an estimate of the cost of the least-cost path from itself to every other node and broadcasts this vector of costs. When receiving everybody else's distance vector, it uses this new info to compute a better estimate of its distance vector with the Bellman-Ford equation, broadcasts this new version and so on.
In normal conditions, the $D_x(y)$ estimate converges to the least-cost $d_x(y)$.

Note that in distance vector algorithms **black-holing** is possible: routers can communicate incorrect path costs ("I have low cost paths to every node") and the error gets propagated along the network.

Poisoned reverse

It could happen that a node y doesn't know its cost to get to x, then it receives a distance vector from z in which z claims to have a path to x, but z has no idea that to do so its path passes through y. Now y will try to reach x through z, even though its direct link to x is cheaper, creating a cycle.

The solution is a technique called **poisoned reverse**:
If z passes through y to get to x:

1. z notifies y that its distance to x is ∞, i.e. $D_z(x) = \infty$, even if z knows this is not the case
    1. until z gets to x passing through y, it will continue to announce $D_z(x) = \infty$
2. y will think that z doesn't have a path to x, therefore it will not try to get to x through z and the cycle will not be created, leading to the costs being communicated correctly

Note that this technique works only for cycles between two adjacent nodes: it does not solve larger loops.

3 Autonomous Systems (5.3)

This view of the network as simply a collection of interconnected routers has a few problems:

- **Scale**: as the number of routers becomes large, the overhead involved in communicating, computing, and storing routing information becomes prohibitive.
- **Administrative autonomy**: the Internet is a network of ISPs, with each ISP consisting of its own network of routers. An ISP generally desires to operate its network as it pleases (for example, to run whatever routing algorithm it chooses within its network) or to hide aspects of its network's internal organization from the outside. Ideally, an organization should be able to operate and administer its network as it wishes, while still being able to connect its network to other outside networks.

Both of these problems can be solved by organizing routers into **autonomous systems (ASs)**, with each AS consisting of a group of routers that are under the same administrative control. Often the routers in an ISP, and the links that interconnect them, constitute a single AS. Some ISPs, however, partition their network into multiple ASs. An autonomous system is identified by its globally unique **autonomous system number** (**ASN**). AS numbers, like IP addresses, are assigned by ICANN regional registries.

Routers within the same AS all run the same routing algorithm and have information about each other. The routing algorithm running within an autonomous system is called an **intra-autonomous system routing protocol**.

3.1 Intra-AS routing: OSPF

Open Shortest Path First (**OSPF**) is widely used for intra-AS routing in the internet.
OSPF is a link-state protocol that uses flooding of link-state information and Dijkstra's least-cost path algorithm. With OSPF, each router constructs a complete topological map (that is, a graph) of the entire autonomous system. Each router then locally runs Dijkstra's shortest-path algorithm to determine a shortest-path tree to all *subnets*, with itself as the root node.
A router broadcasts routing information to all other routers in the AS, not just neighbors, whenever link state changes and also periodically, even if nothing changed.
The OSPF protocol also checks that links are operational (via a `HELLO` message that is sent to an attached neighbor) and allows an OSPF router to obtain a neighboring router's database of network-wide link state.

Individual link costs are configured by the network administrator. The administrator might choose to set all link costs to 1, thus achieving minimum-hop routing, or might choose to set the link weights to be inversely proportional to link capacity in order to discourage traffic from using low-bandwidth links; OSPF itself does not mandate a policy for how link weights are set.

OSPF offers: (1) **security** (exchanges between OSPF routers can be authenticated by using a password, either in plaintext or with MD5), (2) **Multiple same-cost paths** and (3) **integrated support for unicast and multicast routing**.
OSPF also enables **hierarchy within a single AS**: An OSPF autonomous system can be configured hierarchically into areas. Each area runs its own OSPF link-state routing algorithm. Within each area, one or more area border routers are responsible for routing packets outside the area. Lastly, exactly one OSPF area in the AS is configured to be the **backbone area**. The primary role of the backbone area is to route traffic between the other areas in the AS. The backbone always contains all area border routers in the AS and may contain non-border routers as well. Inter-area routing within the AS requires the packet to pass through the backbone to change area.

3.2 Inter-AS routing: BGP (5.4)

When routing a packet between a source and destination within the same AS, the route the packet follows is entirely determined by the intra-AS routing protocol. However, to route a packet across multiple ASs, say from a smartphone in Timbuktu to a server in a datacenter in Silicon Valley, we need an **inter-autonomous system routing protocol**. Since an inter-AS routing protocol involves coordination among multiple ASs, communicating ASs must run the same inter-AS routing protocol. In fact, in the Internet, all ASs run the same inter-AS routing protocol, called the **Border Gateway Protocol** (**BGP**).

In BGP, packets are not routed to a specific destination address, but instead to CIDRized prefixes, with each prefix representing a subnet or a collection of subnets.

BGP provides each router a means to **obtain prefix reachability information from neighboring ASs** and **determine the best routes to the prefixes**.

3.2.1 Advertising BGP Route Information (5.4.2)

For each AS, each router is either a **gateway router** or an **internal router**. In BGP, pairs of routers exchange routing information over semi-permanent TCP connections using port 179: a so-called **BGP connection**. A BGP connection that spans two or more ASs is called an **external BGP** (eBGP) connection, while one internal to a single AS is called an **internal BGP** (iBGP) connection.
Gateway routers advertise prefixes inside their AS to other gateway routers in other ASs over eBGP connections. The receiving gateway routers then propagate this info to all routers inside their AS over iBGP connections. At this point gateway routers propagate this AS route for the initially advertised prefix to other ASs (like `intermediateAS initialAS prefix`).

3.2.2 Determining the Best Routes (5.4.3)

When a router advertises a prefix across a BGP connection, it includes with the prefix several **BGP attributes**. In BGP jargon, a prefix along with its attributes is called a **route**. Two of the more important attributes are AS-PATH and NEXT-HOP.

The **AS-PATH attribute** contains the list of ASs through which the advertisement has passed. To generate the AS-PATH value, when a prefix is passed to an AS, the AS adds its ASN to the existing list in the AS-PATH.
BGP routers also use the AS-PATH attribute to detect and prevent looping advertisements; specifically, if a router sees that its own AS is contained in the path list, it will reject the advertisement.

The **NEXT-HOP attribute** is the *IP address of the router interface that begins the AS-PATH*.

Hot Potato Routing

Hot Potato Routing is one of the simplest routing algorithms for inter-AS routing: the route chosen (from among all possible routes) is that route with the least cost to the NEXT-HOP router beginning that route.
The idea behind hot-potato routing is for the gateway router to get packets out of its AS as quickly as possible (more specifically, with the least cost possible) without worrying about the cost of the remaining portions of the path outside of its AS to the destination: we therefore say that hot potato routing is a selfish algorithm.

Route-Selection Algorithm

In practice, BGP uses an algorithm that is more complicated than hot potato routing, but nevertheless incorporates hot potato routing.
For any given destination prefix, the input into BGP's route-selection algorithm is the set of all routes to that prefix that have been learned and accepted by the router. If there is only one such route, then BGP obviously selects that route. If there are two or more routes to the same prefix, then BGP sequentially invokes the following elimination rules until one route remains:

1. A route is assigned a **local preference** value as one of its attributes. The local preference of a route could have been set by the router or could have been learned from another router in the same AS. The value of the local preference attribute is a policy decision that is left entirely up to the AS's network administrator. The routes with the highest local preference values are selected.
2. From the remaining routes (all with the same highest local preference value), the route with the shortest AS-PATH is selected.
3. From the remaining routes, hot potato routing is used.
4. If more than one route still remains, the router uses BGP identifiers to select the route.

3.2.3 IP-Anycast (5.4.4)

BGP is often used to implement the IP-anycast service, commonly used in DNS.
During the IP-anycast configuration stage, the CDN company assigns the *same* IP address to each of its servers, and uses standard BGP to advertise this IP address from each of the servers. When a BGP router receives multiple route advertisements for this IP address, it treats these advertisements as providing different paths to the same physical location (when, in fact, the advertisements are for different paths to different physical locations). When configuring its routing table, each router will locally use the BGP route-selection algorithm to pick the "best" (for example, closest, as determined by AS-hop counts) route to that IP address.

For example when a client requests the video, the CDN returns to the client the common IP address used by the geographically dispersed servers, no matter where the client is located. When the client sends a request to that IP address, Internet routers then forward the request packet to the "closest" server, as defined by the BGP route-selection algorithm.

In practice CDNs generally choose not to use IP-anycast because BGP routing changes can result in different packets of the same TCP connection arriving at different instances of the Web server.
But IP-anycast is extensively used by the DNS system to direct DNS queries to the closest root DNS server.

Link Layer

#uni

- 1 Introduction
- 2 Error-Detection and Correction (EDC) Techniques
  - 2.1 Parity Checking
  - 2.2 Checksumming (6.2.2)
    - 2.2.1 Internet Checksum
  - 2.3 Cyclic Redundancy Check (CRC) (6.2.3)
- 3 Multiple Access Links and Protocols (6.3)
  - 3.1 MAC protocols taxonomy
    - 3.1.1 Channel partitioning Protocols
    - 3.1.2 Random Access Protocols
    - 3.1.3 Taking-Turns Protocols
- 4 LANs
  - 4.1 MAC addresses
    - 4.1.1 Address Resolution Protocol (ARP)
  - 4.2 Ethernet
  - 4.3 Physical topology for LANs
  - 4.4 Ethernet frame structure
  - 4.5 Ethernet CSMA/CD protocol
- 5 Link Layer Switches
  - 5.1 Ethernet Switch
    - 5.1.1 Forwarding and Filtering
    - 5.1.2 Self-Learning
    - 5.1.3 Multiple Simultaneous Transmissions
- 6 Virtual LANs (VLANs)
  - 6.1.1 VLAN trunking and 802.1Q Ethernet
- 7 Link Virtualization (6.5)
  - 7.1 MPLS
- 8 Wide-Area Packet Switched Networks (WANs)
  - 8.1 Connection Oriented Packet Switched WANs
- 9 Datacenter Networks

1 Introduction

In discussing the link layer, we'll see that there are two fundamentally different types of link-layer channels.
The first type is **broadcast channels**, which connect multiple hosts in wireless LANs, satellite networks, and hybrid fiber-coaxial cable (HFC) access networks. Since many hosts are connected to the same broadcast communication channel, a so-called medium access protocol is needed to coordinate frame transmission.
The second type of link-layer channel is the **point-to-point communication link**, such as that often found between two routers connected by a long-distance link, or between a user's office computer and the nearby Ethernet switch to which it is connected.

In the following chapter we will refer to any device that runs a link-layer as a **node** and to the communication channels that connect adjacent nodes as **links**. Over a given link, a transmitting node encapsulates the datagram in a **link-layer frame** and transmits the frame into the link.

Services Provided by the Link Layer (6.1.1)

We said the basic service of any link layer is to move a datagram from one node to an adjacent node over a single communication link. The link layer can also offer a variety of extra services, here follow some examples:

- **Framing**
- **Link Access**: A medium access control (MAC) protocol specifies the rules by which a frame is transmitted onto the link.
- **Reliable Delivery**
- **Error detection and correction**

Where the Link Layer is implemented (6.1.2)

The data link layer is implemented in a **network interface card** (**NIC**), also called a **network adapter**, or on a chip, in each-and-every host. This NIC or chip attaches to the host's system buses. Because of the need for a NIC, the data link is a mix of hardware and software. Ethernet, wifi card or chip implement both the data link layer and the physical layer.

The sending side encapsulates a datagram in a frame and adds error checking bits, reliable data transfer, flow control etc. and then transmits it onto the physical link. The link-layer controller on the receiving side looks for errors, manages flow control and extracts the datagram and passes it to the upper layer.

2 Error-Detection and Correction (EDC) Techniques

Typically, the data to be protected includes not only the datagram passed down from the network layer for transmission across the link, but also link-level addressing information, sequence numbers, and other fields in the link frame header.
The receiver tries to detect corrupted bits, at which point it can try to automatically correct the bits or ask the transmitter to re-send the corrupted frames.

Keep in mind that a link can be:

- **Half-Duplex**: nodes at both ends of link can transmit, but not at the same time: achieved over a physical link by putting at both end a **transceiver**, which is both a transmitter and a receiver.
- **Full-Duplex**: we put two lines of physical link, one is used to transmit by one end, the other line is used to transmit by the other end.

The link-layer frame includes **R**, the error detection bits (redundancy bits) typically included in the **trailer** (because these are calculated while transmitting the message), and **D**, the data protected by error checking, which may include header fields.
What we do is apply a certain *mathematical operation* to the data, and include the result in the encoded message.
When the receiver gets the message, it applies the same operation to the D data bits, and then checks if the resulting redundancy bits are the same as the received redundancy bits.
So D and R are the sent bits and D' and R' are the received bits. If the result of the operation on D' is different from the received R' bits, the message changed and we have N errors. This could mean the D' bits are wrong, or the R' bits are wrong.

Here follow some of the commonly used operations.

2.1 Parity Checking

This method detects single bit errors.
Only one bit, the **parity bit**, is added as trailer to the D bits.
This parity bit can either be set when there is an even number of 1's, or an uneven number of 1's.

Parity checking can only detect single-bit errors, this means that if two bits flip, the error goes undetected.
If the probability of bit errors is small and errors can be assumed to occur independently from one bit to the next, the probability of multiple bit errors in a packet would be extremely small. In this case, a single parity bit might suffice. However, measurements have shown that, rather than occurring independently, errors are often clustered together in “**bursts**".

What we can do to mitigate this problem is divide the D bits into rows and columns, and compute the parity bit for each row and for each column. With this **two-dimensional parity** scheme, the parity of both the column and the row containing the flipped bit will be in error. The receiver can thus not only *detect* the fact that a single bit error has occurred, but can use the column and row indices of the column and row with parity errors to actually identify the bit that was corrupted and *correct* that error.
A single error in the parity bits themselves is also detectable and correctable. Two-dimensional parity can also detect (but not correct) any combination of two errors in a packet.
The ability of the receiver to both detect and correct errors is known as **forward error correction** (**FEC**) and is particularly valuable because it (1) decreases the number of sender retransmissions required and (2) allows for *immediate* error correction, avoiding the wait for the RTT propagation delay needed for retransmission.

2.2 Checksumming (6.2.2)

In checksumming techniques, the d bits of data are treated as a sequence of k-bit integers, then some sort of sum is applied to these integers and the result is included in the trailer.
The **Internet Checksum** is the most commonly used for internet applications.

Because transport-layer error detection is implemented in software, it is important to have a simple and fast error-detection scheme such as checksumming. On the other hand, error detection at the link layer is implemented in dedicated hardware in adapters, which can rapidly perform more complex (and useful) operations.

2.2.1 Internet Checksum

The sender treats the package as a sequence of 16-bit integers and sums them, the 1s complement of this sum then forms the internet checksum that is carried in the segment header.
The receiver itself calculates the internet checksum and compares it to the received one in the checksum field, if these are not equal an error is detected.
In the TCP and UDP protocols, the Internet checksum is computed over all fields (header and data fields included). In IP the checksum is computed over the IP header (since the UDP or TCP segment has its own checksum).

2.3 Cyclic Redundancy Check (CRC) (6.2.3)

Cyclic Redundancy Check (**CRC**) is a more powerful—and complex—error detecting technique.

Let **D** be the string of d data bits and **G** be a bit pattern (**generator**) of r+1 bits agreed upon in advance by both sender and receiver (the MSB of G needs to be a 1). For a given piece of data, *D*, the sender will choose *r* additional bits, *R*, and append them to *D* such that the resulting *d+r* bit pattern $D+R$ (interpreted as a binary number) is exactly divisible by *G* (i.e., has no remainder) using modulo-2 arithmetic. The process of error checking with CRCs is thus simple: the receiver divides the *d+r* received bits by *G*. If the remainder is nonzero, the receiver knows that an error has occurred; otherwise the data is accepted as being correct.
All CRC calculations are done in modulo-2 arithmetic without carries in addition or borrows in subtraction. This means that addition and subtraction are identical, and both are equivalent to the bitwise exclusive-or (XOR) of the operands.
Therefore the formula for the *d+r* bit pattern is $D \cdot 2^r \text{ XOR } R$. We can calculate *R* as $R = (D \cdot 2^r) \bmod G$.

This operation can detect all burst errors of length at most r bits (i.e. less than $r+1$) and any *odd* number of bit errors (the latter if G contains $x+1$ as a factor).
Widely used in practice, in Ethernet, 802.11 Wifi and more.

3 Multiple Access Links and Protocols (6.3)

As we said in the 1 Introduction, links can be point-to-point links or **broadcast links**: multiple sending and receiving nodes all connected to the same, single, shared broadcast channel.
Using a single shared Broadcast channel leads to **collisions**: a node receives two or more signals at the same time.
The problem of how to coordinate the access of multiple sending and receiving nodes to a shared broadcast channel is known as the **multiple access problem**, which to be resolved requires a **multiple access protocol**: an algorithm that determines how nodes share a channel.

Consider a multiple access channel (**MAC**) of rate R bps, an ideal protocol should have the following characteristics:

1. when only one node wants to transmit, it can send at rate R.
2. *fair*: when M nodes want to transmit, each can send at an average rate of R/M.
3. *fully decentralized*, without a SPF.
4. *simple* (KISS, keep it simple, stupid), so that it is inexpensive to implement.

3.1 MAC protocols taxonomy

We can classify just about any multiple access protocol as belonging to one of three categories: **channel partitioning protocols**, **random access protocols**, and **taking-turns protocols**.

3.1.1 Channel partitioning Protocols

Channel partitioning MAC protocols divide the channel into smaller pieces (time slots, frequency, code), then allocate each piece for exclusive use of a node.

In **time division multiple access** (**TDMA**) the channel is accessed in "rounds": each station gets a fixed length slot in each round (length = packet transmission time).
In **frequency division multiple access** (**FDMA**) the channel is instead divided in different frequency slots.

Channel partitioning is appealing because it eliminates collisions and is perfectly fair: each node gets a dedicated transmission rate of R/N bps. However, it has two major drawbacks. First, a node is limited to an average rate of R/N bps even when it is the only node with packets to send. A second drawback in TDMA is that a node must always wait for its turn in the transmission sequence.

A third channel partitioning protocol is **code division multiple access (CDMA)**. CDMA assigns a different *code* to each node. Each node then uses its unique code to encode the data bits it sends. If the codes are chosen carefully, CDMA networks have the wonderful property that different nodes can transmit *simultaneously* and yet have their respective receivers correctly receive a sender's encoded data bits (assuming the receiver knows the sender's code). CDMA has been used in military systems for some time (due to its anti-jamming properties) and now has widespread civilian use, particularly in cellular telephony.

3.1.2 Random Access Protocols

In a random access protocol, a transmitting node always transmits at the full rate of the channel. When there is a collision, each node involved in the collision repeatedly retransmits its frame until it gets through without a collision. But when a node experiences a collision, it doesn't necessarily retransmit the frame right away. *Instead it waits a random delay before retransmitting the frame*. Each node involved in a collision chooses independent random delays.

A random Access MAC protocol must specify:

1. how to detect collisions
2. how to recover from collisions

Slotted ALOHA

In the following description we make these assumptions: all frames are of exactly L bits, time is divided into equal size slots (the time it takes to transmit 1 frame), nodes start to transmit only at the beginning of a slot, *nodes are synchronized* (**slotted**) and, finally, if two or more nodes transmit in the same slot all nodes detect the collision.

In **slotted aloha** when a node obtains a fresh frame it transmits it in the next available slot: if there's no collision the node can send also the new frame in the next slot.
If, instead, there's a collision, the node at every new slot retransmits the frame with probability p, until it actually sends it: hopefully the two colliding nodes "choose" different slots to retransmit and all goes well.

The main advantage of slotted aloha over channel partitioning is, when only one node desires to transmit, it can do so at full link throughput. It is also decentralized—apart from the synchronization—and extremely simple.

Best value for p

Suppose: N nodes with many frames to send, each transmits in a slot with probability p.

- probability that a given node has a successful transmission in a slot: $p(1-p)^{N-1}$
- probability that any node has success: $Np(1-p)^{N-1}$
- the value of p that maximizes $Np(1-p)^{N-1}$ is $\overline{p} = 1/N$
  - for a large N : $\lim_{N\to\infty} N\overline{p}(1-\overline{p})^{N-1} = \lim_{N\to\infty}\left(1-\frac{1}{N}\right)^{N-1} = 1/e \approx 0,37$
  - so the maximum efficiency (fraction of slots with a successful transmission) is $1/e \approx 0,37$

This means the actual best throughput one can achieve with slotted aloha is 0.37 times the maximum rate for the link.

(Unslotted) ALOHA

Designed by professor Norman Abramson. Aloha means "hello" in Hawaiian.
Unslotted aloha is the original version of the aloha protocol, that doesn't need synchronization between nodes:
When a node obtains a fresh frame it transmits it immediately, the probability of collision increases without synchronization, in the slotted version only a full collision can happen, in the unslotted version only a small part of the frame may collide and invalidate the whole frame:
- a frame sent at t_0 collides with any other frame sent in $[t_0-1, t_0+1]$, where time is measured in frame transmission times

Best value for p

Redoing the math we find that the maximum efficiency is $1/(2e) \approx 0,18$, much worse than in Slotted ALOHA.

CSMA

Carrier sense multiple access (**CSMA**)'s underlying idea is to listen to the channel before transmitting, so as to not interrupt others.
Problem is collisions can still occur with carrier sensing: the propagation delay means two nodes may not hear each other's just started transmission.

We need to introduce **collision detection**: in **CSMA/CD** nodes detect when they have collided and immediately abort their transmissions.
Collision detection is easy with a wired channel, difficult in wireless.

Algorithm

1. NIC receives a datagram from the network layer, and creates a frame.
2. if the NIC senses that the channel is idle it starts frame transmission, otherwise, if it is busy, it waits until the channel is idle, then transmits.
3. if the NIC transmits the entire frame without collision, the NIC is done with the frame and the transmission is considered successful.
4. if the NIC detects another transmission while sending, it aborts and sends a **Jam** signal (a signal stronger than normal transmissions)
5. after aborting, the NIC enters the ***binary exponential backoff***: after the m th collision, the NIC chooses a K at random from $\{0, 1, 2, ..., 2^m - 1\}$ (in ethernet, m is capped at 10). The NIC waits $K \cdot 512$ bit times, then returns to `step 2`.

This leads CSMA/CD to have an efficiency of $E = \frac{1}{1+5\frac{d_{prop}}{d_{trans}}}$.

3.1.3 Taking-Turns Protocols

Taking-turn protocols coordinate access to a shared channel by giving devices opportunities to transmit. Some assign turns explicitly, such as polling or token passing; others let devices contend for access.

Polling Protocol

In the **polling protocol** a master node polls each of the nodes in a round-robin fashion: it lets each of them know when and for how many frames they can transmit over the shared channel. The key difference between polling and TDMA is here no slots get wasted, and the initial delay due to the wait for the assigned slot is diminished, although not eliminated.

Polling presents the following problems: polling overhead, latency and SPF.

Token Ring

In the **token ring protocol** the control **token** is passed from one node to the next sequentially. When a node receives the token it can transmit on the channel up to a certain maximum number of frames, otherwise it must wait for its turn. When a node with the token finishes its transmission (or it has no data to transfer when the token arrives) it gives up the token to the next node in line.

The token ring protocol is highly decentralized but still presents problems: a node could have problems with the release of the token, causing the halt of transmission for the whole channel.
One solution could be to have the node tasked with emitting the token in the network at some point recognize the failure and emit another token in the network, restoring functionality.

4 LANs

Local Area Networks (**LANs**) are a broadcast medium that utilize **MAC protocol** addressing, where MAC stands for **Medium Access Control**.
These cover a limited area (home, building, uni campus) and have a high bit rate (100Mbps - 10 Gbps).

4.1 MAC addresses

A **MAC address** is a **48 bit** (for most LANs) address burned in the **NIC** ROM (sometimes software settable), it is used "locally" to get a frame from one interface to another physically-connected interface. "MAC addresses" are also known as "LAN addresses" and "Physical Addresses", all these terms refer to **link-layer addresses**. MAC addresses are typically expressed in hexadecimal notation, where each byte is expressed as a pair of hexadecimal numbers.
Every interface in a LAN has a unique MAC address and a **locally** unique 32-bit IP address — apart from link-layer switches, which do not have link-layer addresses associated with their interfaces.
The MAC address allocation is administered by the IEEE: every manufacturer buys a chunk of the MAC address space, where each chunk is of 2^{24} addresses, to assure uniqueness. This means when the IEEE allocates a chunk for a company, the first 24 bits are fixed, and the company can change the other 24. Although MAC addresses were designed to be permanent, it is now possible to change an adapter's MAC address via software.

When an adapter wants to send a frame to some destination adapter, the sending adapter inserts the destination adapter's MAC address into the frame and then sends the frame into the LAN. An adapter may receive a frame that isn't addressed to it. Thus, when an adapter receives a frame, it will check to see whether the destination MAC address in the frame matches its own MAC address. If there is a match, the adapter extracts the enclosed datagram and passes the datagram up the protocol stack. If there isn't a match, the adapter discards the frame, without passing the network-layer datagram up, without interrupting the destination host's work. Note that it exists a special **MAC broadcast address**: 48 consecutive 1s, i.e. `FF-FF-FF-FF-FF-FF`.

4.1.1 Address Resolution Protocol (ARP)

Because there are both network-layer addresses and link-layer addresses , there is a need to translate between them, enter the **Address Resolution Protocol** (**ARP**).
An ARP module in a host takes any IP address *on the same LAN* as input, and returns the corresponding MAC address.
Each host and router has an **ARP table** in its memory, which contains mappings of IP addresses to MAC addresses, with their relative TTL (typically 20 minutes).

Resolving IP addresses in the same LAN

If the ARP table does not contain a certain mapping for a desired destination IP address, the sender uses the ARP protocol to resolve the address. First, the sender constructs a special packet called an **ARP packet**. An ARP packet has several fields, including the sending and receiving IP and MAC addresses. The purpose of the ARP query packet is to query all the other hosts and routers on the subnet to determine the MAC address corresponding to the IP address that is being resolved. The sender passes an ARP query packet to the adapter along with an indication that the adapter should send the packet to the MAC broadcast address. The adapter encapsulates the ARP packet in a link-layer frame, uses the broadcast address for the frame's destination address, and transmits the frame into the subnet.
The frame containing the ARP query is received by all the other adapters on the subnet, and (because of the broadcast address) each adapter passes the ARP packet within the frame up to its ARP module. Each of these ARP modules checks to see if its IP address matches the destination IP address in the ARP packet. The one with a match sends back to the querying host a response ARP packet with the desired mapping. Now the sender updates its ARP table with this newly found info and can finally send its original datagram to the now-known destination MAC address.
Note that both ARP query and response packets have the same format.

Sending a datagram off the Subnet

When the sender tries to send a datagram to a host in another subnet, what MAC destination address should it insert in the frame header? The answer is the MAC address of the first-hop router on the path to the destination. This first receiving NIC (the one on the first-hop router) recognizes its MAC address on the frame and therefore passes the included datagram to the network layer, which in turn encapsulates it in a frame with the MAC address of the next-hop router as MAC destination address, and so on and so forth.
Obviously whenever a MAC address is missing, ARP is used.

4.2 Ethernet

The **Ethernet protocol** is the dominant, and first widely used, technology for wired LAN, this because it is fast and a single ethernet NIC supports multiple speeds.
Ethernet is both a link-layer *and* a physical-layer specification and is carried over a variety of physical media including coaxial cable, copper wire, and fiber.

4.3 Physical topology for LANs

The first widely used physical topology for LANs was a **bus**: all nodes were attached to a single shared coaxial cable, so they all belonged to the same collision domain.
Starting in the mid-1990s, Ethernet installations moved to a hub-based star topology.
A **hub** is a physical-layer device that acts on individual bits rather than frames. When a bit, representing a zero or a one, arrives from one interface, the hub simply re-creates the bit, boosts its energy strength, and transmits the bit onto all the other interfaces. In particular, if a hub receives frames from two different interfaces at the same time, a collision occurs and the nodes that created the frames must retransmit. Note that a hub-based star is still a single collision domain, logically equivalent to a bus.

In the early 2000s Ethernet installations continued to use a star topology, but the hub at the center was replaced with a layer-2 **switch**, which is not only “collision-less” but is also a bona-fide store-and-forward packet switch — this is a **switched** topology.

4.4 Ethernet frame structure

The sending interface encapsulates the IP datagram (or some other network layer protocol's packet) in an **Ethernet Frame**:

```
| preamble | dest.address | src.address | type | data(payload) | CRC |
```

- **Preamble**: used to synchronize the receiver and the sender clock rates. It is made of 7 bytes of 10101010 (`0xAA`) followed by one byte of 10101011 (`0xAB`). This is needed because although two NICs would like to transfer at a certain speed, nothing is absolutely perfect and the sending adapter will not transmit the frame at exactly the target rate; there will always be some *drift* from the target rate, a drift which is not known *a priori* by the other adapters on the LAN. A receiving adapter can lock onto adapter A’s clock simply by locking onto the bits in the first 7 bytes of the preamble.
- **Source and Destination addresses**: these are 6 byte - 48 bit MAC addresses. If the receiver's adapter receives a frame with its destination address, or with the broadcast address, it passes the data in the frame to the network layer.
- **type**: this field indicates the higher layer's protocol, mostly IP, and it is used to demultiplex up at the receiver (for example ARP uses a value of `0x0806` for this field).
- **data**: the dimension of the payload must be between 46 bytes and 1500 bytes (MTU), this field carries the IP datagram. If the datagram is too big it is fragmented by the network layer, if instead it is too small **stuffing** (padding) is added to reach the 46-byte minimum: the padding is not counted as data, since the receiving network layer strips it using the length field of the IP header.
- 4 bytes of 2.3 Cyclic Redundancy Check (CRC) (6.2.3). Note that although Ethernet uses CRC, it is NOT reliable, since it doesn't use ACKs — when the receiving NIC receives a frame that fails the CRC check, it simply discards it.

4.5 Ethernet CSMA/CD protocol

Ethernet uses **unslotted CSMA/CD with binary backoff** as its MAC protocol:

1. The NIC receives the data from the network layer and creates a frame
2. if the NIC senses that the channel is idle for **96 bit times** (the interframe gap) it starts the transmission of the frame, otherwise it waits
3. if the NIC transmits the entire frame without collision, the NIC is done with the frame
4. if the NIC detects another transmission while transmitting it aborts and sends a **48 bit** jam signal
5. After aborting, the NIC enters **exponential backoff**:
   after the i -th collision:
   1. the NIC chooses K at random from $\{0, 1, 2, ..., 2^m - 1\}$, where $m = \min(i, 10)$.
   2. the NIC sets a backoff timer at $k \cdot 512 \text{ bit times}$
   3. the NIC waits until the backoff timer expires
   4. NIC returns to step 2

- After 17 trials the frame is dropped.
- $\text{bit time} = 0,1\mu\text{sec}$ for 10 Mbps ethernet

5 Link Layer Switches

The role of the switch is to receive incoming link-layer frames and forward them onto outgoing links, the switch itself is **transparent** to the hosts and routers in the subnet.
Switches' output interfaces may have buffers, much like routers.

To prevent the cycling of broadcast frames, the active topology of a switched network is restricted to a spanning tree. Also, a large switched network would require large ARP tables in the hosts and routers and would generate substantial ARP traffic and processing. Furthermore, switches are susceptible to broadcast storms—if one host goes haywire and transmits an endless stream of Ethernet broadcast frames, the switches will forward all of these frames, causing the entire network to collapse.
With routers instead packets are not restricted to a spanning tree and can use the best path between source and destination. Because routers do not have the spanning tree restriction, they have allowed the Internet to be built with a rich topology.

5.1 Ethernet Switch

The switch is a **link layer** device, so it takes an active role: it stores and forwards ethernet frames.

- it examines the incoming frame's MAC address and selectively forwards the frame to one or more outgoing links, if the frame is to be forwarded on the segment.
- it uses CSMA/CD to access the segment
- switches are transparent: the hosts are unaware of the presence of the switches
- switches are plug-and-play and self-learning

5.1.1 Forwarding and Filtering

Filtering is the switch function that determines whether a frame should be forwarded to some interface or should just be dropped. **Forwarding** is the switch function that determines the interfaces to which a frame should be directed, and then moves the frame to those interfaces. Switch filtering and forwarding are done with a **switch table**. The switch table contains entries for some, but not necessarily all, of the hosts and routers on a LAN. An entry in the switch table contains (1) a MAC address, (2) the switch interface that leads toward that MAC address, and (3) the time at which the entry was placed in the table.

When a frame is received at the switch:

1. the switch records incoming link and MAC address of the sending host.
2. indexes the switch table using the MAC destination address.
3. **if** the entry is found for the destination and the destination is on the interface from which the frame arrived **then** drop the frame.
4. **else if** the entry is found and there is no loop: forward the frame on the interface indicated by the entry
5. **else** (entry not found) flood: forward on all interfaces except the one the frame arrived from.

5.1.2 Self-Learning

We said the switch is self-learning, because it *learns* which hosts can be reached through which interfaces and builds its switch table by itself, without manual intervention needed: when a frame is received, the switch learns the location (connected interface) of the sender (sender's MAC address) and it records the sender/location pair in the switch table (together with a timestamp). This property means switches—as opposed to routers—are **plug-and-play**.

5.1.3 Multiple Simultaneous Transmissions

Every input port of the switch has a dedicated direct connection to every output port of the switch. The switches can buffer the packets.
The ethernet protocol is used on each incoming link, so there are no collisions and **full duplex** is supported: each link is its own collision domain.
All of this means that on a single switch there can be N simultaneous transmissions (as long as every host included in said transmissions is in only one transmission).

6 Virtual LANs (VLANs)

Modern institutional LANs are often configured hierarchically, with each workgroup (department) having its own switched LAN connected to the switched LANs of other groups via a switch hierarchy. While such a configuration works well in an ideal world, three drawbacks can be identified: **lack of traffic isolation**, **inefficient use of switches**, **bad user management** — as LAN sizes grow we get efficiency, security and privacy issues.
We also wish to limit the broadcast to a subsection of a LAN.

Each of these difficulties can be handled by a switch that supports **virtual local area networks** (**VLAN**s). As the name suggests, a switch that supports VLANs allows multiple *virtual* local area networks to be defined over a single *physical* local area network infrastructure. Hosts within a VLAN communicate with each other as if they (and no other hosts) were connected to the switch.
In a **port-based VLAN**, the switch's ports (interfaces) are divided into groups by the network manager. Each group constitutes a VLAN, with the ports in each VLAN forming a broadcast domain.
It is also possible to define VLANs based on MAC addresses: the network manager specifies the set of MAC addresses that belong to each VLAN. VLANs can also be defined based on network-layer protocols (e.g., IPv4, IPv6, or Appletalk) and other criteria.

We now have to make different VLANs able to communicate. One way could be to connect the switch ports of different VLANs to the same router, which will route the packets from different VLANs, this approach is called **router-on-a-stick**.

6.1.1 VLAN trunking and 802.1Q Ethernet

A more scalable approach to interconnect VLAN switches is known as **VLAN trunking**: a **Trunk Port** belongs to all VLANs and carries frames between VLANs defined over multiple physical switches.
We need a way to tell switches to what VLAN packets belong to, we cannot use normal Ethernet 802.1 frames, they must carry VLAN ID info.
The **802.1Q** frame consists of the standard Ethernet frame with a four-byte **VLAN tag** added into the header that carries the identity of the VLAN to which the frame belongs. The VLAN tag consists of a 2-byte **Tag Protocol Identifier** (**TPID**) field (with a fixed hexadecimal value of 81-00), a 2-byte **Tag Control Information** field that contains a 12-bit **VLAN ID field** (**VID**), and a 3-bit **priority field** that is similar in intent to the IP datagram TOS field.
The VLAN tag is added into a frame by the switch at the sending side of a VLAN trunk, parsed, and removed by the switch at the receiving side of the trunk.

7 Link Virtualization (6.5)

Virtualization is a powerful abstraction even in Computer Networks: in a **Virtual Link** the path from one node to another is regarded as a point-to-point virtual link, just like a physical point-to-point link.

7.1 MPLS

Multiprotocol Label Switching (**MPLS**) is a packet-switched virtual-circuit network.
MPLS's goal is not to abandon the destination-based IP datagram-forwarding infrastructure for one based on fixed-length labels and virtual circuits, but to augment it by selectively labeling datagrams and allowing routers to forward datagrams based on fixed-length labels (rather than destination IP addresses) when possible. Importantly, this technique works hand-in-hand with IP, using IP addressing and routing.

A link-layer frame transmitted between MPLS-capable devices has a small MPLS header added between the layer-2 header and layer-3 header. RFC 3032 defines the format of the MPLS header for such links. The most important field, obviously, is the **label**.
An MPLS-capable router is often referred to as a **label-switched router**, since it forwards an MPLS frame by looking up the MPLS label in its forwarding table and then immediately passing the datagram to the appropriate output interface.

8 Wide-Area Packet Switched Networks (WANs)

In **Wide-Area Packet Switched Networks** nodes get identified through a unique address, similar to the ethernet MAC address.
These types of WANs can be:

- **connectionless**: each packet is managed on an individual basis and does not arrive in order — also known as **datagram service**.
- **connection-oriented**: a **virtual circuit** is preliminarily established, after that all packets follow the same path and arrive in order

8.1 Connection Oriented Packet Switched WANs

In order to have a connection-oriented communication, we first need to establish (and define) the connection or, in our case, the **virtual circuit**.

Virtual Circuits

A virtual circuit is a path from source to destination described by **VC numbers**, one number for each link along the path. Formally a VC is made of entries in the forwarding tables of the routers (or switches) along the path.
A packet belonging to a VC carries the VC number, rather than the destination address. The VC number can be changed on each link and the new VC number comes from the forwarding table.

Incoming interface	incoming VC #	outgoing interface	outgoing VC #
...	..	..	..

The switches must maintain some information on the state of the connection.

9 Datacenter Networks

Datacenters are locations with thousands of hosts in close proximity, that are not intended for use of clients, at least not directly.
Some challenges one can face when working with datacenter networks are:

- multiple applications, each serving a potentially massive number of clients
- reliability
- load balancing and load managing
- avoid all kinds of bottlenecks (processing, networking, etc.)

Datacenter networks, top to bottom:
0. Border routers: these interconnect the tier-1 switches and connect the datacenter to the internet.

1. tier-1 switches: connect to (~16) Tier-2 switches below
2. tier-2 switches: connect to (~16) TOR switches below
3. Top of Rack (**TOR**) switches: these switches sit on top of server racks and connect to the server blades (the host).
4. Server Racks: inside a server rack are usually 20-40 server blades, these are the hosts.

Wireless and Mobile Networks

#uni

- 1 Introduction
  - 1.1 Characteristics of different wireless links
- 2 Wireless Links and network characteristics (7.2)
  - 2.1 WiFi: 802.11 Wireless LANs (7.3)
    - 2.1.1 The 802.11 WLAN Architecture (7.3.1)
    - 2.1.2 802.11 AP association
    - 2.1.3 The 802.11 MAC protocol
    - 2.1.4 The IEEE 802.11 Frame (7.3.3)
    - 2.1.5 Mobility within the same subnet (7.3.4)
    - 2.1.6 Advanced Capabilities (7.3.5)
  - 2.2 Wireless LANs: LiFi: Light Fidelity
  - 2.3 Wireless PANs: Bluetooth (7.3.6)
  - 2.4 Cellular networks: 4G and 5G (7.4)
    - 2.4.1 Elements of the 4G LTE architecture (7.4.1)
    - 2.4.2 LTE mobiles: sleep modes
    - 2.4.3 5G (7.4.6)
- 3 Mobility Management (7.5)
  - 3.1 Mobility management
    - 3.1.1 Mobility with indirect routing
    - 3.1.2 Mobility with direct routing
  - 3.2 Mobility's impact on higher-layer protocols (7.7)

1 Introduction

A **wireless host** connects to a **base station** via a **wireless link**. A base station is responsible for sending and receiving data to and from a wireless host that is associated with that base station. When we say a wireless host is “associated” with a base station, we mean that (1) the host is within the wireless communication distance of the base station, and (2) the host uses that base station to relay data between it (the host) and the larger network. **Cell towers** in cellular networks and **access points** in 802.11 wireless LANs are examples of base stations. Hosts associated with a base station are often referred to as operating in **infrastructure mode**, since all traditional network services are provided by the network to which a host is connected via the base station. In **ad hoc networks**, wireless hosts have no such infrastructure with which to connect and must provide for services by themselves. The **network infrastructure** is the larger network with which a wireless host may wish to communicate.

There are many more wireless phone subscribers than wired phone subscribers and more mobile-broadband-connected devices than fixed-broadband-connected devices.
We need to face two different challenges:

- wireless: communication over wireless link
- mobility: handling the mobile user who changes point of attachment to the network

1.1 Characteristics of different wireless links

Different wireless links have different characteristics, e.g., link bandwidth, range, power consumption.
WiFi is the commercial name for the 802.11 technology and Bluetooth is the commercial name for the 802.15.1 technology: short range, low energy, "high" (not by modern standards) bandwidth.

Technology	Data Rate	Range	Environment
802.11ax	14 Gbps	10–200m	Indoor – Outdoor
5G	10 Gbps	10m–4Km	Indoor – Midrange outdoor
802.11ac	3.5 Gbps	10–200m	Indoor – Outdoor
802.11n	600 Mbps	10–200m	Indoor – Outdoor
802.11af, 802.11ah	35–560 Mbps (af); 347 Mbps (ah)	200m–4Km	Midrange outdoor
4G LTE	600 Mbps	4Km–15Km	Long range outdoor
802.11g	54 Mbps	10–30m	Indoor
802.11b	11 Mbps	10–30m	Indoor
Bluetooth	2 Mbps	10–30m	Indoor

Wireless Networks Taxonomy

	Single Hop	Multiple Hops
Infrastructure-based: the base station connects mobiles into a wired network	Host connects to base station which connects to the larger internet: WiFi, cellular networks	Host may have to relay through several wireless nodes to connect to the larger internet: sensor networks
Ad Hoc: there are no base stations and the nodes organize themselves into a network	No base station, no connection to the larger internet: Bluetooth	No base station, no connection to larger internet. May have to relay each other through a given wireless node: MANET, VANET

2 Wireless Links and network characteristics (7.2)

Wireless brings important differences from wired links:

- **decreased signal strength**: the electromagnetic signal attenuates as it propagates through matter (**path loss**). This also causes the **hidden node problem** (i.e., we have 3 nodes A, B, and C: B hears both, A and C only hear B, this means from B's point of view A and C collide, but A and C do not know about each other).
- **interference from other sources**: wireless network frequencies get shared by many devices and also get interference from electromagnetic noise within the environment.
- **multipath propagation**: a radio signal can reflect off objects and ground, taking paths of different lengths and arriving at the destination in multiple instances.

To facilitate the discussion on signal strength, let's introduce the **signal to noise ratio** (**SNR**): the bigger this value the easier it is to extract the signal from noise, which brings a lower **bit error rate** (**BER**). Note that SNR may change with mobility leading us to having to dynamically adapt the physical layer (modulation technique, rate et cetera).

Bit errors will be more common in wireless links than in wired links. For this reason, it is perhaps not surprising that wireless link protocols employ not only powerful CRC[1] error detection codes, but also link-level reliable-data-transfer protocols that retransmit corrupted frames.

2.1 WiFi: 802.11 Wireless LANs (7.3)

There are several 802.11 standards for wireless LAN technology in the IEEE 802.11 (“WiFi”) family, each of these standards uses CSMA/CA for multiple access and shares the same link-layer frame structure. They also all have infrastructure-based and ad-hoc modes.

IEEE standard	Year	Max data rate	Range	Frequency
802.11b	1999	11 Mbps	30m	2.4 GHz
802.11g	2003	54 Mbps	30m	2.4 GHz
802.11n (WiFi 4)	2009	600 Mbps	70m	2.4, 5 GHz
802.11ac (WiFi 5)	2013	3.47 Gbps	70m	5 GHz
802.11ax (WiFi 6)	2020	14 Gbps	70m	2.4, 5 GHz
802.11be (WiFi 7)	2024	46 Gbps	70m	2.4, 5, 6 GHz
802.11af	2014	35 - 560 Mbps	1Km	unused TV bands (54-790 MHz)
802.11ah	2017	347 Mbps	1Km	900 MHz

2.1.1 The 802.11 WLAN Architecture (7.3.1)

In **infrastructure mode** (aka "cell") the **Basic Service Set** (BSS) includes an **Access Point** (**AP**) and one or more **wireless stations**.
In **ad hoc mode** the Basic Service Set includes only wireless hosts.
As with Ethernet devices, each 802.11 wireless station has a MAC address, stored in the firmware of the station's network adapter, i.e., the 802.11 network interface card.

Wireless LANs that deploy APs are often referred to as **infrastructure wireless LANs**, with the “infrastructure” being the APs along with the wired Ethernet infrastructure that interconnects the APs and a router.

IEEE 802.11 stations can also group themselves together to form an **ad hoc network**—a network with no central control and with no connections to the “outside world.” Here, the network is formed “on the fly,” by mobile devices that have found themselves in proximity to each other, that have a need to communicate, and that find no preexisting network infrastructure in their location.

2.1.2 802.11 AP association

802.11 operates in the frequency range of 2.4 GHz to 2.4835 GHz, within this 85 MHz band, 802.11 defines **11 partially overlapping channels**. Any two channels are non-overlapping if and only if they are separated by four or more channels.
When a network administrator installs an AP, the administrator assigns a one- or two-word **Service Set Identifier** (**SSID**) to the access point. The administrator must also assign a channel number to the AP.

The 802.11 standard requires that an AP periodically send **beacon frames**, each of which includes the AP's SSID and MAC address. Your wireless device scans the 11 channels, seeking beacon frames from any APs that may be out there (some of which may be transmitting on the same channel). Having learned about available APs from the beacon frames, you (or your wireless device) select one of the APs for association.
A **WiFi jungle** is any physical location where a wireless station receives a sufficiently strong signal from two or more APs.

The arriving host must **associate** with an AP: it scans the channels, listening for **beacon frames** (which are periodically generated by APs), containing the AP's **SSID** (the **Service Set IDentifier** is the "name" of the access point) and the MAC[2] address.
The host selects an AP to associate with, it then may perform authentication and run DHCP to get an IP address in the AP's subnet.

802.11 does not specify an algorithm for selecting which of the available APs to associate with, this is up to the designers of the firmware and software in the wireless device. Note that the decision cannot only be made on signal strength, since the selected AP may have a strong signal but may be overloaded with other affiliated devices, while another AP could be unloaded because of the weaker signal.

The way of discovering APs that we described is called **passive scanning**: listening for beacon frames periodically generated by APs.
A wireless device can also perform **active scanning**, by broadcasting a probe frame that will be received by all APs within the wireless device's range, which will respond to the probe request frame with a probe response frame. The wireless device can then choose the AP with which to associate from among the responding APs. After selecting the AP with which to associate, the wireless device sends an association request frame to the AP, and the AP responds with an association response frame. Note that this second request/response handshake is needed with either passive or active scanning.
In order to create an association with a particular AP, the wireless device may be required to authenticate itself to the AP. 802.11 wireless LANs provide a number of alternatives for authentication and access. One approach, used by many companies, is to permit access to a wireless network based on a device's MAC address. A second approach, used by many Internet cafés, employs usernames and passwords. In both cases, the AP typically communicates with an authentication server, relaying information between the wireless device and the authentication server using a protocol such as RADIUS or DIAMETER. Separating the authentication server from the AP allows one authentication server to serve many APs, centralizing the decisions of authentication and access within the single server, and keeping AP costs and complexity low.

2.1.3 The 802.11 MAC protocol

Once a wireless device is associated with an AP, it can start sending and receiving data frames to and from the access point. But because multiple wireless devices, or the AP itself may want to transmit data frames at the same time over the same channel, a multiple access protocol is needed to coordinate the transmissions.
See Link Layer > 3 Multiple Access Links and Protocols (6.3) for more information on MAC (medium access control) protocols.

Although both Ethernet and 802.11 use **carrier-sensing multiple access** (CSMA), the two MAC protocols have important differences. First, instead of using collision detection, 802.11 uses **collision-avoidance** (CSMA/CA) techniques, because the transmitting signal is much stronger than the received signal (from the transmitter's point of view), which means hardware able to detect collision would be very costly and there still would be undetectable collisions due to the hidden terminal problem. Second, because of the relatively high bit error rates of wireless channels, 802.11 uses a link-layer acknowledgment/retransmission (**ARQ**) scheme.
Since 802.11 doesn't use collision detection, when a device begins transmitting a frame, it transmits the *entire* frame.

IEEE 802.11 MAC protocol: CSMA/CA

The sender:

1. If initially the station senses the channel idle, it transmits its *entire* frame after a short period of time known as the **Distributed Inter-frame Space (DIFS)**.
2. If it senses the channel busy, it defers until the channel is sensed idle for a DIFS period, then enters binary exponential backoff:
    1. It starts a random backoff timer and counts it down *only while the channel is sensed idle* (freezing when busy and resuming after the channel is idle for DIFS).
    2. When the counter reaches zero, the station transmits the entire frame and then waits for an acknowledgment.
3. When the destination station receives a frame that passes the CRC, it waits a short period of time known as the **Short Interframe Space** (**SIFS**) and then sends back an **ACK**.
4. If the transmitting station does not receive an acknowledgment within a given amount of time, it assumes that an error has occurred and retransmits the frame, using the CSMA/CA protocol to access the channel. If an acknowledgment is not received after some fixed number of retransmissions, the transmitting station gives up and discards the frame.

Because $SFS < DFS$, a destination node waiting to send an ACK (or a node responding to an RTS with a CTS) only waits SFS and will always gain control of the medium before any other node waiting DFS can begin backoff or transmission. This guarantees collision-free ACK/CTS control frame delivery.

Backoff Algorithm

- Backoff interval = a slotted random time with uniform distribution in $[0, CW - 1]$
- Contention window **CW**:
    - initially $CW = CW_{\min}$
    - when an ACK is missed: $CW = 2 \cdot CW$ (capped at $CW_{\max}$)

- when an ACK is received: CW is reset to $CW_{\min}$
- $CW_{\min}$ and $CW_{\max}$ are MAC parameters depending on the physical layer

The solution to the Hidden Terminal

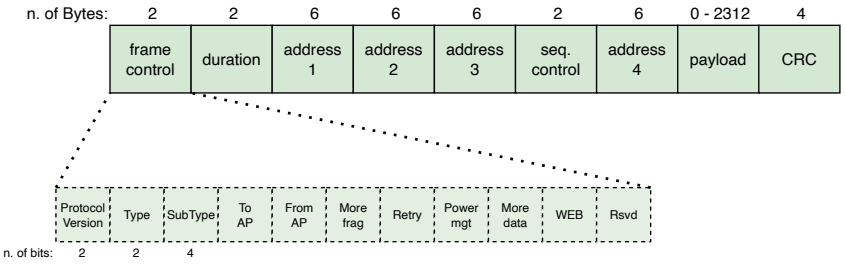
The 802.11 MAC protocol also includes a nifty (but optional) reservation scheme that helps avoid collisions even in the presence of hidden terminals.

The sender reserves a channel for data frames using small reservation control frames:

1. the sender first transmits a small **Request-To-Send** (**RTS**) control frame to the AP using CSMA. This frame indicates the total time required to transmit the DATA frame and the ACK frame.
2. On receipt of the RTS frame, the AP broadcasts a **Clear-To-Send** (**CTS**) frame in response to the RTS.
3. the CTS is heard by all nodes: the sender transmits the data frame and the other stations defer transmissions.

Although the RTS/CTS exchange can help reduce collisions, it also introduces delay and consumes channel resources. For this reason, the RTS/CTS exchange is only used (if at all) to reserve the channel for the transmission of a long DATA frame. In practice, each wireless station can set an RTS threshold such that the RTS/CTS sequence is used only when the frame is longer than the threshold. For many wireless stations, the default RTS threshold value is larger than the maximum frame length, so the RTS/CTS sequence is skipped for all DATA frames sent.

2.1.4 The IEEE 802.11 Frame (7.3.3)

> the 802.11 Frame:
>
> 
>

- **addresses**: three address fields are typically needed for internetworking purposes—specifically, for moving the network-layer datagram from a wireless station through an AP to a router interface.
  - address 1 (receiver): the MAC address of the wireless host or AP directly receiving this frame over the wireless link.
  - address 2 (transmitter): the MAC address of the wireless host or AP transmitting this frame over the wireless link.
  - address 3: MAC address of the *router interface* to which the AP is attached (acts as the destination MAC address when transmitting to the router, and as the source MAC address when receiving from the router).
  - address 4: used only in WDS (Wireless Distribution System) / bridge mode when APs forward frames to each other over the wireless medium.
- **payload**: typically consists of an IP datagram or an ARP packet. Although the field is permitted to be as long as 2,304 bytes, it is typically fewer than 1,500 bytes.
- **sequence number**: the use of sequence numbers allows the receiver to distinguish between a newly transmitted frame and the retransmission of a previous frame.
- **duration**: indicates the time duration (in microseconds) required for the channel to complete the current frame exchange sequence, including SIFS and expected response frames (such as an ACK or CTS frame).
- **frame control**: The **type** and **subtype** fields are used to distinguish the association, RTS, CTS, ACK, and data frames. The **to** and **from** fields are used to define the meanings of the different address fields. (These meanings change depending on whether ad hoc or infrastructure modes are used and, in the case of infrastructure mode, whether a wireless station or an AP is sending the frame.) Finally the WEP field indicates whether encryption is being used or not.

Addressing

When sending from the router to a wireless host, through the AP, the router sends an Ethernet frame addressed to the wireless host's MAC address. When this frame reaches the access point, the AP encapsulates it into an 802.11 frame.
In this Wi-Fi frame, the router's MAC address is placed in the Address 3 field, the AP's MAC address appears as the transmitter, and the wireless host's MAC address appears as the receiver.

When sending instead from a wireless host to a router, through the AP, the wireless host creates a 802.11 frame, containing its MAC address as source and the AP's MAC address as destination, and finally, in the address 3 field, the router interface's MAC address.
When the WiFi frame arrives at the AP, it produces an ethernet frame, containing the router interface's MAC address as destination address and the sender's MAC address as the source address.

2.1.5 Mobility within the same subnet (7.3.4)

Mobility within the same LAN—or VLAN—subnet means a host moves away from its associated AP and associates with another AP connected to the first one via a switch (both APs are in the same subnet).
As the host moves away from the AP, the signal gets weaker and it starts looking for alternatives, at this point it may receive a beacon frame from another AP in the same subnet (often with the same SSID as the first one). The host now disconnects from the first AP and associates with the new one: as the host remains in the same subnet, it can keep its IP address and its ongoing TCP connections.
Now the new AP sends a broadcast frame with the host's MAC address as the source address: the switch connecting the two APs notices that the frame arriving from the host's MAC address changed origin AP; as it is **self-learning** it updates its forwarding table and whenever trying to reach the host, it will do so through the new AP.

2.1.6 Advanced Capabilities (7.3.5)

The following capabilities are not completely specified in the 802.11 standard, but rather are made possible by mechanisms specified in the standard. This allows different vendors to implement these capabilities using their own (proprietary) approaches, presumably giving them an edge over the competition.

Rate Adaptation

The AP and the mobile node dynamically change transmission rate, using a certain physical layer modulation technique based on recent or current channel characteristics.
As the mobile host moves SNR and BER vary: if the node moves away from the base station: the SNR decreases and the BER increases; when the BER becomes too high (2 frames in a row without acknowledgement), the communication switches to a lower transmission rate, leading to a lower BER. If 10 frames in a row are acknowledged, or if a timer that tracks the time since the last fallback expires, the transmission rate increases to the next higher rate. This mechanism shares the same "probing" philosophy as TCP's congestion-control mechanism.

Power Management - Sleep

To save energy, 802.11 utilizes the concept of *sleep*: the node tells the AP it is going to sleep *until the next beacon frame* (by setting the power management bit in the 802.11 frame control field), this way the AP knows not to transmit frames to this node and buffer them until it awakens. The node wakes up (in c.a 250 microseconds) before the arrival of the next beacon frame (which usually will be after 100 milliseconds). This **beacon frame** contains a list of mobiles with AP-to-mobile frames waiting to be sent, if a node recognizes itself in the list, it will stay awake and explicitly request these frames, otherwise it will go to sleep again until the next beacon frame. Using this technique, a host with nothing to send can stay asleep for 99% of the time.

2.2 Wireless LANs: LiFi: Light Fidelity

LiFi is a wireless technology using LED light modulation to transmit information instead of Radio Frequency (RF) based technology (such as WiFi).
LiFi presents the following advantages over RF:

- *speed and bandwidth*: LiFi can deliver multiple Gbps in mobile devices.
- *enhanced security*: light can be contained and secured in a physical space, also LiFi enables additional control as LiFi offers precise localization for asset tracking and user authentication.
- *interference free*: RF is vulnerable to interference from a wide range of devices.
- *low latency*: up to three times lower than WiFi.

2.3 Wireless PANs: Bluetooth (7.3.6)

Bluetooth is the commercial name for the **IEEE 802.15.1** standard, which operates over a short range, at low power and at low cost: Bluetooth's range is short, less than 10 m in diameter—whereas the 802.11 WiFi standard can go up to 100 meters, so it mainly works as a substitute for cables. For this reason, 802.15.1 networks are sometimes referred to as **wireless personal area networks** (**WPAN**s).

802.15.1 networks operate in the 2.4 GHz unlicensed radio band in a TDM manner, with time slots of 625 microseconds. During each time slot, a sender transmits on one of 79 channels, with the channel changing in a known but pseudo-random manner from slot to slot. This form of channel hopping, known as **frequency-hopping spread spectrum** (**FHSS**), spreads transmissions in time over the frequency spectrum. 802.15.1 can provide data rates up to 4 Mbps.

802.15.1 networks are *ad hoc networks*: No network infrastructure (e.g., an access point) is needed to interconnect 802.15.1 devices. Thus, 802.15.1 devices must organize themselves. 802.15.1 devices are first organized into a **piconet**: a small, ad-hoc network of 2 to 8 Bluetooth devices (one master and up to seven active slaves) that communicate over a shared physical channel, forming the basic unit of a Bluetooth network, like your phone connected to a headset or speaker. The master controls the connection, can transmit in each even-numbered slot (while slaves transmit in odd-numbered slots), dictates timing, and manages frequency hopping for all devices, allowing for simple, short-range, point-to-multipoint communication, which can extend into larger scatternets by linking multiple piconets. A slave can transmit only after the master has communicated with it in the previous slot and even then the slave can only transmit to the master: the multiple access protocol is based on **polling**: the master polls one client at a time, the polled client replies with a data packet (or null).
In addition to the slave devices, there can also be up to 255 **parked** (asleep) devices in the network. These devices cannot communicate until their status has been changed from parked to active by the master node.

Choosing the frequency based on congestion parameters is hard, therefore we use this approach where every slot uses a frequency based on the pseudo-random order, decided by the sender. This way two senders will probably not interfere for multiple slots.

2.4 Cellular networks: 4G and 5G (7.4)

Cellular networks are *the* solution for wide-area mobile internet, it has in fact found a widespread deployment/use: there are now more mobile-broadband-connected devices than fixed-broadband-connected devices (5 to 1 in 2019) and in some places availability is reaching completeness (4G availability is 97% in Korea for example). These networks can have transmission rates of up to 100's of Mbps.
Cellular networks differ from wired internet for many reasons: they use different link layer technologies, in cellular networks mobility is a first class service, not as an afterthought, they identify users differently (SIM card vs MAC/IP) and they have different business models (cellular networks are only available as subscriptions). But they both have the same edge/core distinction, they're both networks of networks, they share a lot of protocols, they both separate data and control planes and they are both ultimately connected to the wired internet.

The 4G architecture introduces a unified, all-IP network architecture: unlike 3G, in this technology, both voice and data are carried in IP datagrams to/from the **user equipment** to the **Packet Data Network Gateway** (**P-GW**) that connects the 4G edge network to the rest of the network. 4G instead clearly separates the radio access IP network and the core IP network: everything specific to wireless transmission is managed separately from the core IP network (the **Evolved Packet Core**—**EPC**), which manages subscriber billing, user authentication, mobility tracking, external internet connectivity etc.

2.4.1 Elements of the 4G LTE architecture (7.4.1)

- **user equipment** (**UE**)
- **eNodeB**: the base station, a data-plane device whose role is to forward datagrams between the UE and the S-GW.
- **Packet Data Network Gateway** (**P-GW**): the edge router of the 4G core network, it allocates IP addresses to the UE, performs QoS enforcement, performs datagram encapsulation/decapsulation when forwarding to/from a UE, etc.
- **Serving Gateway** (**S-GW**): the internal data-plane router within the core network, which also acts as the local mobility anchor for all user IP packets moving between cell towers and the core network. Forwards all inbound and outbound user data packets along the data path between the eNodeB access network and the P-GW edge gateway. It collects volume-based usage statistics for billing and enables lawful traffic monitoring.
- **Mobility Management Entity** (**MME**): it performs connection and mobility management on behalf of the UEs resident in the cells it controls. It receives UE subscription information from the HSS.
- **Home Subscriber Server** (**HSS**): contains UE information including roaming access capabilities, quality of service profiles, and authentication information, it obtains this data from the UE's home cellular provider.

LTE Data Plane

LTE's link layer includes 3 protocols:

- **Packet Data Convergence Protocol** (PDCP), which manages header compression and encryption
- **Radio Link Control** (RLC) protocol for fragmentation/reassembly and Reliable data Transfer.
- Medium Access Control (MAC): requesting and use of radio transmission slots.

UEs implement the entire protocol stack (up to the application layer). Base stations (eNodeBs) terminate the radio link layers (PHY, MAC, RLC, PDCP) and tunnel user IP datagrams to/from the S-GW via GTP-U/UDP/IP.

LTE uses a combination of frequency division multiplexing and time division multiplexing on the downstream channel, known as **orthogonal frequency division multiplexing** (**OFDM**). In LTE, the minimum scheduling unit is a Resource Block (RB), spanning a 0.5 ms time slot and 12 subcarrier frequencies (180 kHz total). Transmission scheduling typically allocates pairs of 0.5 ms slots (a 1 ms subframe) across multiple subcarriers.
The decision of which mobile nodes will be allowed to transmit in a given time slot on a given frequency is determined by the scheduling algorithms provided by the LTE equipment vendor and/or the network operator, so it is not mandated by the LTE standard.

The mobile datagram is encapsulated using the GPRS tunneling protocol and sent inside a UDP datagram to the S-GW, which re-tunnels the datagram to the P-GW.

Associating with a Base Station

A mobile device wishing to attach to a cellular carrier network will begin a bootstrap process to learn about, and then associate with, a nearby base station. The mobile device initially searches all channels in all frequency bands for a primary synchronization signal that is periodically broadcast every 5 ms by a base station. Once this signal is found, the mobile device remains on this frequency and locates the secondary synchronization signal. With information found in this second signal, the device can locate (following several further steps) additional information such as channel bandwidth, channel configurations, and the cellular carrier information of that base station. Armed with this information, the mobile device can select a base station to associate with (preferentially attaching to its home network, if available) and establish a control-plane signaling connection across the wireless hop with that base station. This mobile-to-base-station channel will be used through the remainder of the network attachment process.

The Global Cellular Network (7.4.5)

A user's mobile connects via a 4G base station into its home network, operated by a cellular provider and connected to the networks of other cellular carriers and to the global Internet through one or more gateway routers in the home network.
The mobile networks themselves interconnect with each other either via the public Internet or via an Internet Protocol Packet eXchange (IPX) Network. An IPX is a managed network specifically for interconnecting cellular carriers, similar to Internet eXchange Points for peering among ISPs.

2.4.2 LTE mobiles: sleep modes

In LTE we have two different sleep modes: a mobile enters **light sleep** after 100's msec of inactivity, here the device wakes up periodically (100's msec) to check for downstream transmissions, after instead 5 to 10 seconds of inactivity the mobile enters **deep sleep**, in which the device's radio wakes up for buffered downstream connections even less frequently.
During deep sleep the mobile may even change cells, thus, when waking up periodically from this deep sleep, the mobile device will potentially need to re-establish an association with a new base station in order to check for paging messages broadcast by the MME to base stations nearby the base station with which the mobile was last associated.

2.4.3 5G (7.4.6)

It is predicted that 5G will provide roughly a 10x increase in peak bitrate, a 10x decrease in latency, and a 100x increase in traffic capacity over 4G.
Principally, 5G refers to “5G NR (New Radio),” which is the standard adopted by 3GPP. Other 5G technologies besides NR do exist, however. Importantly, the physical layer (that is, wireless) aspects of 5G are not backward-compatible with 4G mobile communications systems such as LTE: in particular, it can’t be delivered to existing smartphones by deploying base station upgrades or software updates. Therefore, in the transition to 5G, wireless carriers will need to make substantial investments in physical infrastructure.

5G uses two frequency bands: FR1 (450 MHz-6GHz) and FR2 (24GHz-52GHz): these are millimeter wave frequencies, thus having much higher data rates, but over shorter distances; pico-cells have diameters of 10-100 meters. This means 5G NR needs massive, dense deployment of new base stations.

If we aim at increasing the network's capacity, another thing we could do is improve the spectral efficiency, but this would normally take much more power, instead we can use **MIMO**-technology, which uses multiple antennas at each base station. Rather than broadcasting signals in all directions, each MIMO antenna employs beam forming and directs the signal at the user. MIMO technology allows a base station to send to 10–20 users at the same time in the same frequency band. Millimeter wave signals are, however, easily blocked by buildings and trees. Small cell stations are needed to fill in coverage gaps between base stations and users. In a highly populous region, the distance between two small cells could vary from 10 to 100 meters.

3 Mobility Management (7.5)

In the broadest sense, a mobile device is one that changes its point of attachment into the network over time. From the network layer’s standpoint, a physically mobile device will present a very different set of challenges to the network layer, depending on how active the device is as it moves between points of attachment to the network, here they are listed from lowest to highest:

1. Device moves between access networks, but is powered down while moving.
2. Device moves within the same access network in a single provider network.
3. Device moves among access networks in a single provider network, while maintaining ongoing connections. Here, the network will need to provide **handover**—a transfer of responsibility for forwarding datagrams to/from one AP or base station to the mobile device—as the device moves among WLANs or among LTE cells.
4. Device moves among multiple provider networks, while maintaining ongoing connections.

When a device is connected to a cellular network, other than its home network, that device is said to be roaming on a visited network. When a mobile device attaches to, and roams on, a visited network, coordination will be required between the home network and the visited network.

The home network is a (paid) service plan with a cellular provider, the home network's **HSS** (**home subscriber server**) stores identity and services info.
A visited network is any network other than your home network.
Networks have a service agreement between them to provide access to a visiting mobile.

ISPs and WiFi have no notion of a global "home": the credentials from the ISP (for example username and password) are stored on device.
ISPs may have national or international presence. Different networks have different credentials, with some exceptions, like eduroam.
Architectures exist (mobile IP) for 4G-like mobility, but they are not used.

3.1 Mobility management

To approach mobility we could either:

- *let the network (so the routers) handle it*: the routers advertise well-known `name,address` (or `name,number`) pairs of visiting mobile nodes via the usual routing table exchange. The internet routing could do this already with *no* changes: the routing tables indicate where each mobile is located via the longest prefix match.
  This approach sadly is not scalable to billions of mobiles.
- *let end-systems handle it* (functionality at the edge):

- *indirect routing*: the communication from the correspondent to the mobile goes through the home network.
- *direct routing*: the correspondent gets the foreign address of the mobile (from the mobile's home agent / HSS), then sends directly to the latter.

4G and 5G use a *functionality at the edge*-approach to mobility: when a mobile finds itself in a network other than the home one, it *associates* with the visited mobility manager, who *registers* the mobile's location with the home HSS[3].

3.1.1 Mobility with indirect routing

The mobile IP standard as well as 4G LTE use **edge indirect routing**.
The correspondent is completely unaware of the mobile's position and just sends the communication to the mobile's fixed *home address*, the home network handles the rest. The home network gateway intercepts this datagram, consults with the HSS to determine the visited network where the mobile device is resident, and forwards the encapsulated datagram toward the visited network gateway router, which will decapsulate it and send it to the destination mobile. If NAT translation is used, the visited network gateway router performs NAT translation. Now the visited gateway's router forwards the reply to the correspondent either via the home network or directly—an approach known as **local breakout**.

Let's summarize our discussion of indirect routing by reviewing the new network-layer functionality required to support mobility:

1. a *mobile-device–to–visited-network association protocol*: the mobile device will need to associate with the visited network, and will similarly need to disassociate when leaving the visited network.
2. A *visited-network–to–home-network-HSS registration protocol*: the visited network will need to register the mobile device's location with the HSS in the home network, and perhaps use information obtained from the HSS in performing device authentication.
3. A *datagram tunneling protocol between the home network gateway and the visited network gateway router*: the sending side performs encapsulation and forwarding of the correspondent's original datagram; on the receiving side, the gateway router performs decapsulation, NAT translation, and forwarding of the original datagram to the mobile device.

This approach is inefficient when the correspondent and the mobile are in the same network, the so called **triangle routing** wastes resources. The mobility of the mobile is transparent to the correspondent: this means for example that TCP connections can be maintained.

3.1.2 Mobility with direct routing

Direct Routing solves the triangle routing problem at the cost of additional complexity: in this approach the correspondent contacts the HSS[3-1] and gets the mobile's visited network, then the correspondent tunnels the datagram to the visited network's gateway router, which finally forwards the datagram to the mobile.

Although this approach overcomes the triangle routing inefficiencies, it is *non-transparent to the correspondent* and when the mobile changes visited network the complexity rises: the new visited network informs the anchor/old visited network about the new mobile's location, the datagram gets sent from correspondent to the old visited network to the new visited network and finally to the mobile.

3.2 Mobility's impact on higher-layer protocols (7.7)

What are the effects of mobility on higher-layer protocols?
Logically the impact *should* be minimal, but performance wise:

- packet loss and delay due to bit-errors and handover loss become a problem
- **TCP interprets loss as congestion, thus decreasing the congestion window unnecessarily**. A possible solution is *splitting*: the TCP connection gets split into two TCP connections, one correspondent–to–home-network/base-station connection and one base-station–to–mobile connection
- delay impairments for real-time traffic
- bandwidth is a scarce resource for wireless links

Researchers realized in the early to mid 1990s that given high bit error rates on wireless links and the possibility of handover loss, TCP's congestion-control response could be problematic in a wireless setting. Three broad classes of approaches are possible for dealing with this problem:

- **local recovery**: dealing with bit errors where they occur.
- **TCP sender awareness of wireless links**: TCP sender and receiver are aware of the existence of a wireless link and distinguish between congestive losses occurring in the wired network and corruption/loss occurring at the wireless link, thus invoking congestion control only in response to congestive wired-network losses.
- **split-connection approaches**: the end-to-end connection between the mobile user and the other end point is broken into two transport-layer connections: one from the mobile host to the wireless access point, and one from the wireless access point to the other communication end point.

---

1. Cyclic Redundancy Check, see Link Layer > Cyclic Redundancy Check (CRC) (6.2.3) for more details ↩
2. Medium Access Control ↩

3. Home Subscriber Server ↩ ↩

Network Security

#uni

- 1 Introduction
- 2 Principles of Cryptography (8.2)
  - 2.1 Breaking encryption
  - 2.2 Symmetric Key cryptography (8.2.1)
    - 2.2.1 Block Ciphers
    - 2.2.2 How entities establish a key
  - 2.3 Public Key Cryptography (8.2.2)
- 3 Message integrity (8.3)
  - 3.1 Message digests
    - 3.1.2 MAC: Message Authentication Code (8.3.2)
  - 3.2 Digital signatures (8.3.3)
    - 3.2.1 CA: public key certification authorities
- 4 Authentication (8.4)
- 5 Securing e-mail (8.5.1)
- 6 Securing TCP connections: TLS (8.6)
  - 6.1 The almost-TLS protocol
  - 6.2 The actual TLS protocol (8.6.2)
- 7 Network Layer Security: IPsec (8.7)
  - 7.1 Virtual Private Networks (VPNs)
  - 7.2 SAs: Security Associations (SAs) (8.7.3)
  - 7.3 IPsec protocols (8.7.2)
    - 7.3.1 ESP tunnel mode
  - 7.4 IPsec sequence Numbers
- 8 Operational security: firewalls and Intrusion detection systems (8.9)
  - 8.1 Firewalls (8.9.1)
    - 8.1.1 Stateless (traditional) packet filters
    - 8.1.2 Stateful packet filtering
    - 8.1.3 Application Gateways
  - 8.2 Intrusion detection systems

1 Introduction

Cryptography is the practice and study of techniques for secure communication in the presence of adversaries. It ensures that information remains confidential, authentic, and intact during transmission and storage. The core principles of cryptography include:

- **Confidentiality**: Ensures that only authorized parties can access the message content. This is achieved through encryption.
- **Authentication**: Verifies the identity of the communicating parties, preventing impersonation attacks.
- **Integrity**: Guarantees that the message has not been altered during transmission.
- **Operational Security**: Ensures secure operational practices to protect systems and data.

2 Principles of Cryptography (8.2)

Cryptography relies on mathematical principles and algorithms to protect data from threats like eavesdropping, tampering, and unauthorized access. It is foundational to modern secure communication systems, including secure email, VPNs, and encrypted web connections.

Bob and Alice want to communicate securely, but Trudy may intercept, delete and add messages. Trudy listens on the channel that Alice and Bob use to communicate.
Bob and Alice are any couple of hosts on the Internet that want to communicate.
Trudy can: eavesdrop, insert/modify/delete messages or content, spoof source address, hijack the communication, deny the service, etc.

Alice's plaintext gets encrypted by an encryption algorithm using Alice's encryption key. The ciphertext travels through the channel and gets delivered to Bob, then gets decrypted into plaintext by a decryption algorithm using Bob's decryption key.

$$
\begin{gathered}
m: \text{ plaintext message} \\
K_A(m): \text{ ciphertext, encrypted with the key } K_A \\
m = K_B(K_A(m))
\end{gathered}
$$

See Crittografia for more details.

2.1 Breaking encryption

There are different ways encryption can be broken depending on the information available to Trudy:

- Trudy only has the ciphertext transmitted over the insecure channel (**cipher-text only attack**), so decryption is only possible either by brute force or by statistical analysis.
- Trudy has known plaintext (some `plaintext , ciphertext` pairs).
- Trudy makes a chosen-plaintext attack: the intruder is able to choose the plaintext message and obtain its corresponding ciphertext form.

2.2 Symmetric Key cryptography (8.2.1)

In Symmetric key cryptography Bob and Alice share the same key k (hence "symmetric"). $K_A = K_B = K$.
Alice and Bob need to agree on the value of the key, without leaking it to third parties.

Symmetric Key Cryptography can be done either bit-by-bit (so called **Stream Ciphers**) or in blocks of k bits (**block ciphers**).
Stream Ciphers are used for Bluetooth connections and cellular and 4G connections. Older SSL/TLS versions used the RC4 stream cipher, but it has since been banned: modern TLS only uses block ciphers in AEAD modes.

2.2.1 Block Ciphers

In this approach plaintext messages get broken into equal-size blocks, then each block gets encrypted as a unit. To encode a block, the cipher uses a one-to-one mapping to map the k-bit block of cleartext to a k-bit block of ciphertext.
Although full-table block ciphers with moderate values of k can produce robust symmetric key encryption schemes, they are unfortunately difficult to implement. Instead, block ciphers typically use functions that simulate randomly permuted tables, for example by breaking blocks into even smaller blocks and using smaller-sized tables for the conversion into n encrypted smaller blocks, which then get permuted and form a k-sized block. This output is then routed back into the system to get encrypted again, for many more rounds, to try and make every input bit affect every output bit. The key for this block cipher algorithm would be the eight permutation tables (assuming the scramble function is publicly known). Today most block ciphers use functions instead of actual predetermined tables and use a string of bits for a key.

Today there are a number of popular block ciphers, including DES (standing for Data Encryption Standard), 3DES, and AES (standing for Advanced Encryption Standard).

Cipher Block Chaining (CBC)

Cipher Block Chaining (CBC) is a way to use a block cipher—such as AES—to encrypt data longer than one block.
Before encrypting each block, CBC combines it with the previous ciphertext block using XOR. For the first block, it uses a random **initialization vector** (IV) instead.
To decrypt, the receiver decrypts each ciphertext block, then XORs the result with the previous ciphertext block.
This chaining means identical plaintext blocks usually encrypt to different ciphertext blocks depending on what came before them. The IV doesn't need to be secret, but it must be unpredictable and sent along with the ciphertext. CBC also requires padding when the message length isn't a multiple of the block size.

Data Encryption Standard (DES)

The US **Data Encryption Standard** (**DES**) uses a 56-bit symmetric key and 64-bit blocks of plaintext input, it is a block cipher with cipher block chaining.
It has been brute forced in less than a day, but there is no known good analytic attack.
It gets more secure with 3DES, by encrypting 3 times with 3 different keys.

AES: advanced encryption standard

This is a symmetric-key NIST standard that replaced DES in November 2001.
Blocks are 128-bit wide and keys can be 128, 192 or 256 bit.
Brute force decryption that takes one second on DES, takes 149 trillion years on AES.

2.2.2 How entities establish a key

Symmetric key cryptography algorithms need the two communicating entities to establish a shared secret key.
Possible solutions are:

- **direct exchange**
- **key distribution center** (**KDC**): a trusted entity acts as an intermediary between the two entities
- **using public key cryptography** for communicating the key

Key distribution center

Alice and Bob know their own symmetric key (K_{A-KDC} and K_{B-KDC}) for communicating with the KDC. These keys need to be retrieved in person at the KDC.

1. Alice sends to the KDC $K_{A-KDC}(A, B)$

2. The KDC responds with $K_{A-KDC}(R1, K_{B-KDC}(A, R1))$, now Alice knows the session key $R1$
3. Alice now sends Bob $K_{B-KDC}(A, R1)$, now Bob knows the session key $R1$
4. Alice and Bob communicate using the shared session key $R1$

2.3 Public Key Cryptography (8.2.2)

Public Key cryptography uses a radically different approach: the sender and the receiver do not share the secret key, every user now has a **public** encryption key, known to all, and a **private** decryption key known only to the receiver.
Alice encrypts the plaintext with Bob's *public* key K_B^+, Bob decrypts the message using his *private key* K_B^-: $m = K_B^-(K_B^+(m))$. Note that $m = K_B^-(K_B^+(m)) \implies m = K_B^+(K_B^-(m))$.

Unfortunately public key cryptography is much more computationally demanding than symmetric cryptography: 100 times slower than DES. Because of this we use the slow public key cryptography to establish a secret symmetric session key for fast communication: Bob and Alice use RSA to exchange a symmetric session key K_S, then use K_S for symmetric key cryptography communication.

Public key cryptography also needs digital signatures: since the key is public, everyone can send to the receiver encrypted messages, even Trudy, as opposed to single shared key cryptography, where knowing the key IS the signature (see 3.2 Digital signatures (8.3.3) for more information on digital signatures).

Requirements for Public Key Cryptography are that $m = K_B^-(K_B^+(m))$ and that it should be impossible to compute the private key from the given public key.

RSA: Rivest, Shamir, Adleman

While there may be many algorithms that address these concerns, the 1977 **RSA** algorithm (named after its founders, Ron Rivest, Adi Shamir, and Leonard Adleman) has become almost synonymous with public key cryptography and won its inventors the Turing Award in 2002.
RSA makes extensive use of arithmetic operations using modulo-n arithmetic.

3 Message integrity (8.3)

Message integrity consists in allowing parties to verify that the received messages are authentic, i.e.:

- the sender is who you think it is
- the contents are what the sender intended them to be
- the message has not been replayed
- the sequence of the messages is maintained

3.1 Message digests

To implement this we introduce **message digests**: fixed-length, easy-to-compute digital footprints.
We apply a **hash function** H to m and get a fixed-size message digest $H(m)$, known as a **hash**.

Cryptographic Hash functions require the following properties:

- **easy to compute**
- **irreversibility**: given message digest x, it must be unfeasible to find m such that $x = H(m)$
- **collision resistance**: given $[m, H(m)]$ it must be *computationally unfeasible* to produce m' ($m' \neq m$) such that $H(m') = H(m)$. We are fine with unfeasible instead of impossible since it is not possible to map an infinite space to a finite space without collisions.
- **seemingly random output**

The internet checksum has some of the properties of a hash function: it produces a fixed length output and is many-to-one, but, given a message with its hash value, it is easy to find another message with the same hash value: it is not collision resistant.

Hash function algorithms (8.3.1)

MD5

MD5 computes a 128-bit message digest in a 4-step process.
Given an arbitrary 128-bit string x, it appears difficult to find a message m such that MD5 $(m) = x$: it appears collision-resistant.
In reality MD5 is broken: collisions have been demonstrated since 2004 (Wang et al.), so it must not be used anymore.

SHA-1

SHA-1 is the US standard. It produces a 160-bit message digest.
A practical collision was demonstrated in 2017 (the "SHAttered" attack), so SHA-1 is deprecated for security applications.

3.1.2 MAC: Message Authentication Code (8.3.2)

A first approach would be to create a message, append to it its hash and send it. This doesn't work because an intruder could simply intercept the message, change it and append to it its hash, and the receiver will have no way to know who sent it.

We need a way to perform message integrity: we introduce a **shared secret**: a string only known to sender and receiver.

1. The sender computes the hash of `< message , shared secret >`: the **message authentication code** (**MAC**) $H(s,m)$.
2. the message and the MAC travel to the receiver
3. the receiver again computes the MAC of `< received message , shared secret >`, then compares it with the received MAC
4. if the two MACs are equal the message has not been altered AND the sender is the expected one.

This approach uses no encryption and is also called "keyed hash".
As you might expect, a number of different standards for MACs have been proposed over the years. The most popular standard today is **HMAC**, which can be used either with MD5 or SHA-1. HMAC actually concatenates the authentication key in front of the message and runs data and the key through the hash function twice.
For example OSPF makes use of MACs.

Playback attack

More precisely if the two MACs correspond, we know that the MAC was *generated* by Alice, but we don't know if the `< message , MAC >` was *transmitted* by Alice: Trudy can intercept the message, and play it back to Bob.

Defense

To defend from the "playback attack", we include the timestamp of generation in the message before computing the MAC. But this is quite difficult to implement, because sender and receiver clock may not be synchronized. The real solution is using **one time passwords** (**OTP**s). Alice contacts Bob, Bob answers with an OTP and Alice computes the MAC with the secret key AND the OTP: Bob expects ONE message PER sent OTP.

3.2 Digital signatures (8.3.3)

Digital signatures are a cryptographic technique that allows the sender to digitally sign a document.
A digital signature is **non-forgeable** (the sender can prove that no one else signed the message) and **verifiable** (the recipient can verify and prove that the sender and no one else signed the document). Digital signatures also allow **non repudiation** (the recipient can prove that the sender signed `m` and not `m'`) and **message integrity** (the sender can prove that he signed `m` and not `m'`).

Note that MAC cannot be used as a digital signature because the secret is *shared* between the two communicating parties.
For authentication we use asymmetric cryptography (2.3 Public Key Cryptography (8.2.2)): Bob signs m by encrypting with his private key K_B^- and obtains the signed message $K_B^-(m)$, Alice can verify that the sender is Bob if the message can be decrypted with Bob's public key K_B^+.
But as we said asymmetric cryptography is computationally demanding, a smarter approach would be to sign only the digest, not the message (the message can be arbitrarily long, the digest is usually much smaller):

1. Bob computes the digest: $H(m)$
2. Bob signs (encrypts) the digest with his private key
3. Bob sends the message m with the encrypted message digest $K_B^-(H(m))$
4. Alice computes the digest of the received message: $H(m)$
5. Alice decrypts the received digest using Bob's public key and obtains the message digest $H'(m)$
6. Alice compares $H(m)$ and the decrypted messages digest $H'(m)$, if they are equal the message and the sender can be trusted.

3.2.1 CA: public key certification authorities

Who guarantees that Alice actually has Bob's public key? Trudy could create a `public,private` key pair, encrypt some message with their private key and send their public key to Alice, with the latter thinking it's Bob's public key. We need some way to securely obtain public keys.

A **certification authority** (**CA**) binds a public key to a particular entity E. The entity E registers its public key with the CA, providing a "proof of identity":

1. The CA creates a certificate binding the identity of E to E's public key
2. the certificate gets digitally signed by the CA using the CA's private key
3. when Alice wants to ensure Bob's identity, it decrypts the certificate using the CA's public key
   The CA's public key is supposed to be known by everyone.

A certificate includes:

- Issuer name
- Entity's name, address, domain name, etc (some sort of identification)
- Entity's public key
- Digital signature
  and is signed with the issuer's private key.

4 Authentication (8.4)

End-point authentication is the process of one entity proving its identity to another entity over a computer network. We focus here on authenticating a “live” party, at the point in time when communication is actually occurring. Authentication must be done solely on the basis of messages and data exchanged as part of an authentication protocol. Typically, an authentication protocol would run before the two communicating parties run some other protocol: the authentication protocol first establishes the identities of the parties to each other’s satisfaction; only after authentication do the parties get down to the work at hand.

- AP1.0: "I am Alice"
- AP2.0: Alice says "I am Alice and this is my IP address"
- AP3.0: "I am Alice" and sends her secret password together with her IP address (Trudy listens to the password the first time and then copies it)
- AP3.1: AP3.0 with encrypted password (Trudy can still paste the encrypted password: a playback attack)
- AP4.0: Bob sends Alice a **nonce** R, Alice must return R encrypted with a shared secret key. Bob decrypts the message with the shared secret key and if the nonce is correct he knows Alice is *live*. This approach requires a shared symmetric key.
- AP5.0: AP4.0 but using public key cryptography: Alice contacts Bob, who answers with a nonce R, Alice sends the nonce encrypted with her private key, now Bob asks for her public key and when he gets it, he decrypts the nonce and if it is correct the parties have exchanged keys and can now communicate using public key cryptography.
  This protocol is susceptible to "man in the middle" attacks: when Alice sends "I am Alice", Trudy can intercept it and send it herself, now Bob will send Trudy the **nonce**.
- AP6.0: AP5.0 but with CAs.

5 Securing e-mail (8.5.1)

For securing e-mail we need **Sender and Receiver Authentication**, **confidentiality** and **message integrity**.
Pretty Good Privacy (**PGP**) is such a secure email protocol and one of the first security technologies to be broadly used in the internet.

To exchange mail with confidentiality: Alice generates a random symmetric private key, K_S, encrypts message with K_S (for efficiency) and also encrypts K_S with Bob's public key, then finally sends both $K_S(m)$ and $K_B^+(K_S)$ to Bob. Bob uses his private key to decrypt and recover K_S and uses K_S to decrypt $K_S(m)$, obtaining m.
Alas, to add message integrity: Alice digitally signs the hash of her message with her private key, Bob will decrypt the digest with Alice's public key, compute a digest for the received message and finally compare the two digests, if they are equal, the message has not been tampered with.

Final secure Email Protocol

When summing these practices we get E-Mail with confidentiality, message integrity and authentication:

Alice:

1. generates a random symmetric private key, K_S
2. encrypts message with K_S (for efficiency), $K_S(m)$
3. encrypts K_S with Bob's public key, obtaining $K_B^+(K_S)$
4. computes the digest of the message, $H(m)$
5. encrypts the digest with her private key, $K_A^-(H(m))$
6. sends both $K_S(m)$, $K_B^+(K_S)$ and $K_A^-(H(m))$ to Bob

Bob:

1. uses his private key to decrypt and recover K_S
2. uses K_S to decrypt $K_S(m)$ and obtains m'
3. uses Alice's public key to decrypt $K_A^-(H(m))$, obtaining the message digest $H(m)$
4. computes the digest $H(m')$ and compares it with $H(m)$, if they are equal, the message was sent by Alice, has not been modified and is confidential.

Note that in standard PGP, the digital signature $K_A^-(H(m))$ is encrypted inside the symmetric payload $K_S(\dots)$ alongside the message m, rather than sent separately in the clear.

6 Securing TCP connections: TLS (8.6)

Transport Layer Security (**TLS**) is a widely deployed security protocol built on TCP (see Transport Layer > 5 Connection-Oriented transport: TCP (3.5) for more information on TCP), and is supported by almost all browsers and web servers at https (port 443). TLS (Transport Layer Security) is the modern, secure, and standardized successor to **SSL** (Secure Sockets Layer).
This protocol provides: (1) confidentiality via symmetric cryptography, (2) integrity via cryptographic hashing and (3) authentication via public key cryptography.

TLS provides a simple Application Programmer Interface (API) with sockets, which is similar and analogous to TCP’s API. When an application wants to employ TLS, the application includes SSL classes/libraries. As shown in Figure 8.24, although TLS technically resides in the

application layer, from the developer's perspective it is a transport protocol that provides TCP's services enhanced with security services.

TLS has four phases:

1. handshake: Alice and Bob use their certificates and private keys to authenticate each other and exchange or create shared secrets
2. key derivation: Alice and Bob use the shared secret to derive a set of keys
3. data transfer: transfer of the stream of data in records
4. connection closure: a special message to securely close the connection

6.1 The almost-TLS protocol

T-TLS: (1) initial Handshake

During the handshake phase, Bob needs to (a) establish a TCP connection with Alice, (b) verify that Alice is really Alice, and (c) send Alice a master secret key, which will be used by both Alice and Bob to generate all the symmetric keys they need for the TLS session.
Note that once the TCP connection is established, Bob sends Alice a hello message. Alice then responds with her certificate, which contains her public key.
Bob then generates a Master Secret (MS) (which will only be used for this TLS session), encrypts the MS with Alice's public key to create the Encrypted Master Secret (EMS), and sends the EMS to Alice. Alice decrypts the EMS with her private key to get the MS. After this phase, both Bob and Alice (and no one else) know the master secret for this TLS session.

T-TLS: (2) key derivation

In principle, the MS, now shared by Bob and Alice, could be used as the symmetric session key for all subsequent encryption and data integrity checking. It is, however, generally considered safer for Alice and Bob to each use different cryptographic keys, and also to use different keys for encryption and integrity checking. Thus, both Alice and Bob use the MS to generate four keys:

- E_B = session **encryption key** for data sent from Bob to Alice
- M_B = session **HMAC key** for data sent from Bob to Alice, where HMAC is a standardized hashed message authentication code (MAC)
- E_A = session encryption key for data sent from Alice to Bob
- M_A = session HMAC key for data sent from Alice to Bob

Alice and Bob each generate the four keys from the MS. This could be done by simply slicing the MS into four keys. *The two encryption keys will be used to encrypt data*; *the two HMAC keys will be used to verify the integrity of the data*.

T-TLS: (3) data transfer

Now that Alice and Bob share the same four session keys, they can start to send secured data to each other over the TCP connection. Although TCP is a byte-stream protocol we cannot encrypt application data on the fly and then pass the encrypted data on the fly to TCP because we would have nowhere to put the HMAC for the integrity check. TLS breaks the data stream into records, appends an HMAC (computed with the relative HMAC key M) to each record for integrity checking, and then encrypts the record together with the HMAC with the encryption key E. This encrypted package is then passed to TCP for transport over the Internet.

But an intruder could intercept two segments, invert their order and adjust their TCP sequence numbers, which are not encrypted, now Bob's application layer would receive a byte stream not in the correct order. To solve this Bob maintains a sequence number counter, which begins at zero and is incremented for each TLS record he sends. Bob doesn't actually include a sequence number in the record itself, but when he calculates the HMAC, he includes the sequence number in the HMAC calculation. Thus, the HMAC is now a hash of the data plus the HMAC key MB plus the current sequence number. Alice tracks Bob's sequence numbers, allowing her to verify the data integrity of a record by including the appropriate sequence number in the HMAC calculation.

TLS Record Format:



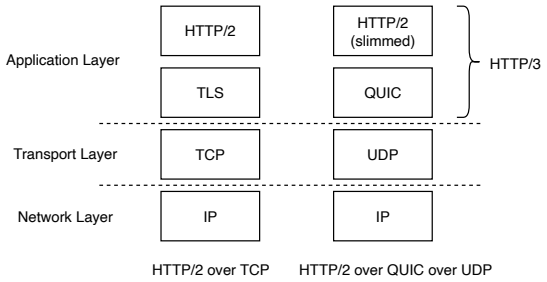
T-TLS: (4) connection close

A **truncation attack** is possible: the attacker forges a TCP connection close segment so that one or both sides think there is less data than there actually is.
The solution is to have different record types: type 0 for data and type 1 for close.

6.2 The actual TLS protocol (8.6.2)

The previous subsection covered the almost-TLS protocol; it served to give us a basic understanding of the why and how of TLS. Now that we have a basic understanding, we can dig a little deeper and examine the essentials of the actual TLS protocol.

TLS provides an API that *any* application can use.

> TLS vs QUIC:
>
> 
>

TLS needs a **Cipher Suite**, which must include:

- an Algorithm for key generation
- a Public-key encryption algorithm
- a Symmetric-key encryption algorithm
- a MAC algorithm

TLS 1.3 offers a more limited cipher suite choice than TLS 1.2 (5 rather than 37 choices). Client and server need to agree on the used cipher suite during the handshake.

TLS handshake

1. the client in the **TLS client hello** message sends:
   - the supported cipher suites
   - the client nonce
2. the **TLS server hello** message includes:
   - the selected cipher suite
   - the server certificate
   - the server nonce
3. the client:
   - checks the server certificate and extracts the server's public key
   - generates the pre-master secret (PMS), encrypts it with the server's public key and sends it to the server
4. Using the same key derivation function (as specified by the TLS standard), the client and server independently compute the Master Secret (MS) from the PMS and nonces. The MS is then sliced up to generate the two encryption and two HMAC keys. All messages from now on are encrypted and authenticated.
5. The client sends the HMAC of all the handshake messages
6. The server sends the HMAC of all the handshake messages

7 Network Layer Security: IPsec (8.7)

The **IP security protocol** (**IPsec**) provides datagram-level (so network layer level) encryption and integrity, for both user traffic and control traffic. IPsec is often used to create Virtual Private Networks (VPNs).
With **network-layer confidentiality** between a pair of network entities (for example, between two routers, between two hosts, or between a router and a host), the sending entity encrypts the payloads of all the datagrams it sends to the receiving entity. The encrypted payload could be a TCP segment, a UDP segment, an ICMP message, etc. IPsec doesn't only provide confidentiality, but also **source authentication**, **data integrity** and **replay-attack prevention**.

IPsec has two modes:

- **transport mode**: only the datagram's payload is encrypted and authenticated
- **tunnel mode**: the entire datagram gets encrypted and authenticated, and the encrypted datagram is encapsulated in a new datagram, with a new IP header, tunneled to the destination.

7.1 Virtual Private Networks (VPNs)

An institution that extends over multiple geographical regions often desires its own IP network, so that its hosts and servers can send data to each other in a secure and confidential manner. To achieve this goal, the institution could actually deploy a stand-alone physical network—including routers, links, and a DNS infrastructure—that is completely separate from the public Internet. Such a disjoint network, dedicated to a particular institution, is called a **private network** and can be very costly.

The solution is a **virtual private network** (**VPN**): the institution's inter-office traffic is encrypted and sent over public Internet instead.

7.2 SAs: Security Associations (SAs) (8.7.3)

Before sending IPsec datagrams from source entity to destination entity, the source and destination entities create a network-layer logical connection. This logical connection is called a **security association** (**SA**). An SA is a simplex logical connection; that is, it is unidirectional from source to destination. If both entities want to send secure datagrams to each other, then two SAs (that is, two logical connections) need to be established, one in each direction.

Both sender and receiver maintain *state information* about the SA in its **Security Association Database** (**SAD**), such state information is:

- **Security Parameter Index (SPI)**: a 32 bit identifier for the SA
- origin and destination SA interface
- type of encryption used
- encryption key
- type of integrity check used
- authentication key

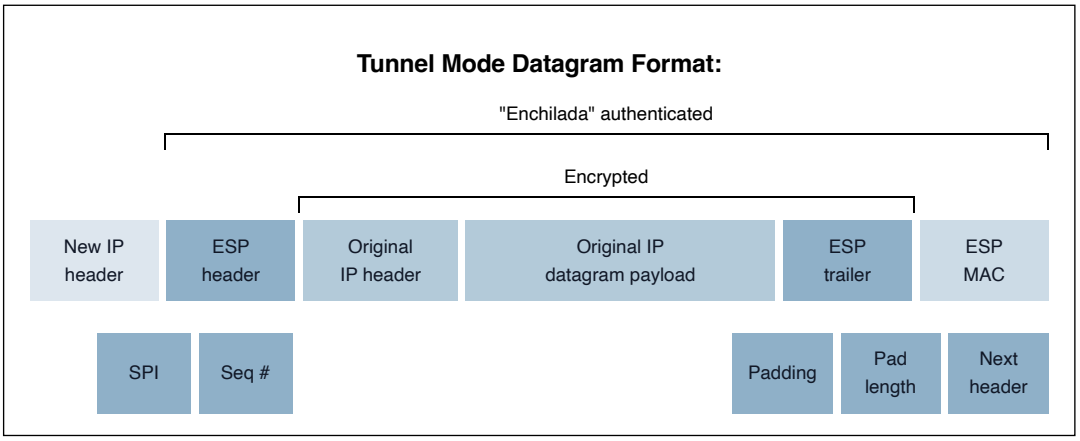
IPsec actually uses another database: the **Security Policy Database** (**SPD**). This database determines the policy to be used: what types of datagrams are to be IPsec processed. So in a way the SPD specifies *what to do* and the SAD specifies *how to do it*.

7.3 IPsec protocols (8.7.2)

In the IPsec protocol suite, there are two principal protocols:

- **authentication header (AH) protocol**: provides source authentication and data integrity but *not* confidentiality
- **Encapsulating Security Payload (ESP)**: provides source authentication, data integrity *and* confidentiality. It is more widely used than AH.

7.3.1 ESP tunnel mode



These are the rules to convert the original IPv4 datagram into an IPsec datagram:

1. Appends the ESP trailer to the original datagram (which includes the original header fields)
2. encrypts the result using the algorithm and the key specified by the SA
3. appends the ESP header to the front of this encrypted quantity, the resulting packet is called the *enchilada*
4. creates an authentication MAC using the algorithm and the key specified by the SA on the whole enchilada
5. appends the MAC to the back of the enchilada, forming the *payload*
6. creates a new IP header: new IP header fields and the addresses to the tunnel endpoint

7.4 IPsec sequence Numbers

For a new SA, the sender initializes the sequence number to 0, each time a datagram is sent over the SA:

- the sender increments the sequence number
- the sender places the value in the sequence number field
  The goal is to prevent the attacker from sniffing and replaying a packet, because the receipt of duplicate authenticated IP packets may disrupt the service.
  To solve this the receiver uses a window (it doesn't keep track of *all* the packets) to check for duplicates.

8 Operational security: firewalls and Intrusion detection systems (8.9)

In a computer network, when traffic entering/leaving a network is security-checked, logged, dropped, or forwarded, it is done by operational devices known as firewalls, intrusion detection systems (IDSs), and intrusion prevention systems (IPSs).

8.1 Firewalls (8.9.1)

A **firewall** is a combination of hardware and software that isolates an organization’s internal network from the Internet at large, allowing some packets to pass and blocking others. A firewall allows a network administrator to control access between the outside world and resources within the administered network by managing the traffic flow to and from these resources. A firewall has three goals: (1) review every packet from outside to inside and vice versa, (2) allow only specific packets to pass—defending the organization's network—and (3) be itself immune to penetration.
Firewalls can be classified in three categories: **traditional packet filters**, **stateful filters**, and **application gateways**.

Here are some limitations of firewalls and gateways:

- The router is susceptible to IP spoofing, it can't know if the data "really" comes from the claimed source.
- If multiple applications need a special treatment, each needs its own application gateway.
- Client software must know how to contact the gateway, for example by setting the IP address of the proxy in the web server
- filters often use an "all or nothing" policy for UDP
- *tradeoff*: the degree of communication with the outside world in exchange for the level of security
- many highly protected sites still suffer from attacks, even using firewalls

8.1.1 Stateless (traditional) packet filters

The internal network gets connected to the Internet via a router **firewall**, which filters **packet-by-packet** and decides to forward/drop a packet based on information in the IP header:

- source IP address, destination IP address
- TCP/UDP source and destination port numbers
- ICMP message type
- TCP SYN and ACK bits

In doing so the firewall uses an **access control list** (**ACL**): a table of rules, applied top to bottom to incoming packets, that looks like the OpenFlow forwarding table.

action	source address	destination address	protocol	source port	destination port	flag bit

Stateful packet filtering solves this by keeping track of every TCP connection, it in fact tracks connection setup (SYN) and teardown (FIN) to determine whether or not a packet makes sense and times out inactive connections, no longer admitting packets from these. Stateful packet filtering needs an augmented ACL, which for every record states if the state of connection must be checked.

8.1.2 Application Gateways

To have finer-level security, firewalls must combine packet filters with application gateways. **Application gateways** look beyond the IP/TCP/UDP headers and make policy decisions based on application data. An application gateway is an application-specific server through which all application data (inbound and outbound) must pass. Multiple application gateways can run on the same host, but each gateway is a separate server with its own processes. An application gateway is a separate host from the firewall. It Sits inline as a proxy/middleman: it terminates TCP connections on both sides (Client - Gateway - Server).

A possible application is: the firewall blocks every telnet connection, except the ones incoming from the application gateway, for authorized users the gateway sets up a telnet connection to the destination host by relaying the data between these two connections.

8.2 Intrusion detection systems

An **intrusion detection system** (**IDS**) performs a deep packet inspection by also looking at the packet contents (for example checking character strings in the packet against a database of known virus and attack strings). It matches packet payloads against attack signatures (viruses, worms, exploits) or statistical traffic anomalies. An organization may use multiple IDSs, each with different types of checking at different locations.
An IDS also examines the correlation among multiple packets to detect port scanning, network mapping, DoS attacks, etc.